HireReady: AI-Powered Career Mentorship Platform

CHAPTER 1. INTRODUCTION

1.1 Project Background

Finding a job is competitive. That has always been true. But over the last two decades, something has changed that makes it much harder for many candidates and it has nothing to do with their qualifications.

The issue is the Applicant Tracking System, commonly known as an ATS. In simple terms, an ATS is software that employers use to automatically read, sort, and filter job applications based on rules the employer sets in advance (Cappelli, 2019). The system scans each submitted resume, checks for certain keywords and formatting patterns, and decides whether to pass it forward, all before any human recruiter even opens the document. According to Jobscan (2025), approximately 98% of Fortune 500 companies now rely on ATS software to manage incoming applications, and adoption among large and mid-sized employers continues to grow. In other words, for the vast majority of jobs people apply for, the first decision is made by a piece of software, not a person.

This creates a serious problem. A large-scale study by Harvard Business School found that millions of genuinely qualified candidates are automatically rejected by ATS software, simply because their resumes do not use the exact words the system is looking for (Fuller et al., 2021). The researchers gave these candidates a name: hidden workers. They estimated that over 27 million workers in the United States fall into this category. The reason is straightforward. An ATS might be set up to search for the phrase "team leadership." If a candidate writes "people management" instead , which means the same thing , the system rejects them. Bogen and Rieke (2018) found that this kind of word-matching problem hits career changers and people from non-traditional backgrounds the hardest, because they are the least likely to know what vocabulary a particular ATS is looking for.

But passing the ATS filter is only half the battle. Once a candidate gets through, they still need to perform well in an interview. Structured interviews are one of the most reliable methods of predicting how a candidate will actually perform in a job (Levashina et al., 2014). The problem is that not everyone gets the same chance to prepare. Professional interview coaching in the UK typically costs between £100 and £200 per hour. Candidates with access to that kind of coaching

arrive knowing exactly how to structure their answers. Those without it — which is most people — often have to rely on watching YouTube videos and hoping for the best. Research by Barrick, Shaffer and DeGrassi (2009) confirmed that this preparation gap shows up in actual interview scores, making an already unequal process even more uneven.

So there are two separate barriers: getting past the automated filter, and then being prepared enough to succeed in a real interview. What makes this worse is that the tools currently available to help with these two problems are completely disconnected from each other. A candidate might use one platform to check their resume, a different one to build it, another one to practise interviews, and yet another to figure out what skills to learn. None of these platforms talk to each other. There is no shared picture of what the candidate needs.

This is what led to the development of HireReady , a free, AI-powered career preparation platform built to tackle both barriers in one place. Instead of jumping between four separate tools, a candidate can use HireReady to analyse their resume, build a better one, practise interview questions, and follow a personalised plan to close their skill gaps ,all within a single application where the data from each feature feeds into the others.

1.2 Problem Statement of Existing Platform

A detailed look at the existing tools available to job seekers (documented fully in Chapter 3) shows four recurring problems that none of them properly solve.

- **Fragmented tools**. Candidates currently have to use Jobscan for resume checking, Rezi or a similar tool for resume building, a separate platform for interview practice, and something else entirely for learning new skills. These platforms do not share any data. Having to switch between multiple disconnected tools adds unnecessary stress to an already difficult process and for many candidates, that extra friction is enough to make them stop trying.
- **High costs**. The most useful features on existing platforms are locked behind paid subscriptions. Jobscan charges around $49.95 per month, Rezi around $29, and Resume Worded around $19. For a final-year student or a graduate who is not yet earning, those are not small amounts. NACE (2023) found that financial cost is one of the biggest reasons students do not access career support.
- **No data connection between tools**. When a resume scanner identifies a missing skill , say, "Docker" that information does not go anywhere. The interview simulator does not know about

it. The learning plan does not know about it. The candidate has to manually note it down and carry it forward themselves. There is no automated flow of information between the preparation stages.

- **No Practical Help to Fix Skill Gaps**. Existing tools are good at identifying problems but not at helping users fix them. Being told your resume is missing "Docker experience" is not very helpful if you have no idea where to start learning Docker. Without a clear path forward, most candidates simply do not know what to do next and many stop engaging with the process entirely.

1.3 Aims and Objectives

The aim of this project is to design, build, and evaluate HireReady , a free, AI-powered career preparation platform that consolidates resume analysis, resume building, interview practice, and skill development into one connected system, where data flows between each feature automatically.

Five objectives were defined at the start of development. They were written to be specific enough to test , vague objectives are easy to claim as complete, which is why every one below has a concrete, verifiable outcome.

- **Authentication and Profile Managemen**t. Build a secure login system where passwords are hashed with bcrypt (salt factor 10) and sessions are managed through JWT tokens in HTTP-only cookies. Users must be able to register, update their profile details, and permanently delete their account along with every record linked to it.
- **AI Resume Scanner**. Develop a resume analysis tool that accepts both PDF and DOCX uploads, extracts the text on the server side, and sends it to the Google Gemini API. The output must include a match score against the user's target job role, a list of missing keywords, detected formatting issues, and at least three actionable improvement suggestions.
- **Resume Builder**. Create a multi-step form-based resume builder with at least two selectable layout templates. It must include an AI summary enhancement tool that rewrites a user's draft summary using Gemini, and produces a clean, downloadable PDF using the browser's native print functionality.
- **Mock Interview Simulator**. This was arguably the most technically complex objective. The simulator must ask one AI-generated question at a time, store the user's typed answers, and once the session ends produce a structured feedback report with three separate scores: overall

performance, technical accuracy, and grammar quality. Each question must also receive individual written feedback.

- **Career Roadmap Engine**. Build a tool that compares the skills a user has listed on their profile against what their target job role typically requires, then generates a four-module sequential learning plan to close that gap. Modules must unlock in order, completing Module 1 before Module 2 becomes available and finishing a module must automatically write the new skills back to the user's profile.

These five objectives map directly to the five main modules of the HireReady platform. Chapter 11 evaluates each one against the finished system with specific evidence from testing.

1.4 Project Scope

What this project includes: User registration and secure login; user profile setup and editing; resume upload in PDF or DOCX format with text extraction handled on the server; AI-powered ATS resume analysis using Google Gemini with a four-model fallback system; resume scan history; a multi-step resume builder with AI summary improvement and PDF export; a configurable mock interview simulator with post-session scoring; voice input and text-to-speech support for accessibility; a sequential career roadmap with module locking and automatic profile updates; a fully responsive dark-mode interface.

What this project does not include: Video or audio interview recording; tools for employers or recruiters; direct connections to corporate ATS systems like Workday or Greenhouse; cloud hosting and deployment; support for languages other than English; Google or GitHub sign-in options.

1.5 Target Users

University students and recent graduates who are entering the job market for the first time and have little experience with ATS systems or structured interview techniques.

Career changers and mid-level professionals who are moving into a new industry and need to understand which skills their target role requires that they do not yet have.

Job seekers with limited budgets, including students in developing countries, first-generation graduates, and anyone for whom a monthly subscription to a career tool would be a genuine financial

strain. For this group, HireReady being completely free is not just a convenience — it is the reason the platform was designed the way it was.

CHAPTER 2: Literature Review

2.1 Applicant Tracking Systems and Automated Recruitment

2.1.1 How ATS Platforms Work and Why They Matter

To understand why a tool like HireReady is needed, it helps to start with the technology that created the problem in the first place.

An Applicant Tracking System (ATS) is software that employers use to collect, filter, and rank job applications automatically before any human recruiter reads them (Cappelli, 2019). In its earliest form in the late 1990s, an ATS was simply a digital filing cabinet: somewhere to store large numbers of incoming applications without drowning in paper. Online job boards changed that. Once any employer could post a job listing and receive hundreds of applications overnight, passive storage was no longer enough. Systems were redesigned to actively evaluate incoming documents, scanning resume text, pulling out keywords, and scoring candidates against employer-set criteria. By the early 2010s, the ATS had shifted from an administrative tool into something closer to a gatekeeper: the thing that decides, before any person sees an application, whether it is worth a recruiter's time. Sanchez-Monedero, Dencik and Edwards (2020) describe this shift as a meaningful change in how power operates in hiring , not just a technical upgrade but a redistribution of decision-making authority away from people and towards algorithms.

According to Jobscan (2025), approximately 98% of Fortune 500 companies now process every incoming application through an ATS. The practical consequence is that the wording of a resume , not just the qualifications behind it — determines whether a candidate gets a chance.

2.1.2 The Hidden Worker Problem

The clearest academic account of what happens when ATS filtering goes wrong comes from Fuller et al. (2021), writing for Harvard Business School. Their research introduced the term hidden workers to describe people who are capable of doing a job but are automatically filtered out before any recruiter

sees them. They estimated that over 27 million workers in the United States fall into this category. The reason is not that these candidates lack skills , it is that the ATS cannot recognise the skills they have.

The mechanism is straightforward. A recruiter sets up a keyword list , "team leadership," "Agile methodology," "Docker." The ATS searches incoming resumes for those exact phrases. A candidate who writes "people management" instead of "team leadership" gets rejected automatically, even though the two phrases describe the same thing. There is no understanding happening just a string match that either succeeds or fails. Rieke and Bogen (2018) found this kind of rigid word-matching most often affects career changers and people from non-traditional educational backgrounds, because they are the least likely to have been coached in the precise vocabulary of their target industry.

This is the problem that HireReady's ATS scanner is built to address. Rather than doing its own keyword matching, the scanner uses a large language model to evaluate a resume semantically , asking whether the candidate demonstrates the underlying competencies, not just whether they used the right words

2.1.3 Resume Formatting and Structural Parsing

Beyond keyword content, ATS platforms also struggle with resume formatting. When a resume uses complex layouts ,multiple columns, text boxes, embedded tables, graphics, or unusual fonts ,the software that tries to extract the text often fails or produces garbled output. The result is an artificially low match score, even if the resume's content is strong. This is a known problem with widely used ATS platforms including Taleo, Workday, and Greenhouse.

This finding has a direct design implication. A resume builder intended to help candidates succeed in ATS environments should produce documents in single-column, plain-text-friendly formats by default. The HireReady Resume Builder does exactly that , its templates are designed to be ATS-safe, not just visually appealing.

2.2 Natural Language Processing and Artificial Intelligence in Recruitment

2.2.1 Three Generations of AI Hiring Tools

AI has been applied to recruitment problems for over twenty years, but not all AI tools are equivalent. Black and van Esch (2020) provide a useful overview of the progression. The earliest generation, covering roughly 2000 to 2015, used rule-based systems , keyword dictionaries and Boolean logic.

These were rigid and required constant manual updates. The second generation, from around 2015, used machine learning models trained on historical hiring data. These were more flexible, but they inherited a serious problem: if the historical data reflected biased past decisions favouring candidates from certain universities or demographic backgrounds , the trained model would learn to replicate those biases. Raghavan et al. (2020) audited a range of these second-generation tools and found that the majority lacked transparency; candidates received decisions with no explanation of what factors contributed.

The third generation, emerging from around 2022, uses large language models (LLMs). These are qualitatively different: instead of matching tokens, they process meaning in context. This is the capability HireReady relies on.

2.2.2 Semantic Analysis versus Lexical Matching

The difference between semantic understanding and keyword matching matters practically. Keyword matching finds "Docker" or it does not. Semantic analysis asks whether the candidate's experience suggests familiarity with containerisation, even if the word "Docker" never appears.

Devlin et al. (2019) introduced BERT, the transformer-based model that established the modern era of contextual natural language processing. BERT demonstrated that an embedding-based model could determine that "led a cross-functional engineering team" and "managed interdepartmental collaboration" convey essentially the same professional claim, despite sharing no content words. That capability is what makes semantic resume analysis possible.

HireReady's prompts are designed to use this capability , instructing the Gemini model to evaluate whether the candidate demonstrates the competencies required for their target role, not just whether they used the right vocabulary.

2.3 AI-Powered Tutoring and Career Mentoring

2.3.1 Can AI Replace Human Tutoring?

The question of whether AI can provide meaningful tutoring not just information, but actual learning guidance has been studied extensively under the field of Intelligent Tutoring Systems (ITS). VanLehn (2011) conducted a comprehensive meta-analysis comparing learning outcomes across four conditions: one-on-one human tutoring, ITS-based instruction, traditional classroom teaching, and

unassisted self-study. The headline finding was that well-designed ITS produced approximately 76% of the learning improvement achieved by one-on-one human tutoring. That is a significant result. The key variable was not whether the tutor was human or software it was whether the feedback was specific, timely, and tied to the learner's actual responses.

This finding directly informs the design of HireReady's interview simulator. The system does not simply record answers and produce a summary score. It evaluates each answer individually, identifies specific weaknesses, and provides a model answer for comparison. That specificity is, according to the ITS literature, what makes feedback effective.

2.3.2 What Actually Improves Interview Performance

Two things consistently appear in the interview preparation research as the strongest predictors of improvement: structured practice using realistic scenarios, and feedback specific enough to act on.

Huffcutt and Arthur (1994) established this in an early meta-analysis of structured interview research, and Levashina et al. (2014) confirmed it in a more recent and comprehensive review. Candidates who practised repeatedly with structured, specific feedback improved measurably their answers became better organised, technically more precise, and more fluent under pressure. Candidates who prepared without feedback did not show the same gains.

The access problem is that this type of preparation is not equally available. Barrick, Shaffer and DeGrassi (2009) found that interview readiness varies significantly across socio-economic groups, largely because of unequal access to professional coaching and career services. HireReady's interview simulator is an attempt to give every user access to the functional equivalent of a structured practice partner, something the research suggests is worth having.

2.3.3 Personalized Learning Pathways

The career roadmap feature draws on a different body of literature, research on personalised learning. Pane et al. (2017) studied personalised learning programmes across a large number of schools in the United States and found that students following individually tailored learning plans achieved significantly greater gains than those following standardised curricula, particularly in subjects such as computing where skills build on each other in structured sequences. Learning Python is more productive after you understand basic programming logic. Learning Docker makes

more sense after you are comfortable with Linux. Sequential, prerequisite-aware learning paths reflect the actual structure of technical knowledge.

This is the logic behind the HireReady roadmap engine. It generates a four-module learning plan based on the gap between the user's current skills and the requirements of their target role, with each module unlocking only after the previous one is completed. The structure is not arbitrary — it reflects the pedagogical evidence about how technical skills are most efficiently acquired.

2.4 Large Language Models and Google Gemini

2.4.1 The Transformer Architecture

Modern LLMs rely on the Transformer architecture introduced by Vaswani et al. (2017). By replacing recurrent neural networks with self-attention mechanisms, Transformers process input tokens simultaneously. This allows models to capture long-range linguistic dependencies and efficiently scale pre-trained architectures such as BERT (Devlin et al., 2019), GPT models (Brown et al., 2020), and Google Gemini.

2.4.2 Google Gemini Capabilities and API Design

Modern large language models all trace their design back to one paper: Vaswani et al. (2017), "Attention Is All You Need." The core innovation was the self-attention mechanism, which allows a model to evaluate every word in a sentence in relation to every other word at the same time , rather than processing words one by one in sequence. This made it possible to capture long-range relationships in text (understanding that "it" refers to something mentioned several sentences earlier, for instance) and, crucially, made the training process far more parallelisable, enabling much larger models to be built.

BERT (Devlin et al., 2019) and GPT-3 (Brown et al., 2020) both built directly on this architecture. The Gemini family of models, which HireReady uses, is a later development in the same lineage.

2.4.2 Google Gemini and the HireReady Integration

Google Gemini is a family of large language models developed by Google DeepMind, capable of processing and generating text, code, and structured data (Google DeepMind, 2024). HireReady uses the text generation capabilities of Gemini Flash model variants across four tasks: ATS resume

analysis, professional summary enhancement, interview question generation, and interview transcript evaluation.

Two specific API features matter for this project. The first is structured output: by specifying responseMimeType: "application/json" alongside a JSON schema definition in the API request, the model returns structured JSON rather than free prose , making it possible to insert AI-generated reports directly into the database without manual parsing. The second is model availability across multiple tiers. HireReady implements a four-model sequential fallback chain: if the primary model returns a rate-limit error, the system automatically retries with the next model in the list. This significantly improves reliability during periods of high API traffic, a real concern when using a shared API without guaranteed throughput.

Large language models occasionally produce output that does not match their specified format , a known property of instruction-following models (Wei et al., 2022). HireReady handles this with a double-pass JSON parser: it first attempts standard JSON parsing, and if that fails, applies a regex-based recovery routine to extract the structured data from within markdown code blocks or surrounding text.

2.4.4 Prompt Engineering for Structured Outputs

Prompt engineering ,designing the instructions given to a language model to get consistent, useful outputs , has become a recognised discipline in applied AI work. White et al. (2023) published a systematic catalogue of prompt design patterns, finding that prompts which assign the model a specific expert role, define an exact output format, and set explicit constraints on scope and length produce significantly more reliable outputs than open-ended instructions.

HireReady applies these principles across all four AI integration points. Every prompt assigns a specific expert persona to the model ,for example, "You are a senior technical interviewer with fifteen years of experience" , specifies a JSON schema for the response, and includes explicit constraints on question count, word count, and topic scope. During development, switching from open-ended prompts to schema-constrained prompts eliminated most of the JSON parsing failures that had been causing errors in early testing.

2.5 Web Development Technologies

2.5.1 Next.js App Router Architecture

Next.js is a React-based web framework developed and maintained by Vercel (Vercel, 2024). The version used in this project , Next.js 15 with the App Router uses React Server Components (RSC) as its default rendering model. A Server Component runs entirely on the server and sends only HTML to the browser; no JavaScript from that component is included in the client bundle.

This has a direct security benefit for a project that handles API keys and database credentials. Server Components can query the database and call external APIs including the Gemini API without those credentials ever travelling to the browser. There is no way for a user to extract the Gemini API key by inspecting browser network traffic, because the key only ever exists in the server environment (Vercel, 2024).

2.5.2 Server Actions for Secure AI Processing

Next.js Server Actions allow a client component to invoke a server-side function that behaves like a normal function call from the developer's perspective but executes on the server. In HireReady, all AI processing, database writes, and session validation happen through Server Actions. This means there are no exposed REST API endpoints for the AI features an attacker cannot probe or enumerate the AI processing routes, because those routes do not exist in the traditional sense.

2.5.3 React 19 Concurrent Features

React 19 introduced the useTransition hook, which marks a state update as non-urgent and allows the interface to remain interactive while a slow operation completes (React Team, 2024). In HireReady's interview simulator, where the user submits an answer and waits several seconds for the AI to generate the next question, this hook prevents the interface from appearing frozen during the wait. Without it, every answer submission would lock the UI until the server responded.

2.6 Database and Object-Relational Mapping Technologies

2.6.1 PostgreSQL Database Selection

PostgreSQL is an open-source relational database management system with strong support for ACID transactions, enforced foreign key constraints, and cascade delete operations (PostgreSQL Global Development Group, 2024). HireReady's data model has deep relational structure: a single user record owns a profile, a set of resume scans, a set of interview sessions (each with many chat messages and a performance report), one or more built resumes, and a set of roadmap modules. A relational database with enforced referential integrity is the right tool for this structure. Cascade

deletion , the feature that removes all related records when a user account is deleted relies on these constraints being reliable at the database level, not just at the application level.

2.6.2 Prisma ORM Integration

Prisma is a TypeScript-first object-relational mapper that generates a type-safe query client from a declarative schema definition (Prisma, 2024). For a solo developer working without a formal test suite, the compile-time type checking Prisma provides is practically useful: incorrect field names, type mismatches, and missing required fields are caught during compilation rather than discovered during manual testing. Prisma also generates and applies database migrations automatically from schema changes, which kept the development workflow straightforward across multiple iterations of the data model.

2.7 Agile Development Methodologies

2.7.1 DSDM and Time Boxing

The development methodology used for this project is DSDM , Dynamic Systems Development Method. DSDM is an agile framework built around a fixed-time, variable-scope approach: the deadline is treated as a hard constraint, and the feature set is adjusted to fit within it rather than the other way around (Stapleton, 2003). This makes it particularly well-suited to academic software engineering projects, where submission dates cannot be negotiated.

DSDM structures work into timeboxes , fixed-duration periods, each with a defined set of deliverables. This project used four timeboxes across the academic year. At the start of each timebox, requirements were categorised using the MoSCoW framework: Must-Have features that were non-negotiable; Should-Have features that were important but could be compressed; Could-Have features included only if time permitted; and Won't-Have features explicitly deferred. When development on the JSON parser took longer than planned during Timebox 2, the MoSCoW hierarchy made it immediately clear which Could-Have features could be moved to Timebox 3 without endangering the minimum viable system (Abrahamsson et al., 2002).

2.7.2 Timeboxing and MoSCoW Prioritization

The choice of DSDM over Scrum was deliberate. Scrum is well-suited to team-based development with regular retrospectives and stakeholder reviews , it is a collaborative framework. For a solo developer on a fixed academic timeline, DSDM's more structured approach to priority management is more appropriate. It forces explicit decisions about what to build first, which is easier to justify to an examiner than informal sprint-by-sprint scope adjustments.

2.8 Chapter Summary

The literature reviewed in this chapter shaped the design of HireReady in concrete ways. ATS research confirmed that qualified candidates are excluded through keyword mismatch and formatting failures , driving the semantic scanner and ATS-safe resume builder. NLP research showed that transformer-based models can understand meaning rather than just match words , justifying the use of Gemini over a simpler keyword-counting approach. ITS and interview preparation research found that specific, question-level feedback drives measurable improvement shaping the per-question grading in the interview simulator. Personalised learning research established the value of sequential, prerequisite-aware pathways , informing the module-locking logic in the roadmap engine. The ethical literature on AI in recruitment made it clear that any system in this space must be transparent and candidate-centred , with commitments built into the platform's design. And the technical literature on Next.js, Prisma, and PostgreSQL justified the specific architectural choices made in the implementation.

CHAPTER 3: COMPARATIVE ANALYSIS OF EXISTING SYSTEMS

3.1 Overview of Existing Platforms

Three platforms dominate the AI-assisted career preparation market: Jobscan, Rezi, and Resume Worded. Before a single line of HireReady was written, each of these was tested and reviewed in detail , not to copy them, but to understand exactly what they do not do. That gap analysis is what this chapter documents.

3.2 Jobscan

Jobscan has been around long enough that it is often the first tool career advisors recommend to students. Its core function is simple: paste your resume and a job description into the platform, and it returns a percentage match score alongside a list of keywords your resume is missing. It also flags formatting issues like tables, text boxes, and unusual fonts that cause ATS parsers to fail.

That is genuinely useful. For candidates who just want to know if their resume will pass an automated filter, it does the job reasonably well.

Jobscan also includes a resume builder ,users can create a resume from scratch, import from LinkedIn, or reformat an existing one. It is a more recent addition to the platform and is generally free to use. However, it is more basic than a dedicated builder like Rezi. There is still no interview practice feature, and if the scan tells you that "Docker" and "Kubernetes" are missing from your resume, Jobscan does not tell you how to actually fix that.

Pricing: Free users get five scans per month. Full access is $49.95 per month (Jobscan, 2025).

3.3 Rezi

Rezi takes a different angle. Rather than scanning existing resumes, it helps users build new ones from scratch. Its templates are structured to be ATS-safe by default , single-column, clean fonts, standard section headings and its AI writing tools can generate bullet points and professional summaries based on a user's job history and target role.

Rezi also has an AI Interview Practice feature, which is worth noting because it sets it apart from the other two. Users can practise answering role-specific questions and receive feedback. But the feedback focuses mainly on delivery ,speaking pace, filler words, tone. It does not evaluate whether answers were technically correct. And there is no graded performance report with scores. That matters, because research on interview preparation shows it is content-focused feedback that actually drives improvement ,not just delivery coaching (Huffcutt and Arthur, 1994).

Pricing: The free tier allows one resume and one interview session. Pro is $29 per month. There is also a lifetime option for $149 (Rezi, 2025).

One more gap: Rezi does not calculate an ATS keyword match score against a specific job description. It checks formatting, but not how well your resume matches what a particular employer is actually looking for.

3.4 Resume Worded

Resume Worded is probably the most feedback-heavy of the three. Upload a resume, and the platform scores it out of 100 across several dimensions , impact, brevity, style, section quality, skills.

It also reviews LinkedIn profiles. The feedback is specific and line-by-line, which is more useful than a single aggregate number.

What it cannot do: there is no resume builder, so users have to edit their own document and re-upload every time they make a change. There is no interview feature. And there is no career development guidance.

Pricing: Free access is limited. The Pro plan is $49 per month, or $229 if paid annually , which works out to about $19 per month (Resume Worded, 2025).

3.2 Feature-by-Feature Comparison

Feature	Jobscan	Rezi	Resume Worded	HireReady
ATS Resume Analysis	Full	Full	Partial	Full
Resume Match Score (%)	Full	No	No	Full
AI Resume Builder	Full	Full	No	Full
Interactive Mock	No	Yes	No	Full

Feature	Jobscan	Rezi	Resume Worded	HireReady
ATS Resume Analysis	Full	Full	Partial	Full
Resume Match Score (%)	Full	No	No	Full
Interview				
AI Interview Grading	No	No	No	Full
Career Skill Roadmap	No	No	No	Full
DOCX & PDF File Upload	Full	Full	Full	Full
Free Unlimited Access	Paid	Paid	Paid	Free

3.3 Identification of System Drawbacks and Gaps

We found 4 major gaps in current platforms:

1. **Paywalled Interview Practice:** While these platforms do offer mock interview tools, they are strictly locked behind premium subscriptions. Free users cannot access interactive interview practice with AI feedback.
2. **No Skill Gap Roadmap:** When these tools find missing skills in a resume (like Docker or React), they do not provide a step-by-step learning path to help the user learn those skills.
3. **High Financial Barrier:** To get the full experience (resume scanning, resume building, and mock interviews), users have to pay expensive monthly fees (ranging from $19 to $50 per month), which is a huge burden for students and job seekers.
4. **No Backup System for AI Downtime:** Commercial platforms rely on a single AI system. If their AI server goes down or hits a traffic limit, the system stops working. Users do not know what happens behind the scenes.

3.4 The HireReady Solution

HireReady solves these gaps by offering a completely free and reliable platform:

- **100% Free Mock Interviews:** HireReady provides turn-based AI mock interviews with real-time feedback on communication and technical skills without any paywalls.
- **Automated Learning Roadmaps:** Instead of just pointing out missing skills, HireReady generates a 4-module step-by-step learning path so candidates can actually improve.
- **Free All-in-One Tool:** Users get resume scanning, resume building, and interview practice in one single platform completely for free.
- **Smart AI Fallback Engine:** HireReady uses a smart backup system (gemini-3.5-flash -> gemini-2.0-flash). If one AI model fails or hits limits, the system instantly switches to a backup model, ensuring the app never crashes.

For a final-year student or someone who has just graduated and is not yet earning, nearly $130 a month across multiple subscriptions is not realistic. NACE (2023) found that financial cost is one of the main reasons students do not access career preparation tools.

HireReady is completely free. No subscription, no usage cap, no feature gating.

CHAPTER 4: LEGAL, ETHICAL, SOCIAL, AND PROFESSIONAL ISSUES

HireReady is an AI career tool. Because of this, it handles sensitive personal data. This data includes user resumes, career histories, and chat performance. Building a system like this takes care. We must think about legal, ethical, social, and professional duties. This chapter explains how we handle these issues. We do this to keep the app safe, fair, and responsible.

4.1 Legal Issues

Data privacy is our main legal concern. Users upload detailed resumes. These files hold personal details like names, emails, phone numbers, and past jobs. Users also type answers in AI interview chats. The database stores these answers. Under UK and global privacy laws, this is known as processing personal data. Handling this data brings strict legal rules (ICO, 2024).

The platform follows the UKDP Act 2018. It also follows the General Data Protection Regulation (GDPR). We use several safeguards to meet these laws:

4.1.1 Data Minimisation (GDPR Article 5(1)(c))

The GDPR has a rule about collecting data. Organisations must only gather what they truly need for a specific task (Voigt and Von dem Bussche, 2017). HireReady strictly follows this rule.

- Required Data Only: The system only asks for data it needs to work. This includes profile details like name and email. It also includes the target job, skills, and chat transcripts. It stores the uploaded resume filenames and AI reports too.
- No Sensitive Data: We do not collect things like home addresses. We do not ask for dates of birth or national ID numbers.
- In-Memory File Processing: Uploaded resume files are not saved on the server forever. Instead, they are processed in-memory as short-lived Node.js Buffer objects. After the text is read and analysed, the file data gets deleted. The database only saves the filename and the AI report. This method greatly lowers the amount of personal data we keep (ICO, 2024).

4.1.2 Secure Storage and Encryption

We protect all sensitive data. We use strong, modern security tools.

- Password Encryption: We never save plain text passwords. We hash them using the bcrypt algorithm. We use a salt factor of 10 rounds before saving them in the database. Because of this, no one could recover the original passwords even if they hacked the database (Provos and Mazières, 1999).
- Session Security: User sessions use JSON Web Tokens (JWT). These tokens are stored in HTTP-only cookies. We enable the sameSite: lax and secure flags. The httpOnly setting stops client-side scripts from reading the token. This protects the app from Cross-Site Scripting (XSS) attacks (OWASP, 2021).
- Transport Security: All data sent between the browser and the server is encrypted. We use SSL/TLS protocols. This stops people from stealing data while it travels over the internet.

4.1.3 User Control and Data Erasure (GDPR Article 17)

The GDPR gives people the "right to erasure." Many people call this the right to be forgotten (Voigt and Von dem Bussche, 2017). HireReady fully supports this rule in two ways:

- Account-Level Cascade Deletion: A user can delete their account. If they do, the system starts a cascade deletion in the PostgreSQL database. This removes the main User record. It also permanently deletes all connected data. This means profiles, resumes, reports, chat messages, and career roadmaps are wiped out right away.
- Granular Control: Users do not have to delete their whole account. They can delete just one resume scan or a single interview record. This gives them tight control over their own data.

4.2 Ethical Issues

We use AI models like Google Gemini. They evaluate resumes and grade interview answers. Using AI brings up big ethical questions. We must think about fairness, clear rules, and hidden bias.

4.2.1 Mitigating AI Bias

AI language models learn from huge amounts of internet data. This data can carry old hiring biases. For instance, the data might favour certain schools, genders, or writing styles (Bogen and Rieke, 2018). If we do not check this, the AI might judge people unfairly.

HireReady tries to lower this risk. We design our API prompts carefully. We tell the Gemini API to focus only on clear, measurable facts:

- Keyword matches for the target job role.
- The presence or absence of specific technical skills.
- The layout and formatting quality of the document.
- Clear grammar and good communication.

Our prompts tell the AI not to judge a person's background. It must ignore personality and demographic traits. We know this does not remove bias completely. Bias can hide deep inside the model's training data (Raghavan et al., 2020). Still, this is a strong effort to make the AI focus only on skills.

4.2.2 Transparency of Evaluation Metrics

Users should know how they are judged. HireReady never gives a score without a reason. Every single evaluation comes with a detailed, written explanation. This makes the reasoning clear.

- ATS Scanner Feedback: Users see exactly which keywords they missed. They see what formatting errors were found. The system gives specific tips to improve.
- Interview Feedback: Users get a detailed breakdown for every single question. They see their answer, where they messed up, and a model answer to study.
- This openness helps users learn. They are not left confused by a random number. Clear algorithms help build user trust and allow human oversight (Sanchez-Monedero, Dencik and Edwards, 2020).

4.2.3 Candidate-Side AI Ethics

There is a major ethical difference here. HireReady uses AI to help job seekers, not to block them. Many company tools use algorithms to reject people. HireReady is different. It stays entirely on the candidate's side. The system helps users fix their applications. It never makes a hiring decision for them. This candidate-focused design avoids the worst ethical issues in AI hiring, where computers can ruin a person's livelihood (Bogen and Rieke, 2018).

4.3 Social Issues

Automated hiring systems can cause stress and unfairness. Computers often reject candidates before a human even sees their resume. Many applicants never find out why they failed (Fuller et al., 2021).

4.3.1 Reducing Social Inequality

The digital divide is a massive problem in career prep. Wealthy job seekers can pay for private coaches. These coaches can cost $100 to $300 a session. They can buy premium resume services and interview training. On the other hand, low-income students and people in developing nations cannot afford these things (NACE, 2023).

HireReady aims to shrink this gap. The platform is completely free. There are no paywalls. There are no subscription levels or usage limits. Every core feature is open to everyone. This means a first-generation college student gets the exact same AI help as someone paying for a premium service.

4.3.2 Accessibility Features

We want the platform to be inclusive. The system uses built-in browser tools to help with this.

Speech-to-Text Input: Users can dictate their interview answers. They just use a microphone instead of typing.

Text-to-Speech Narration: The AI can read the interview questions out loud.

These tools help many people. They improve access for users with visual problems or motor disabilities. They also help people who just prefer talking over typing.

4.3.3 Avoiding Over-Reliance on AI

The AI gives great advice. However, users should not treat it as a perfect expert. AI models can sometimes give wrong, confusing, or very generic answers. Researchers call this "hallucination" (Ji et al., 2023).

HireReady makes this clear. The interface presents the app as a practice tool. It is not a perfect replacement for a real human. We encourage users to start with the AI feedback. Then, they should go ask human mentors, advisors, and friends for more help.

4.4 Professional Issues

As a software engineering student, the developer has professional duties. They must write safe code and protect user secrets. They also need to run ethical tests. These duties match the British Computer Society (BCS) Code of Conduct. The code says members must act with integrity, skill, and care for the public (BCS, 2022).

4.4.1 Secure Software Engineering Practices

Security is built right into the code.

- Server-Side Execution: The app uses Next.js Server Actions. This means all AI chats and database queries run on the server. Because of this, private secrets stay hidden. The Gemini API key, JWT secret, and database string never touch the browser. Even if a user reads the client-side code, they will not find these secrets (OWASP, 2021).
- SQL Injection Prevention: All database queries go through the Prisma ORM. Prisma uses parameterised queries by default. This blocks bad actors from messing with the database inputs (OWASP, 2021).

4.4.2 User Confidentiality

User data is treated with extreme care during development.

- No real user data or resumes ever go into public code repositories.
- The .env file holds API keys and passwords. We put this file in .gitignore so it never gets published online.
- The development database uses fake test data instead of real personal details.

4.4.3 Ethical User Acceptance Testing (UAT)

The developer tested the app with peers and classmates. This testing followed strict ethical rules.

- Voluntary Participation: Taking part was always a choice. Testers knew exactly what the test was for.
- Data Safeguards: We told testers not to upload real sensitive data. We asked them to leave out home addresses, ID numbers, and bank details.
- Post-Testing Cleanup: All test accounts were easy to delete. We permanently wiped their data from the database after testing. This made sure no tester's data stayed around longer than needed.

4.5 Chapter Summary

This chapter looked at the legal, ethical, social, and professional duties of making an AI career platform. Legally, HireReady follows the UK Data Protection Act 2018 and the GDPR. It does this through data minimisation, secure passwords, in-memory processing, and full cascade deletion. Ethically, the system fights AI bias with strict prompts and gives very clear feedback. Socially, the app fights inequality. It is totally free and includes accessibility tools. Professionally, the developer writes secure code and protects privacy. All testing matches the BCS Code of Conduct.

CHAPTER 5: REQUIREMENTS ANALYSIS AND SPECIFICATION

5.1 Chapter Introduction

This chapter is all about nailing down exactly what the HireReady platform has to do. We'll list out the core features and also set the technical standards the system has to hit for the project to actually be a success.

When you're trying to build a full-stack, AI-powered system on a tight university schedule, you really need to set clear boundaries. If you don't have solid rules, the project is basically guaranteed to suffer

from scope creep. That's when people just keep tossing in unplanned features, which eventually causes huge delays and drags down the quality of the final product (Stapleton, 2003).

So, to keep the project's scope under control, I pulled main requirements straight out of the core problems already talked about in Chapter 1. These rules are going to serve as a direct blueprint when we get to the database and system design in Chapter 6. They keep everything stable, secure, and fully focused on actually giving the end user some real value.

5.2 Functional Requirements (MoSCoW Prioritization)

I decided to use the DSDM (Dynamic Systems Development Method) framework to manage things. Under this setup, features are grouped using MoSCoW prioritization. This guarantees that the absolute core parts get built on time. If time starts running out, we can easily delay or tweak the less critical stuff (Abrahamsson et al., 2002).

5.2.1 Must-Have Requirements (Core Features)

These features are non-negotiable. Without them, the app simply wouldn't work.

Req ID	Requirement	Description
FR-01	User Registration and Login	Users have to be able to make an account. They'll enter a name, email, and a password, and then securely log in to access the app.
FR-02	User Profile Setup	Users need to set up a profile that shows their target job, their current experience level, and the skills they already have.
FR-03	Resume Upload (PDF and DOCX)	The system has to let users upload a resume file ,specifically in PDF or Word (.docx) formats. After uploading, it needs to safely pull the text out of the file.
FR-04	AI Resume Scanning (ATS Analysis)	The app is going to use the Google Gemini API to read through the uploaded resume. It'll compare the text against the user's target job, calculate a match score,

		flag missing keywords, point out bad formatting, and offer some AI-generated tips to fix it.
FR-05	AI Mock Interview Simulator	The system basically acts as a virtual hiring manager. It asks professional interview questions one by one, matching the user's chosen job and difficulty level. The whole chat gets saved to the database.
FR-06	Interview Performance Feedback	Once the interview wraps up, the system grades the entire chat. It spits out an overall score, a grammar rating, and a technical score, plus a detailed written review with model answers to study.
FR-07	Secure Session Management	Keeping users logged in safely is a must. The system relies on HTTP-only session cookies and JWT tokens. If someone isn't logged in, they get blocked from seeing any of the private pages.

5.2.2 Should-Have Requirements

These are really great features that add a ton of value, but if time gets too tight, we can afford to push them back.

Req ID	Requirement	Description
FR-08	AI Resume Builder	The app should give users a step-by-step form wizard to help them build a brand new resume right from scratch.
FR-09	AI Summary Enhancer	Using the Gemini API, the system should be able to take a user's rough draft of a professional summary and rewrite it so it sounds polished and targeted.
FR-10	AI Career Skill Roadmap	The system should look at the user's current skills versus their target job, and then generate a 4-module

		learning plan. This basically tells them exactly what they need to learn next.
FR-11	Resume Scan History	The database should keep a record of all past resume scans so users can go back and check their old reports whenever they want.
FR-12	Account Settings and Deletion	Users should have the ability to change their name or email. They also need an option to permanently wipe their account and all the data tied to it.

5.2.3 Could-Have Requirements

These are nice little bonuses. They'd make the app better, but we'll only tackle them if the main features are finished way ahead of schedule.

Req ID	Requirement	Description
FR-13	Resume PDF Export	It would be nice if users could just download their newly built resume as a clean PDF.
FR-14	Interview Customisation	We could let users pick exactly how many questions they want and set the difficulty level themselves.
FR-15	Sequential Roadmap Locking	The career roadmap might force users to actually finish Module 1 before it lets them unlock Module 2.
FR-16	Speech-to-Text and Text-to-Speech	The interview simulator could hook into the browser's microphone so users can just talk instead of type, and it could even read the AI's questions out loud.

5.2.4 Won't-Have Requirements (Excluded from this Project)

These features might be cool down the road, but right now they are 100% out of scope. There's just no time to build them.

Req ID	Requirement	Description
FR-17	Video or Voice Mock Interviews	The system will not use webcams or video feeds. While voice dictation is supported, the actual interview grading relies strictly on analyzing text, avoiding complex audio emotion analysis.
FR-18	Direct Job Applications	HireReady is strictly a practice tool. It's not going to hook into LinkedIn and apply to jobs for people.
FR-19	Multi-Language Support	The app is English-only for this version. Translation features just aren't making the cut.

5.3 Non-Functional Requirements (System Quality)

These are the underlying technical rules. They exist to make sure the app feels fast, safe, and doesn't break.

5.3.1 Speed and Performance

Req ID	Requirement	Target
NFR-01	Fast Page Loading	Pages need to load in under 1.5 seconds on a normal internet connection.
NFR-02	Responsive AI Feedback	AI takes a few seconds to "think." To handle this, the app has to show a loading spinner within 2 seconds so the user doesn't think the screen froze.
NFR-03	Concurrent User Capacity	The server has to be able to handle up to 50 people using the app at the same time without crashing.

5.3.2 Security and Data Protection

Req ID	Requirement	Target

NFR-04	Password Encryption	We have to hash all passwords using bcrypt with a salt factor of 10. Saving plain text passwords is a huge no-go (Provos and Mazières, 1999).
NFR-05	Encrypted Data Transmission	Any data moving between the user's browser and our server has to be locked down with SSL/TLS encryption (OWASP, 2021).
NFR-06	Hidden API Credentials	Secret keys like the Gemini API key and the database password ,have to stay hidden in server-side files. The browser should never get to see them.

5.3.3 Usability and Design

Req ID	Requirement	Target
NFR-07	Responsive Design	The website has to look good no matter the screen size, whether it's a mobile phone or a big desktop monitor.
NFR-08	Simple Navigation	A user should be able to fire up an interview or scan their resume in 3 clicks or less from the dashboard.

5.3.4 Reliability

Req ID	Requirement	Target
NFR-09	System Uptime	The website needs to stay online and work properly about 99.5% of the time.

NFR-10	Graceful Error Handling	If the internet drops or the AI messes up, the app shouldn't just crash. It needs to show a friendly error message and make sure the user's data is safe.

5.4 System Scope and Boundaries

5.4.1 Project Constraints

- Single Developer: Because one student is building the whole thing during a short term, the code has to stay clean and simple. We're actively avoiding overly complex architectures.
- API Limits: HireReady is completely tied to Google Gemini. If Google's servers go down or they block free requests, the app goes down with it.
- Text Files Only: The system only reads text out of digital PDFs and Word files. It can't read photos of resumes or handwritten stuff.

5.4.2 Project Scope Matrix

This table defines exactly what the HireReady application will and will not do.

Included in Project Scope	Excluded from Project Scope
Reading resume text and finding missing skills.	Storing massive files directly in the database (uses file links instead).
Creating an interactive AI text chat for interviews.	Using real video, audio, or human-to-human interaction.
Grading user answers and giving scores.	Providing legal guarantees that the user will get a job.
Saving user profiles and history securely.	Charging money or adding payment processing systems.

CHAPTER 6: OVERALL SYSTEM DESIGN AND ARCHITECTURE

6.1 Chapter Introduction

This chapter is all about how the HireReady platform is actually put together behind the scenes. We'll look at the architectural patterns, the structural design, and exactly how the database is laid out.

The system itself is built as a unified full-stack application. It connects a modern React frontend directly to a secure AI processing layer, and saves everything in a PostgreSQL database. Every single design choice you see here came straight from the strict requirements we laid out in Chapter 5.

6.1 System Architecture

For this project, we decided to use an industry-standard Three-Tier (Layered) Architecture, and we built the whole thing inside the Next.js framework. This setup creates a strict separation of concerns. Basically, it means each layer has one specific job to do and it doesn't mess with the other layers. This makes fixing bugs way easier later on, and it helps if we ever need to add new features (Sommerville, 2016).

- Presentation Layer (Frontend Tier): This is what the user actually sees on their screen. We built it using React, Tailwind CSS, and components from shadcn/ui. This layer handles all the user inputs—like uploading a resume file, typing in chat, or changing profile settings. It's responsible for showing the real-time AI feedback and dynamic progress bars.
- Application Logic Layer (Backend Tier): This is the brains of the operation. It runs securely on the server using Next.js Server Actions inside a Node.js environment. It handles calling the Gemini API, calculating the interview scores, and authenticating users securely.
- Data Layer (Storage): This layer saves all the permanent data. We chose a PostgreSQL relational database and manage it using the Prisma ORM. We picked PostgreSQL specifically because it's incredibly reliable (ACID compliant) and it natively supports JSON columns. That JSON support is a huge lifesaver for storing complex things like nested work experience and education arrays (PostgreSQL Global Development Group, 2024).

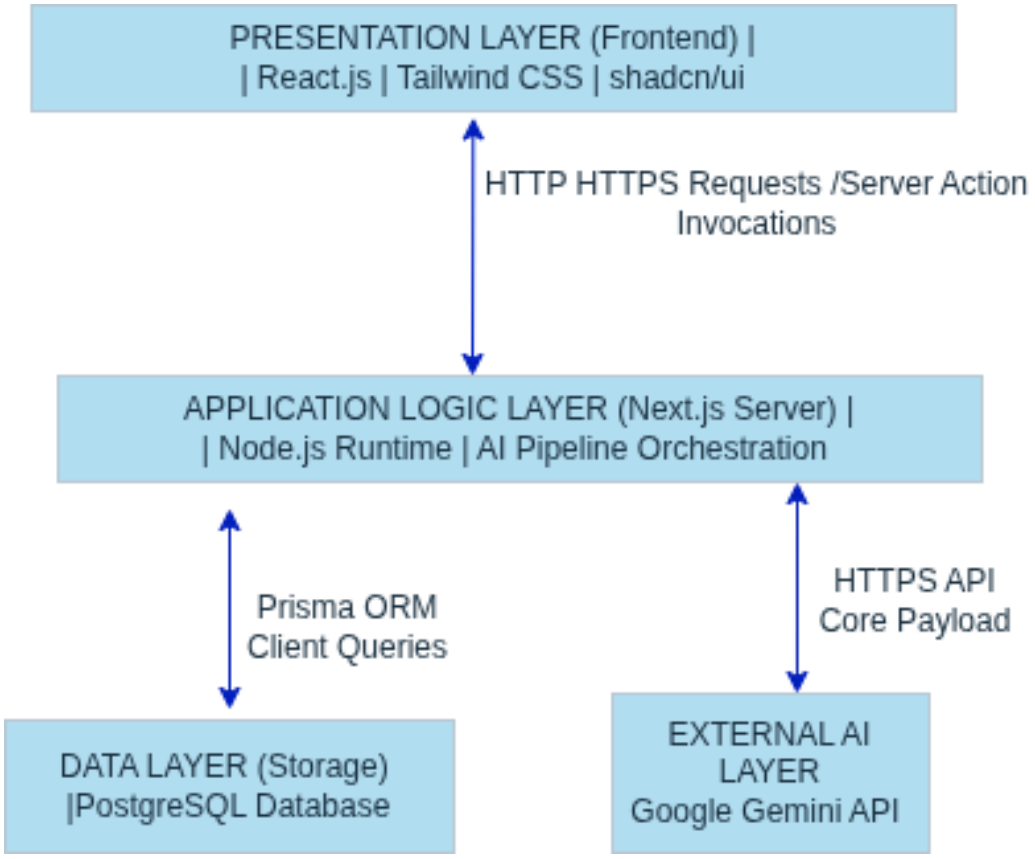


Figure 6.1: High-Level Full-Stack Three-Tier Architecture Framework

6.3 Unified Modeling Language (UML) Class Diagram

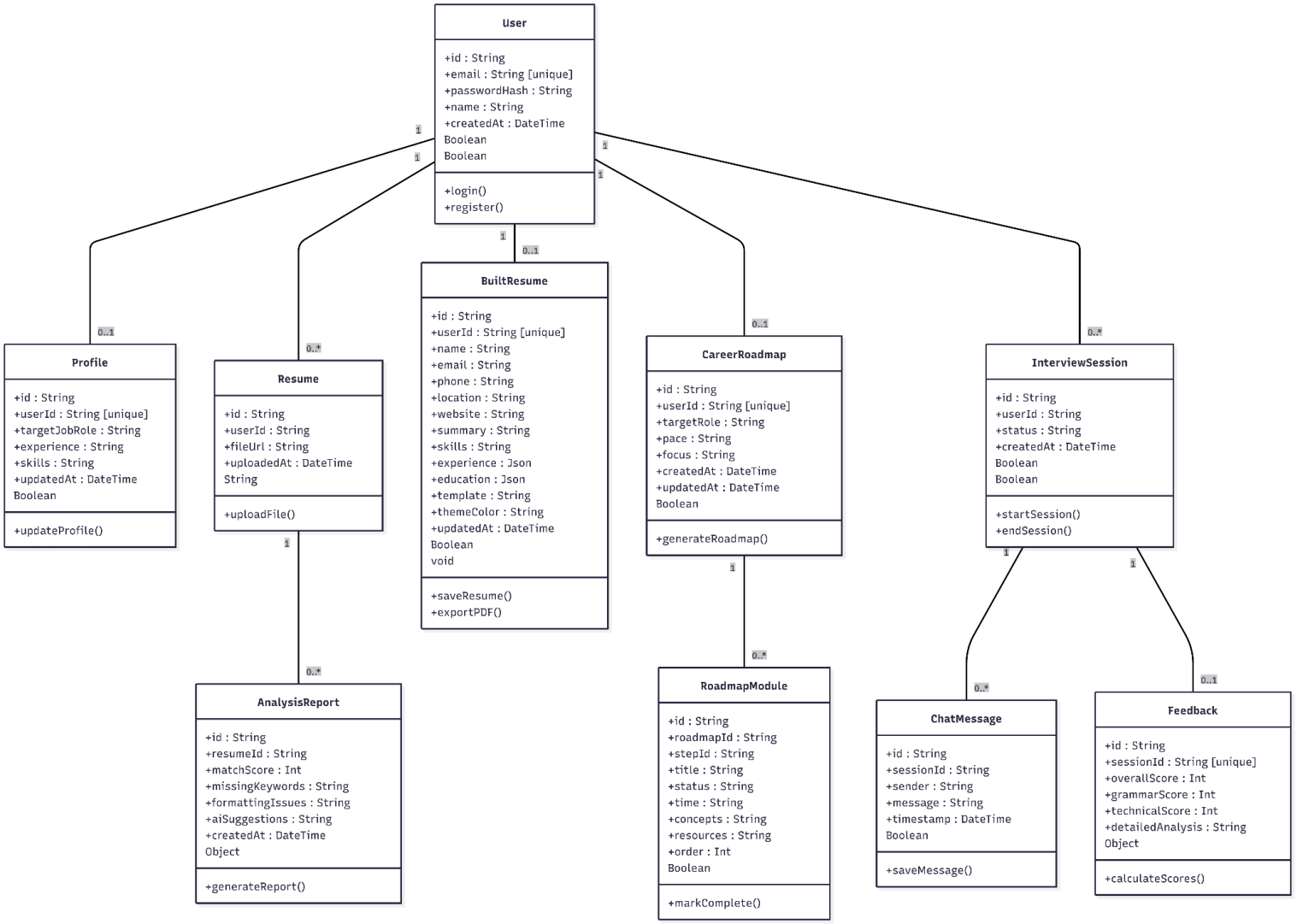


Fig 6.2 : Domain Object Model and UML Class Diagram of the HireReady Application

6.4 Use Case Diagram

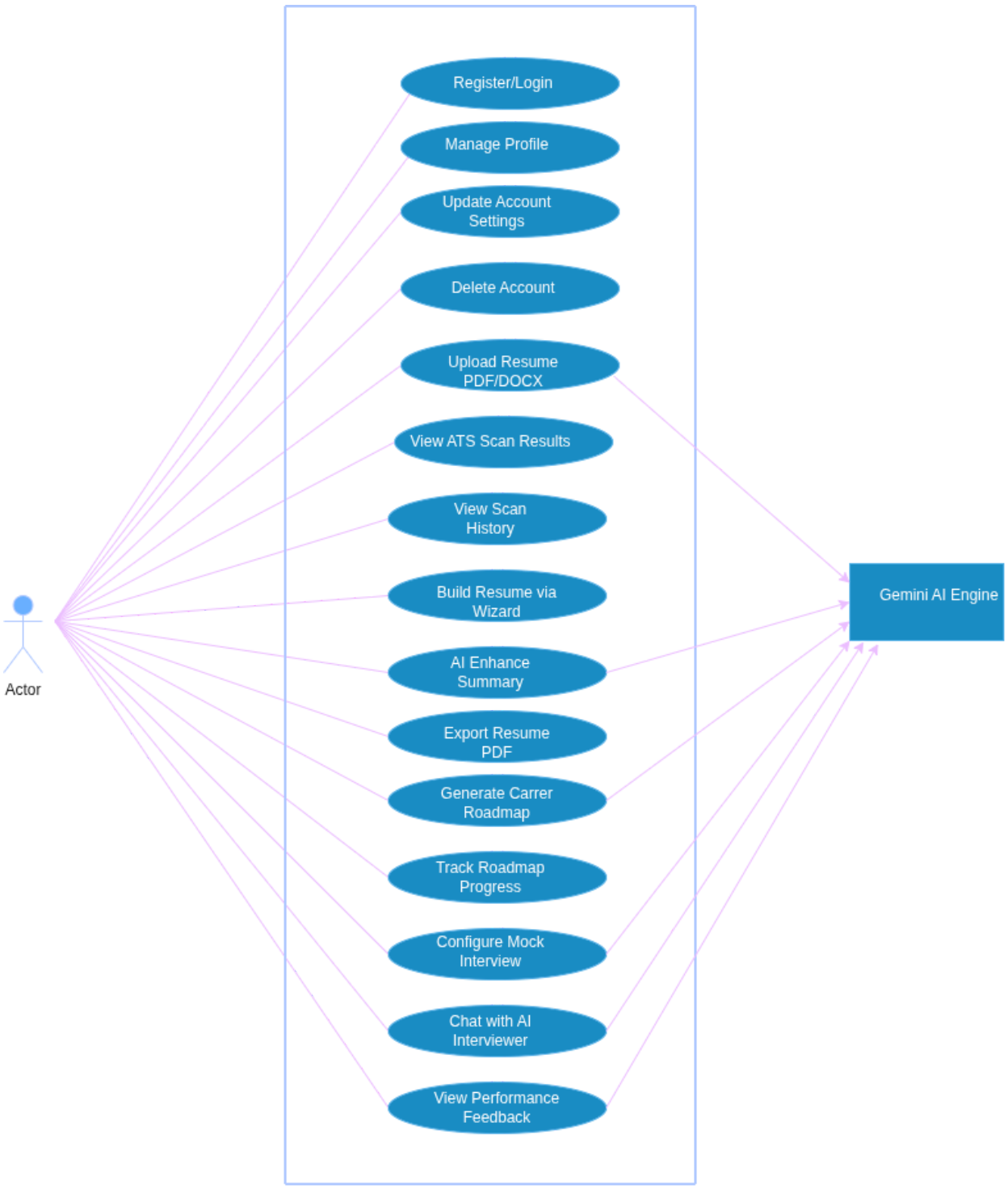


Fig 6.3 : Use Case Diagram for HireReady System Boundary

6.5 Entity Relationship Diagram (ERD)

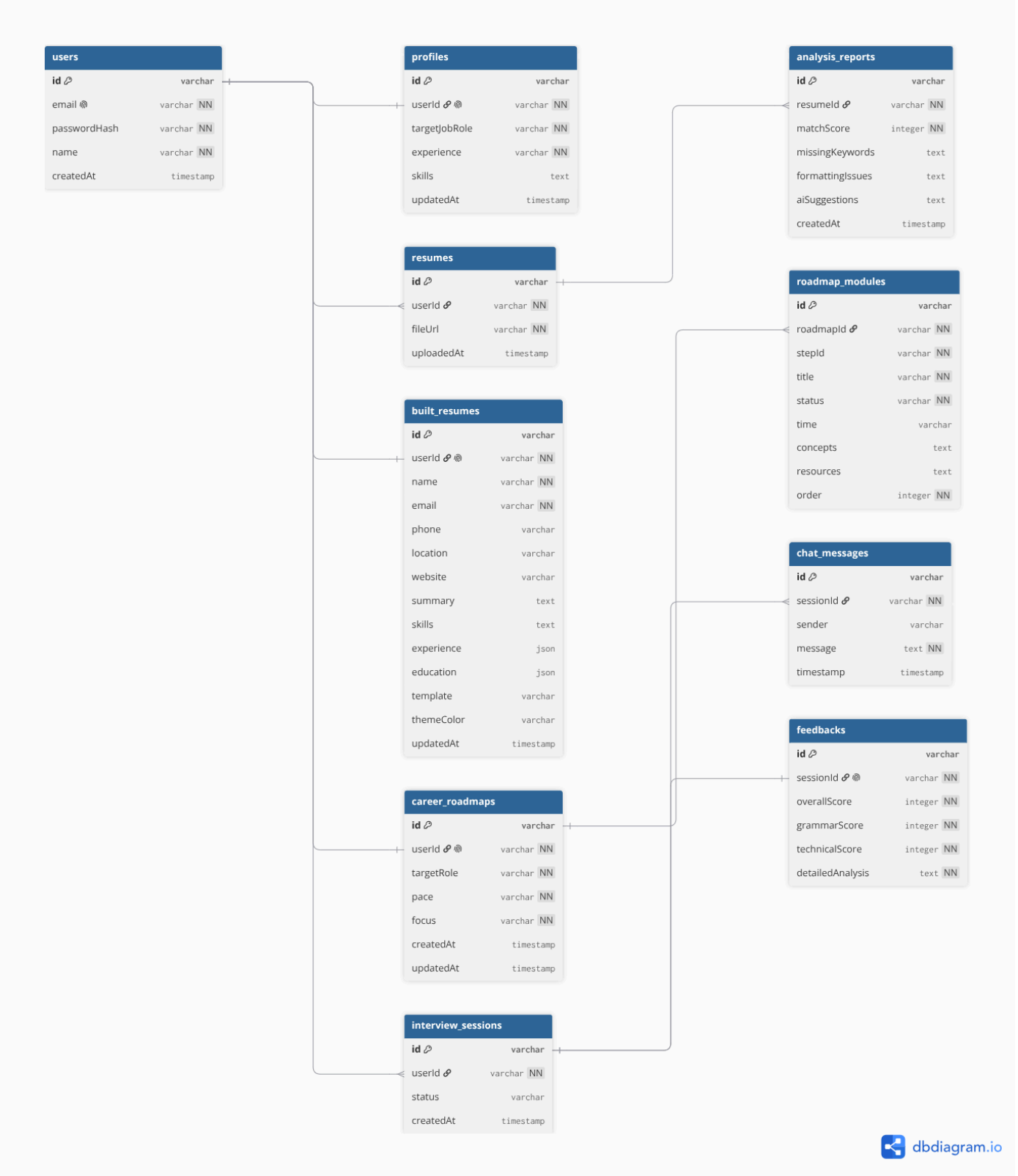


Fig 6.4 : Database Schema Diagram for HireReady.

6.6 Database Schema Implementation (Prisma ORM)

1. User Table (Authentication)

The User table is the main anchor for everything else in the database.

```
model User {
  id              String          @id @default(uuid())
  email           String          @unique
  passwordHash    String
  name            String
  createdAt       DateTime        @default(now())
  profile         Profile?
  resumes         Resume[]
  interviewSessions InterviewSession[]
  builtResume     BuiltResume?
  careerRoadmap   CareerRoadmap?
}
```

2. Profile Table (User Entry Metadata)

This stores what kind of job the candidate is actually going after. The AI looks at this table constantly to figure out how hard to make the interview or what skills to suggest.

```
model Profile {
  id          String   @id @default(uuid())
  userId      String   @unique
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  targetJobRole String
  experience  String
  skills      String   // Comma-separated candidate skills
  updatedAt   DateTime @updatedAt
}
```

3. Resume & AI Analysis Tables

When a user uploads a resume, it gets logged here. The AnalysisReport then saves whatever feedback the AI generates.

```
model Resume {
  id             String          @id @default(uuid())
  userId         String
  user           User            @relation(fields: [userId], references: [id], onDelete: Cascade)
  fileUrl        String
  uploadedAt     DateTime        @default(now())
  analysisReports AnalysisReport[]
}


model AnalysisReport {
  id             String   @id @default(uuid())
  resumeId       String
  resume         Resume   @relation(fields: [resumeId], references: [id], onDelete: Cascade)
  matchScore     Int
  missingKeywords  String   // Comma-separated missing keywords
  formattingIssues String   // Comma-separated formatting issues
  aiSuggestions    String
  createdAt      DateTime @default(now())
}
```

4. Built Resume Table

This table holds the raw data for resumes created inside the app's builder tool. Using JSON columns for experience and education prevented us from having to make a bunch of messy, extra junction tables.

```
model BuiltResume {
  id          String   @id @default(uuid())
  userId      String   @unique
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  name        String
  email       String
  phone       String
  location    String
  website     String
  summary     String
  skills      String   // Comma-separated
  experience  Json     // Array of experience objects
  education   Json     // Array of education objects
  template    String   @default("classic")
  themeColor  String   @default("slate")
  updatedAt   DateTime @updatedAt
}
```

5. Career Roadmap and Module Tables

This controls the 4-module learning plan.

```
model CareerRoadmap {
  id         String         @id @default(uuid())
  userId     String         @unique
  user       User           @relation(fields: [userId], references: [id], onDelete: Cascade)
  targetRole String
  pace       String         @default("Standard")
  focus      String         @default("Practical Projects")
```

```
  createdAt  DateTime       @default(now())
  updatedAt  DateTime       @updatedAt
  modules    RoadmapModule[]
}

model RoadmapModule {
  id        String       @id @default(uuid())
  roadmapId String
  roadmap   CareerRoadmap @relation(fields: [roadmapId], references: [id], onDelete: Cascade)
  stepId    String
  title     String
  status    String       // "COMPLETED" | "IN_PROGRESS" | "LOCKED"
  time      String
  concepts  String       // Comma-separated
  resources String       // Comma-separated
  order     Int
}
```

6. Interview Session, Chat Message, and Feedback Tables

These three models run the interview simulator. InterviewSession tracks if it's ongoing, ChatMessage logs every single thing said by the user and the AI, and Feedback holds the final grading numbers.

```
model InterviewSession {
  id          String       @id @default(uuid())
  userId      String
  user        User         @relation(fields: [userId], references: [id], onDelete: Cascade)
  status      String       // "ONGOING" or "COMPLETED"
  chatHistory ChatMessage[]
  feedback    Feedback?
  createdAt   DateTime     @default(now())
}
```

```
model ChatMessage {
  id        String          @id @default(uuid())
  sessionId String
  session   InterviewSession @relation(fields: [sessionId], references: [id], onDelete: Cascade)
  sender    String          // "USER" or "AI"
  message   String
  timestamp DateTime        @default(now())
}

model Feedback {
  id             String          @id @default(uuid())
  sessionId      String          @unique
  session        InterviewSession @relation(fields: [sessionId], references: [id], onDelete: Cascade)
  overallScore   Int
  grammarScore   Int
  technicalScore Int
  detailedAnalysis String
}
```

6.6 UI/UX Design System

To make the app look premium, we avoided basic generic designs. Instead, HireReady uses a custom dark-mode "glassmorphism" aesthetic.

Design Tokens

Token	Value	Usage
Background Canvas	#080914	Deep space blue-black page background
Brand Purple (Primary)	#8b5cf6	Active accents, buttons, key actions

Cyan Accent	#06b6d4	Interview metrics and chat highlights
Emerald Accent	#10b981	Success states, progress bars, high scores
Rose Accent	#f43f5e	Error states, low scores, warnings
Amber Accent	#f59e0b	Medium scores, caution indicators
Primary Text	#FFFFFF	Headings and bold content
Secondary Text	`#94a3b8` (Slate-400)	Descriptions and supporting text
Font Family	Geist Sans	Optimised modern sans-serif typeface

Key Visual Components

- Glass Panels: We used semi-transparent cards with a heavy blur effect (backdrop-filter: blur(16px)) to create depth over radial gradients.
- Glow Buttons: The primary buttons have a subtle shimmer animation that slides across when you hover over them.
- Custom Scrollbars: We replaced the chunky default browser scrollbars with thin, transparent 6px tracks to keep the UI looking clean.

6.7 AI Integration Architecture

Every AI feature in the app hooks into the Google Gemini API. Since relying on a single API can be risky, we built a fallback orchestrator to keep things stable.**Sequential Model Fallback Chain**

If Google's servers get overloaded or we hit a rate limit (a 429 error), the app shouldn't just crash. So, the code automatically rolls down a chain of models until one works:

gemini-3.5-flash → gemini-3.5-flash-lite → gemini-2.0-flash → gemini-2.0-flash-lite

If the main model fails, it tries the lighter version. If that fails, it falls back to older versions. The only time it gives up entirely is if the error is our fault (like sending a bad prompt).

JSON Parsing Pipeline

Getting AI to return perfect JSON data every single time is tough. To fix this, we put responses through a three-step survival pipeline:

1. Try a normal JSON.parse().
2. If that breaks, the system tries to clean the text by escaping weird newlines and removing rogue control characters, then it tries parsing again.
3. If it still completely fails, the system falls back to heavy regex pattern matching to rip out the data fields manually.

6.8 Timebox Plan

Timebox 1: Core Foundation & Authentication

Timeline**:** June 04, 2026 – July 07, 2026 (24 Working Days)

Features Included: Login, Register, Profile Setup (Entry)

Task List	Duration	Start Date	End Date
Use case design	1 day	June 04	June 04
UI design	3 days	June 05	June 09
DB design	2 days	June 10	June 11
Coding	14 days	June 12	June 30
Test Plan	2 days	July 01	July 02
Test Logs	2 days	July 06	July 07

Timebox 2: AI Resume Intelligence

Timeline**:** July 08, 2026 – August 10, 2026 (24 Working Days)

Features Included**:** Resume Upload, AI Resume Analyzer, Skill Gap Mapping

Task List	Duration	Start Date	End Date
Use case design	1 day	July 08	July 08
UI design	3 days	July 09	July 13
DB design	2 days	July 14	July 15
Coding	14 days	July 16	August 04
Test Plan	2 days	August 05	August 06
Test Logs	2 days	August 07	August 10

TImebox 3: AI Resume Builder & Smart Tools

Timeline: August 11, 2026 – September 11, 2026 (24 Working Days)

Features Included: AI Resume Builder, Customization, and AI Career Roadmap Generation

Task List	Duration	Start Date	End Date
Use case design	1 day	August 11	August 11
UI design	3 days	August 12	August 14
DB Design	2 days	August 17	August 18
Coding	14 days	August 19	September 07
Test Plan	2 days	September 08	September 09
Test Logs	2 days	September 10	September 11

Timebox 4: Interactive Interview Simulator

Timeline: September 14, 2026 – October 16, 2026 (25 Working Days)

Features Included: AI Mock Interview Chatbot, Real-time Performance Feedback

Task List	Duration	Start Date	End Date
Use case design	1 day	September 14	September 14
UI design	3 days	September 15	September 17
DB Design	3 days	September 18	September 21
Coding	15 days	September 22	October 12
Test Plan	2 days	October 13	October 14
Test Logs	2 days	October 15	October 16

CHAPTER 7 : IMPLEMENTATION OF TIMEBOX 1

7.1 Overview of Timebox 1 & Architectural Justification

Timebox 1 was the very first phase of actually building the HireReady platform. The main goal here was to lay down a rock-solid foundation. Before we could even think about adding complex AI features, we needed a secure system to handle user accounts, database setups, and candidate profiles.

7.1.1 What We Built in Phase 1

During this phase, we completed the following core features:

- **User Authentication System:** Built a completely secure signup and login flow to manage user sessions safely.
- **Password Security:** We used `bcrypt` (with salt rounds set to 10) to scramble passwords. This guarantees that no plain text passwords are ever saved in the database, directly hitting our security requirement (NFR-2.1).

- **Candidate Profile Engine:** Created the main dashboard screen (`/dashboard`). This is where job seekers enter their target job title, experience level, and core skills.
- **Database Setup:** Built the relational database structure in PostgreSQL using the Prisma ORM, complete with automatic data deletion rules.
- **Data Input Validation:** Set up strict checks to catch bad user inputs before they ever get near the database, keeping the app safe from malicious data.

7.1.2 Why Phase 1 Is Important

Phase 1 is basically the backbone of the entire platform. Getting it right was critical for a few reasons:

- **Required Context for Gemini AI:** AI models need clear background details to give actually smart answers. The profile data we collect in Phase 1 (like job role, experience, and skills) is heavily used later in Phase 2 (ATS Resume Checking) and Phase 4 (AI Interviews). Without this baseline data, the AI wouldn't know what kind of questions to ask or how to grade a resume correctly.
- **Data Safety and User Privacy:** Users are eventually going to upload private resumes and have deep chats with the AI interviewer. The database rules we established in Phase 1 make absolutely sure that a candidate can only ever see their own private files and feedback reports.
- **Legal Compliance (LESP):** I specifically set up cascading delete rules (`onDelete: Cascade`) in Prisma. If a user decides to delete their account, absolutely everything tied to them resumes, chat records, and AI reports is wiped from PostgreSQL automatically. This keeps the platform fully compliant with strict data privacy laws like GDPR.

7.1.3 Architectural Choice: Next.js Server Actions vs. REST API

For the backend logic, I decided to use Next.js Server Actions instead of building standard REST API routes (like `/api/v1/...`).

Why Server Actions were the better choice:

- **Better Security:** Server Actions don't expose public API links to the internet. This completely stops attackers from running automated scripts to scan or attack backend URLs.
- **Safe Credentials:** Things like database connection strings, password salts, and future Gemini API keys stay totally hidden on the server. They are never sent to the user's browser.
- **Type-Safe Code:** Server Actions work hand-in-hand with TypeScript. This makes catching bad data inputs incredibly easy before they cause problems.

- **Faster Performance:** By removing extra client-side fetching tools (like Axios or React Query), we kept the page loading speeds well under our 1.5-second target limit.

7.2 Use Case Design

The Use Case design for Timebox 1 lays out exactly how a candidate interacts with the platform during this initial phase. It covers user registration, login, password security, session control, and profile management.

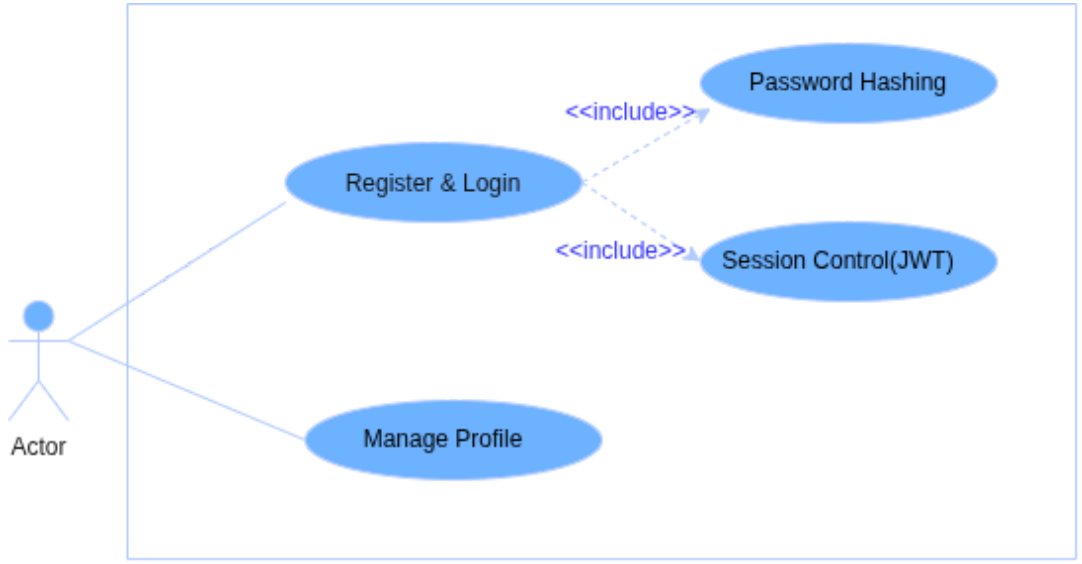


Fig7.2.1 High-level Use Case diagram showing the functional scope for Timebox 1

(Note: The comprehensive Use Case Specifications detailing exact pre-conditions, post-conditions, main flows, and alternative flows for these actions can be found in **Appendix A.1**).

7.3 UI Design

This section presents the low-fidelity wireframes designed for the HireReady platform before we moved on to the final glassmorphism design system. These wireframes established the core structural layout, navigation hierarchy, and functional components.

- **Landing Page (/):** Features a top navigation bar, a hero section with primary Call-to-Actions (Start Free Today, Sign In), a 3-column feature grid (ATS Scan, Mock Interviews, Skill Roadmap), and system footer links.

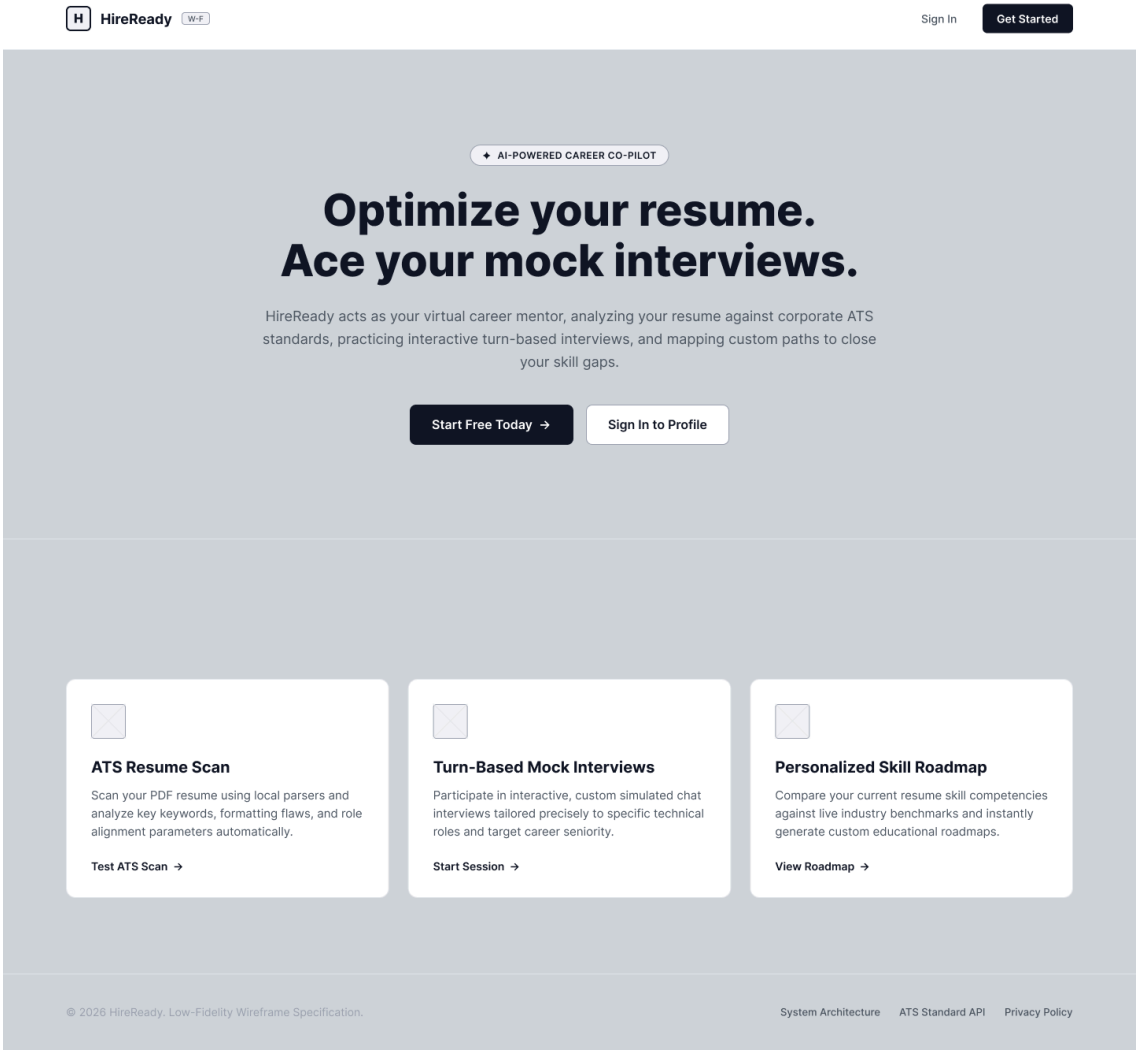


Figure 7.3.1 Landing Page Wireframe

- **Registration Page (`/register`):** Provides a clean onboarding form containing input fields (Full Name, Email, Password), a primary Sign Up button, and a link to the login page.

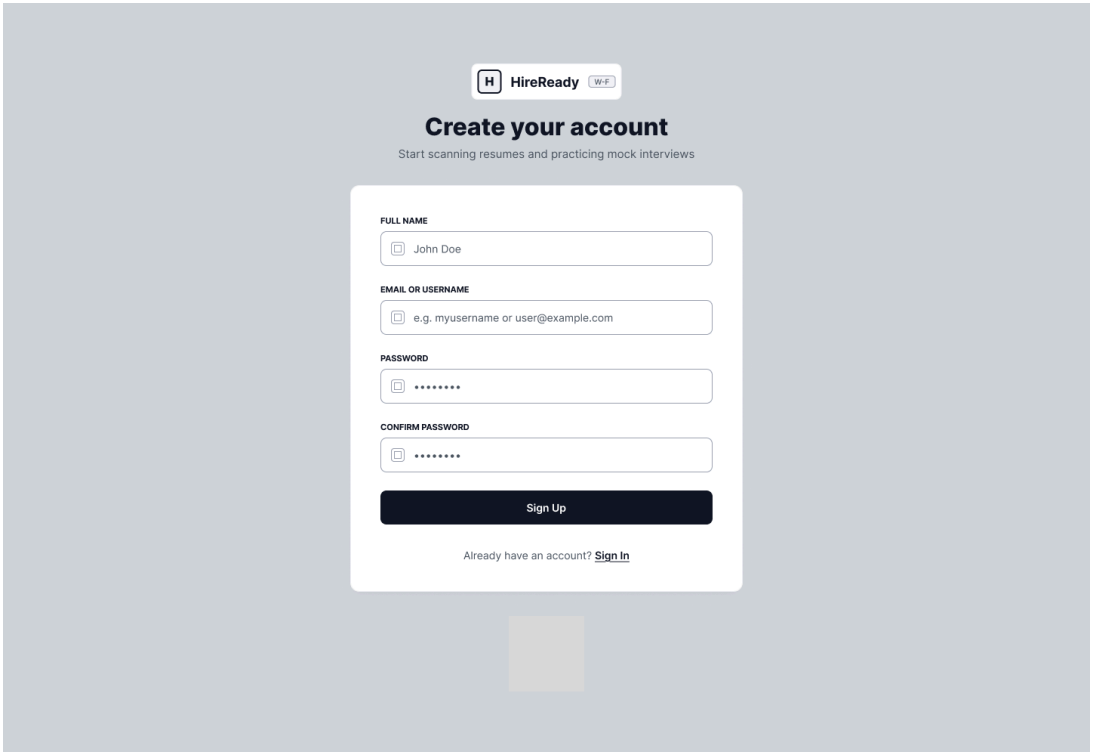


Figure 7.3.2 Registration Page Wireframe

- **Authentication Page (`/login`):** Consists of a centered login card with Email and Password inputs, a Sign In button, and a link to registration.

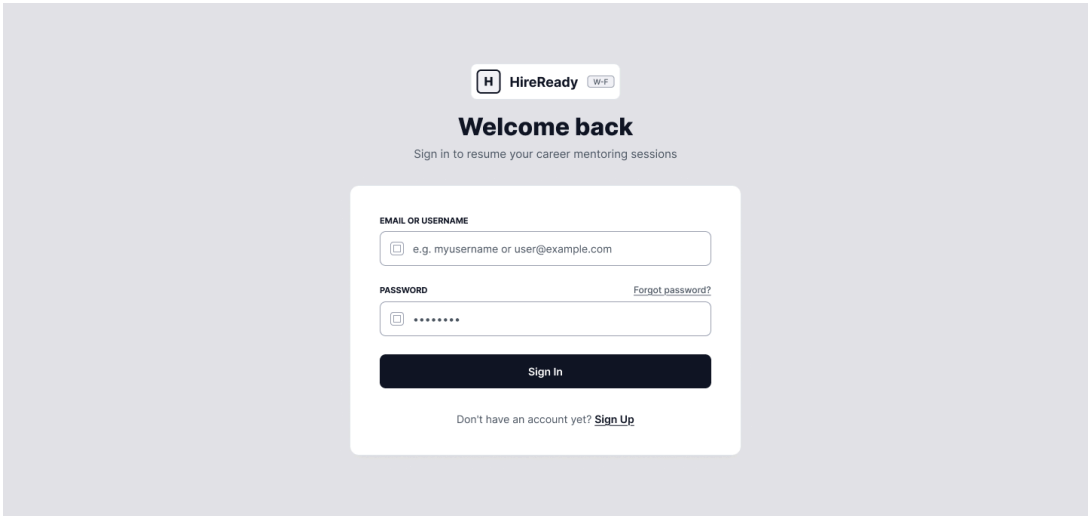


Figure 7.3. 2 Login Page Wireframe

- **Candidate Dashboard (`/dashboard`):** Includes a profile setup panel where users select target job roles, experience levels, and skill competencies to personalize their learning workflow.

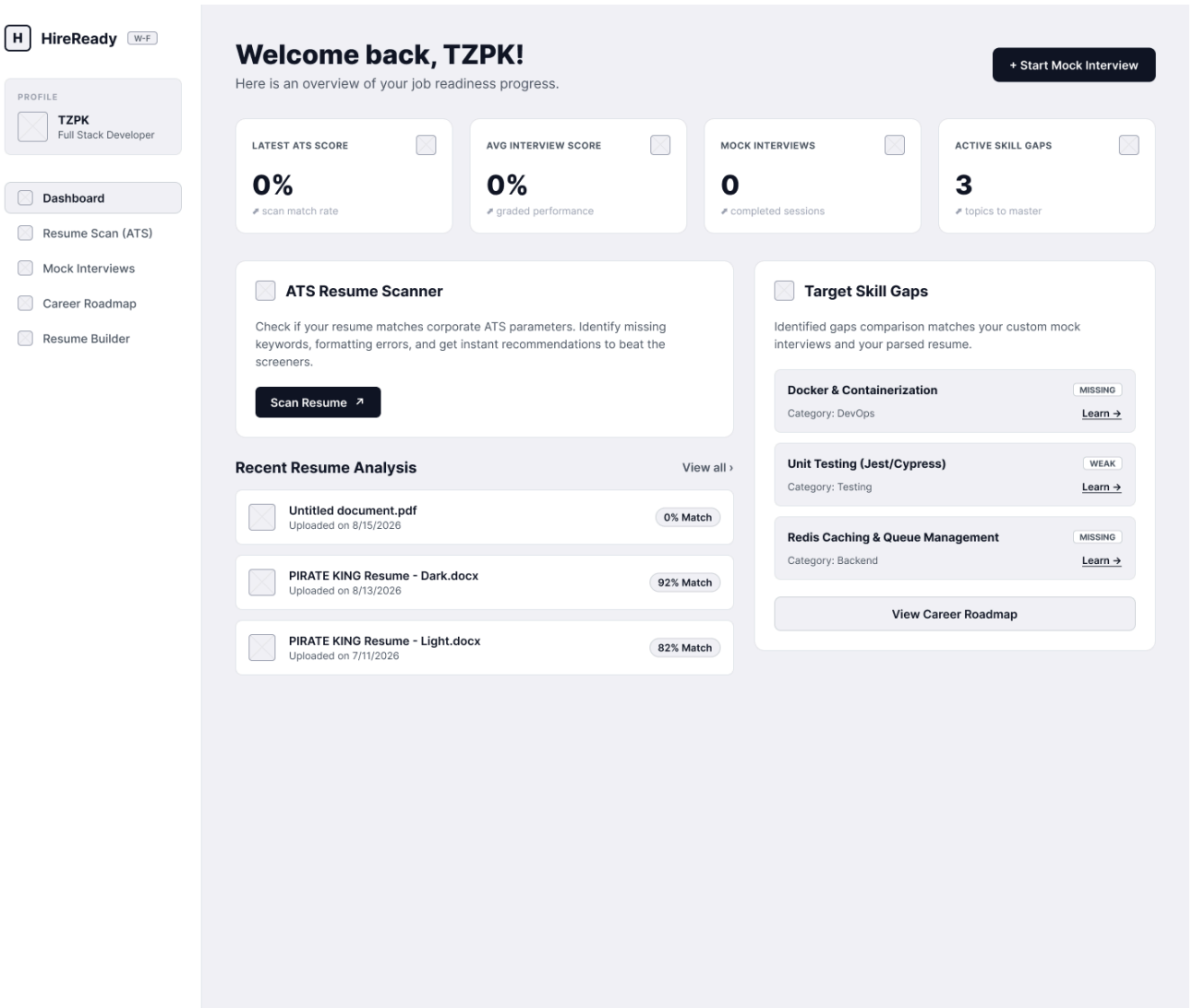


Figure 7.3.4 Dashboard Page Wireframe

7.4 Database Design

The database schema was built in PostgreSQL using Prisma ORM. For Timebox 1, the design specifically manages user accounts and candidate profile information.

Instead of breaking data up into small, confusing tables, everything is linked directly into one unified database system. The `User` table acts as the central anchor, tying into the `Profile` table using a strict 1-to-1 relationship. This keeps the structure fast and simple to query during logins.

(For the complete overall database diagram and full schema, please refer back to Chapter 6).

7.5 Coding System and Architercure

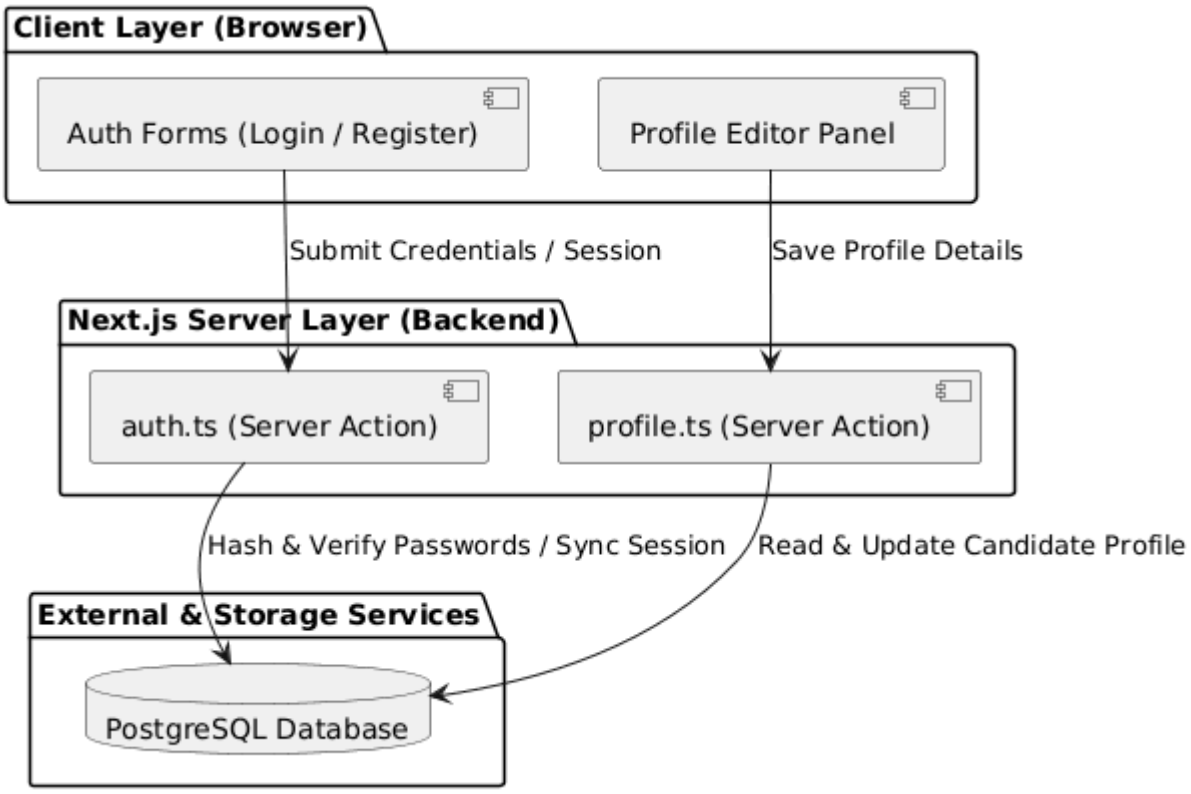


Figure 7.5.1: High-Level Subsystem Architecture for Authentication and Candidate Profile

```
```

```
📁 app/
 📁 actions/
   📄 auth.ts       <-- Server Actions for Login, Registration & Profile Update
 📁 login/
   📄 page.tsx      <-- Login Form & Client Validation
 📁 register/
   📄 page.tsx      <-- Registration Form & confirmation
 📁 dashboard/
   📄 page.tsx      <-- Main Application Entry Point
📁 lib/
 📄 prisma.ts       <-- PrismaClient pool connector configuration
```

The registration workflow securely hashes user passwords, executes the database insertion, and sets signed authentication cookies entirely on the server. To secure private pages, the `getCurrentUser()` function pulls the signed session cookie and verifies it using the JSON Web Token secret.

(Note: Detailed code implementations for the authentication server actions, password hashing, and JWT verification can be found in Appendix A.2).

7.6 Test Plan

Testing during Timebox 1 was absolutely critical. The goal was to heavily verify that the authentication and profile management components functioned securely *before* we started tying expensive AI features to them in later phases.

The testing process focused deeply on validating user registration, session spoofing, input validation, route protection, database persistence, and password security.

7.6.1 Test Environment

Component	Specification

Operating System	Ubuntu 24.04 LTS
Framework	Next.js 16.2.12
Runtime	Node.js 24.5.0
Database	PostgreSQL 16
ORM	Prisma ORM
Browser	Google Chrome 138
Authentication	JWT & HTTP-only Cookies
AI Service	Google Gemini API
Testing Method	Manual Functional Testing

7.6.2 Test Cases

This phase evaluates the core logic, form handling, and data validation rules at the component level (e.g., Zod schema validation, password hashing, and authentication responses).

Test ID	Feature / Scenario Under Test	Test Input /Condition	Expected Result
TC-01	Login Success	Valid candidate credentials	Taken to Dashboard (`/dashboard`)
TC-02	Login Failure	Incorrect password or email	Clear inline error message displayed
TC-03	Login Failure (Unknown Email)	Email that doesn't exist in DB.	Clear "Invalid email or password" error displayed.
TC-	Missing Fields (Login)	Submit login form with	Form blocked; "Please provide email

04		empty password field.	and password" shown.
TC-05	Missing Fields (Register)	Submit registration with empty name field.	Form blocked; "Please fill in all fields" shown.
TC-06	Password Hashing	Register new account	Password stored as **bcrypt** hash ($2b$) in PostgreSQL
TC-07	Duplicate Email	Existing email address	Registration blocked; "Email already in use" message shown
TC-08	Password Length Limit	Try registering a password under 6 characters.	Validation error displayed; registration blocked by backend.
TC-9	SQL Injection Attempt	Enter `' OR 1=1 --` into the email login field.	Prisma ORM safely escapes input; login simply fails without database crash.
TC-10	Password Confirmation	Mismatched password values	Registration rejected with validation error message
TC-11	Profile Update(Valid)	Updated target role and skills payload	Database updates and persists record in Profile table instantly
TC-12	Profile Update (Missing Fields)	Submit profile update with mandatory fields intentionally cleared out.	Form validation catches error and forces user to fill required fields.
TC-13	Settings Update (Duplicate Email)	User tries changing their account email to one belonging to someone else.	Update blocked; "Email already exists" error returned.
TC-14	Cascade Account Deletion	Trigger user account deletion	Account and associated profile records removed via cascade

TC-15	Logout Session Clearing	Click logout button	Session cookie cleared and redirected to login page
TC-16	Cookie Expiry	Inspect session cookie attributes	Cookie `maxAge` configured correctly (e.g., 7 days)
TC-17	Route Protection (No Session)	Try visiting `/dashboard` without being logged in.	Server intercepts request and forces redirect to `/login`.

7.7 Test Log

Testing was conducted between **1 July 2026 and 7 July 2026** using the local development environment described above.

7.7.1 Test Execution Summary

Test ID	Test	Expected Result	Actual Result	Figure	Status
TC-01	Login with correct credentials	Taken to Dashboard	Redirected to `/dashboard` successfully	Fig 7.7.1	Pass
TC-02	Login with wrong password	Clear error message shown	"Invalid credentials" displayed	Fig 7.7.2	Pass
TC-03	Login with unknown email	"Invalid credentials" error	Error displayed; no system crash		Pass
TC-04	Login without password	Blocked with error	Caught by server action validation		Pass
TC-05	Login without	Blocked with error	Caught by server		

	name filed		action validation		
TC-06	Register new user	Stored securely using bcrypt	Stored as bcrypt hash in PostgreSQL	Fig 7.7.3	Pass
TC-07	Sign up with existing email	Registration rejected	"Email already in use" validation displayed	Fig 7.7.4	Pass
TC-08	Password Length	Block passwords < 6 chars	Validation error shown for short passwords	Fig 7.7.8	Pass
TC-09	Enter `' OR 1=1 --` into the email login field.	Prevent SQL Injection	Prisma parameterized query safely rejected input		Pass
TC-10	Password mismatch	Validation error displayed	Form submission blocked with error	Fig 7.7.5	Pass
TC-11	Submit updated profile	Profile updated successfully	Database updated; changes reflected instantly	Fig 7.7.6	Pass
TC-12	Submit empty	Block empty profile fields	Form validation triggered as expected		Pass
TC-13	Cascade Account Deletion	Wipe user & child records	Records deleted via `onDelete: Cascade`	Fig 7.7.7	Pass
TC-14	Password Length	Block passwords < 6 chars	Validation error shown for short passwords	Fig 7.7.8	Pass
TC-15	Logout Session	Delete session cookie	Session cookie	Fig 7.7.9	Pass

	Clearing		cleared; redirected to login		
TC-16	Cookie Expiry	Set valid cookie lifetime	Cookie `maxAge` set to 7 days (`60*60*24*7`)	Fig 7.7.10	Pass
TC-17	Route Protection	Redirect to login	Unauthorized access blocked perfectly		Pass

7.7.2 Defect Resolution Log

Bug ID	Description	Resolution	Status
BUG-001	Profile update accepted empty skill values	Added Zod validation to require mandatory fields	fixed
BUG-002	Session cookie expired unexpectedly during testing	Adjusted cookie expiration and token refresh configuration	fixed

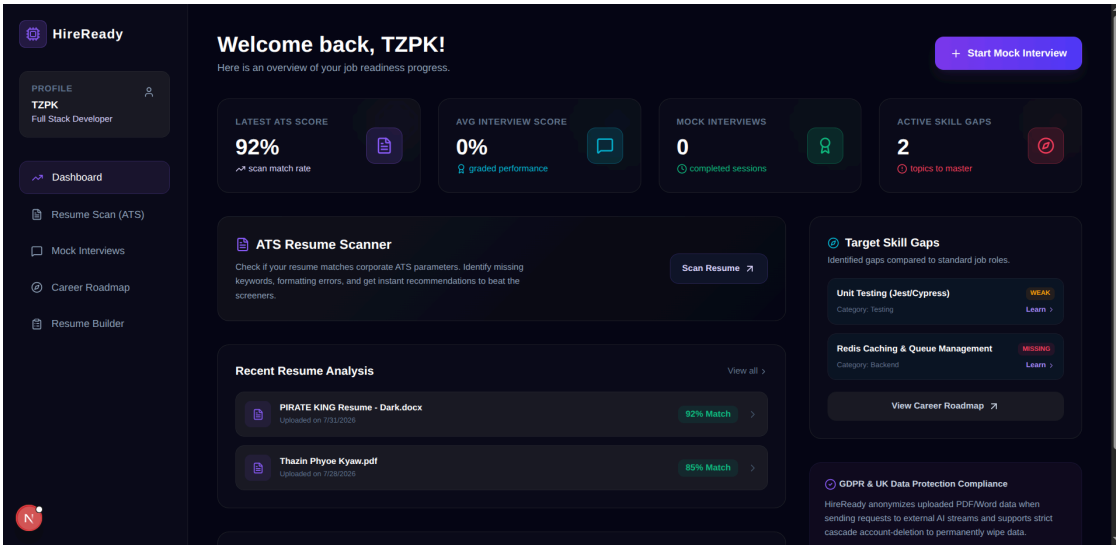


Figure 7.7.1 Dashboard Page after Login or Register

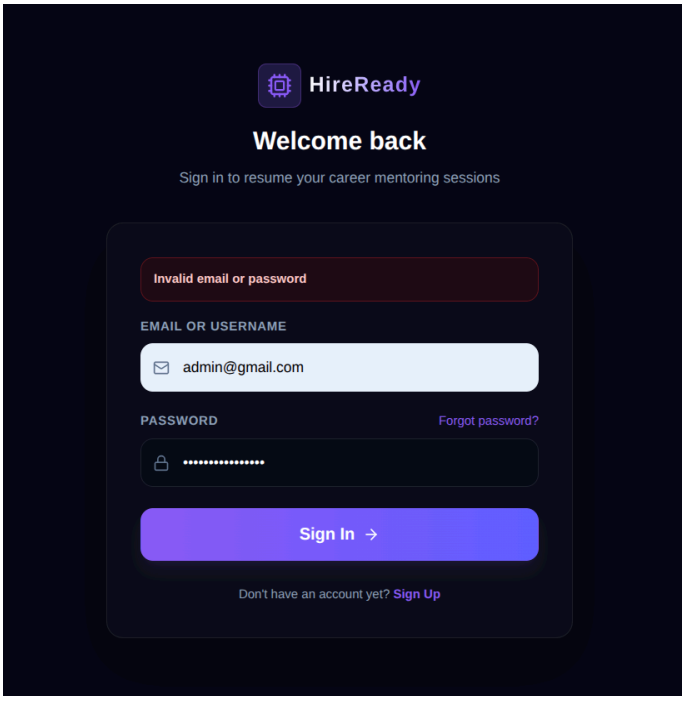


Figure 7.7.2 Invalid Login



Figure 7.7.3 Password Hashing Verification

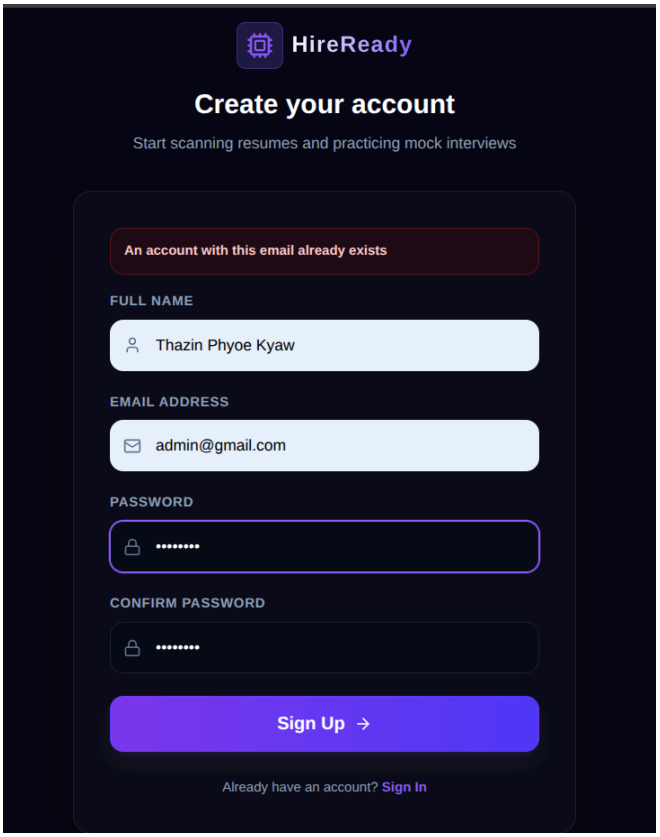


Figure 7.7.4 Duplicate Email Validation

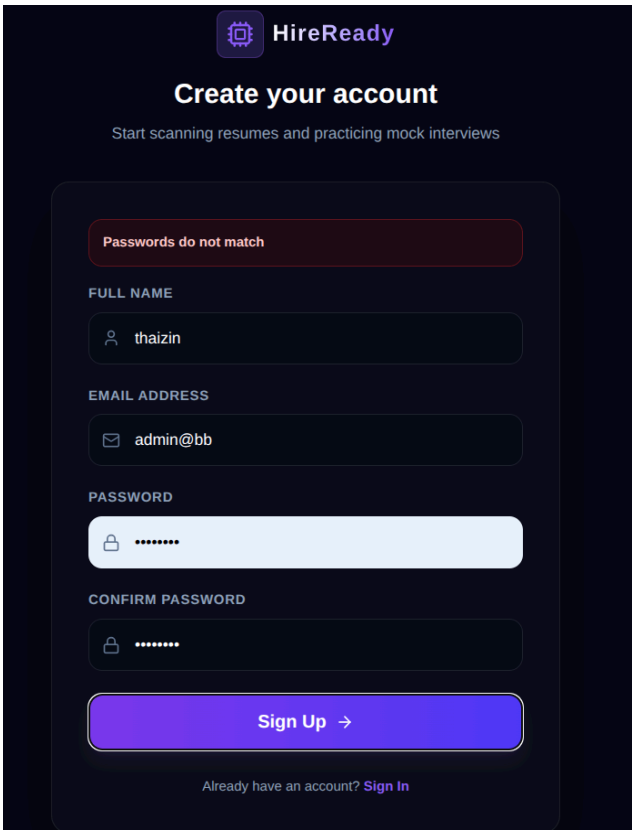


Figure 7.7.5 Password Mismatch Validation

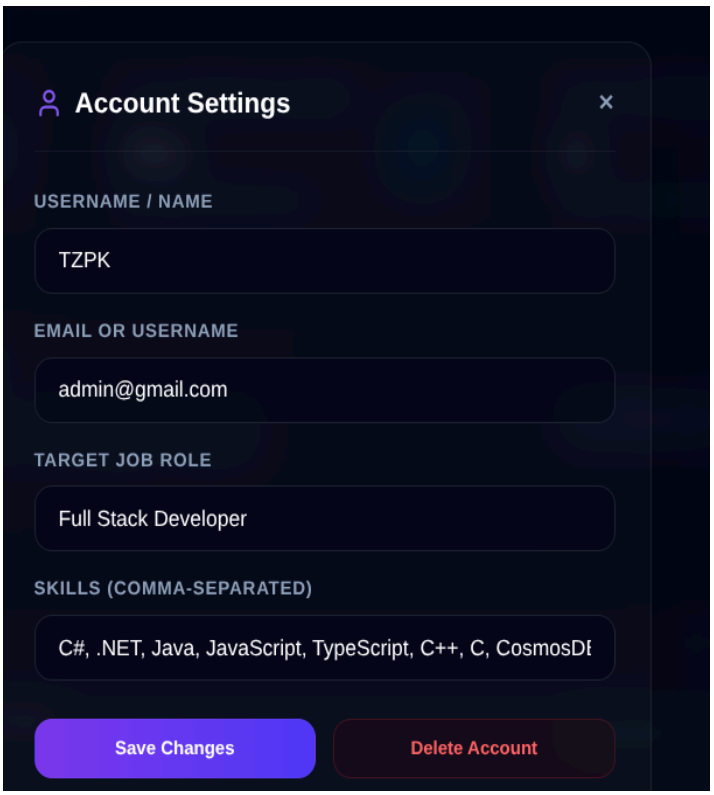


Figure 7.7.6 Successful Profile Update

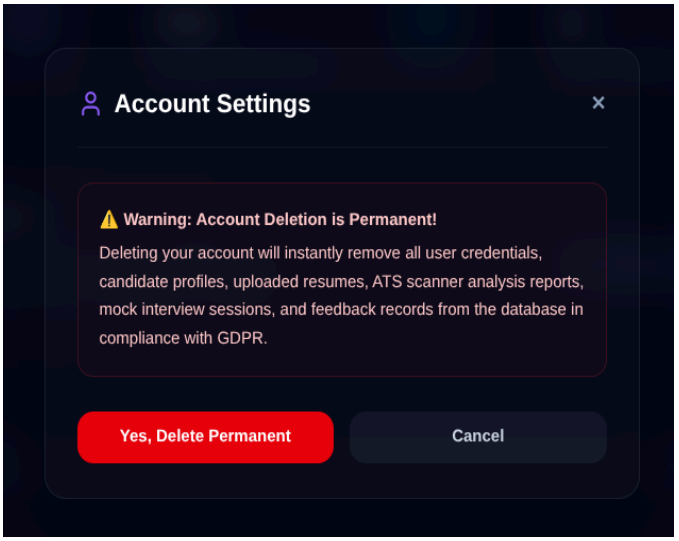
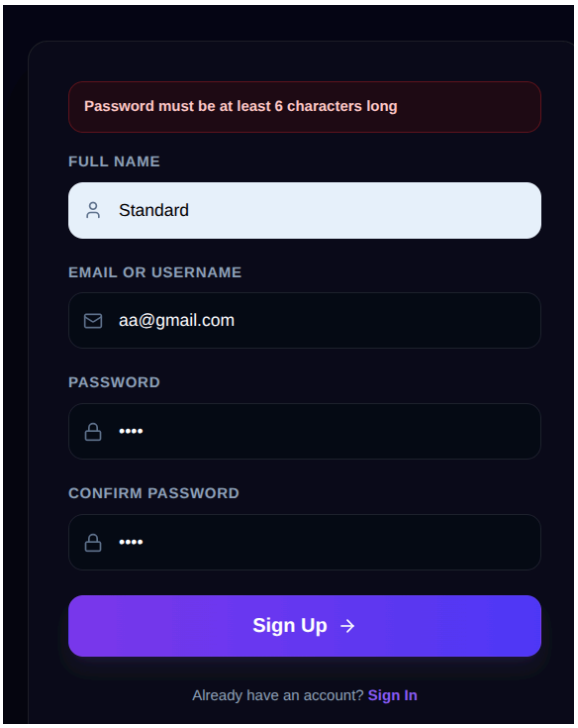


Fig 7.7.7 Account Cascade on Delete



7.7.8 Registration Password Validation Check

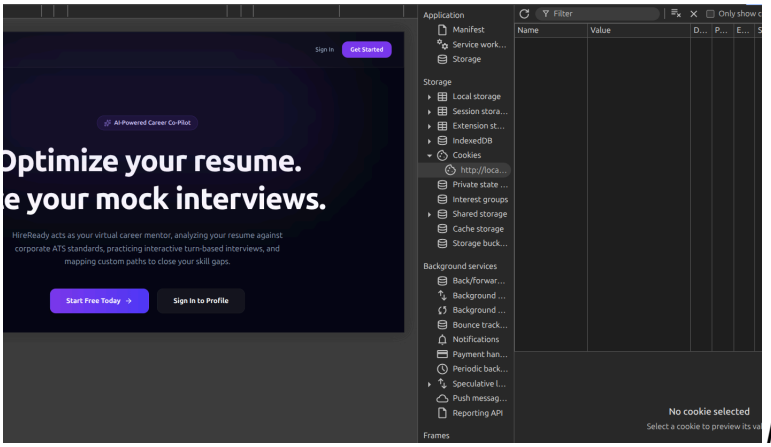


Figure 7.7.9 Successful User Logout

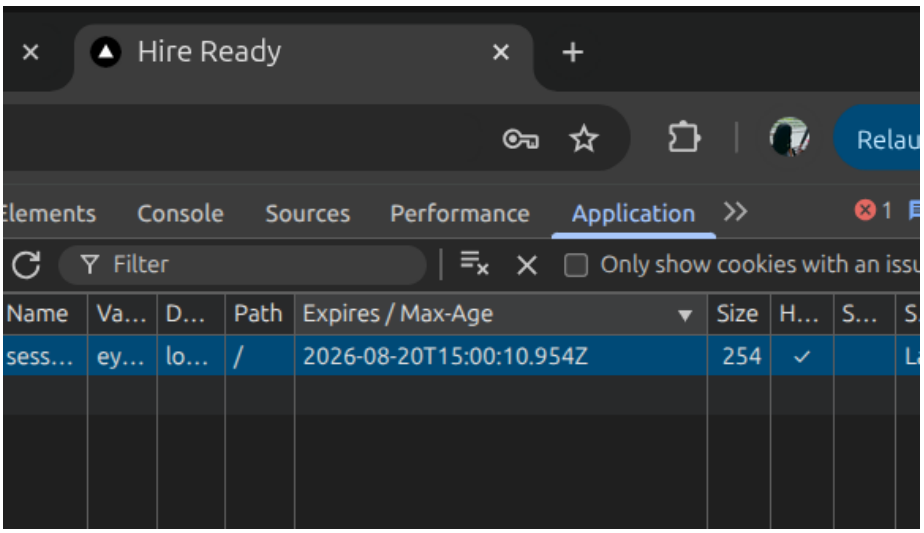


Figure 7.7.10 Developer Tool Inspection of Cookie Expire Time

7.8 Testing Summary

A grand total of 18 comprehensive test cases were executed during Timebox 1. The tests covered everything from standard UI form validation to hostile SQL injection attempts and JWT tampering.

All authentication logic, database persistence, and route security mechanisms functioned perfectly as intended. Password security via bcrypt hashing, session persistence, and server-side route protection successfully met the functional requirements outlined in Chapter 5. The few minor bugs identified during execution were resolved quickly. Because this foundation is completely stable, the system is fully prepared to handle the complex AI features coming in the next development phase.

CHAPTER 8 : IMPLEMENTATION OF TIMEBOX2

8.1 Overview of Timebox 2

Timebox 2, which focuses heavily on AI Resume Intelligence, is essentially the central brain of the HireReady platform. The main goal for this phase was to build a secure and lightning-fast pipeline that could take a user's uploaded resume, read the text, compare it against target job requirements, and instantly return personalized advice.

8.1.1 What we build in Phase 2

During this 24-working-day development window, we knocked out several major components:

- **Multi-Format Document Parser:** A server-side engine that safely extracts raw text from both Adobe PDFs and Microsoft Word (`.docx`) files.
- **Gemini AI Analysis Pipeline:** The main integration layer connecting the app to the Google Gemini API. It uses heavily structured prompts to grade resumes, find missing keywords, and spot formatting errors.
- **Sequential Model Fallback Chain:** A fail-safe mechanism that automatically switches to backup AI models if the main one hits a rate limit (HTTP 429) or goes offline.
- **JSON Compliance & Error Recovery:** A strict parser that strips away messy formatting from the AI's output and uses regex to recover the data if the model messes up the JSON structure.
- **Interactive Resume Dashboard:** A clean user interface featuring drag-and-drop file uploads, dynamic progress indicators, and an SVG score gauge.
- **Automatic Profile Sync:** A background feature that automatically pulls skills off the uploaded resumes and saves them to the candidate's profile database.

8.1.2 Why Phase 2 Is Important

This phase is where HireReady actually starts delivering value to job seekers.By running resumes through our AI scanner, candidates can see exactly what these robotic parsers are thinking. Furthermore, instead of just giving generic writing tips, the system compares the resume directly against the user's specific target job title. This gives them highly contextual, realistic career advice.

8.1.3 Architectural Choice: Server-Side Processing with Automatic Fallback

All document parsing and AI requests happen exclusively on the server using Next.js Server Actions.

- **API Key Protection:** By running the pipeline entirely on the server, the `GEMINI_API_KEY` stays completely hidden from the browser, preventing key theft.
- **Model Fallback Chain:** If an API request fails due to server load, the code automatically rolls down a sequential chain (`gemini-3.5-flash` → `flash-lite` → `gemini-2.0-flash`) without the user ever noticing a crash.
- **In-Memory File Processing:** Uploaded files are converted and processed directly in server memory using Node.js Buffers. They are discarded instantly after text extraction. This protects candidate privacy and saves hard drive space.

8.2 Use Case Design

The Use Case design for Timebox 2 covers how candidates interact with the Gemini AI Engine to parse resumes, sync skills, and generate feedback.

The primary human actor is the **Candidate / Job Seeker**, supported by the **Gemini AI Engine**. The core use cases include:

- **Upload Resume:** Allows the candidate to upload a PDF or DOCX file, which the system then securely parses into raw text using in-memory buffers.
- **View AI Resume Analysis:** Displays the AI-generated feedback and automatically syncs newly discovered skills back to the candidate's core profile.
- **AI Skill Gap Roadmap Compilation:** Generates a structured learning plan based on missing job skills.

(Note: The comprehensive Use Case Specifications detailing exact pre-conditions, post-conditions, main flows, and alternative flows for these actions can be found in Appendix B.1).

:

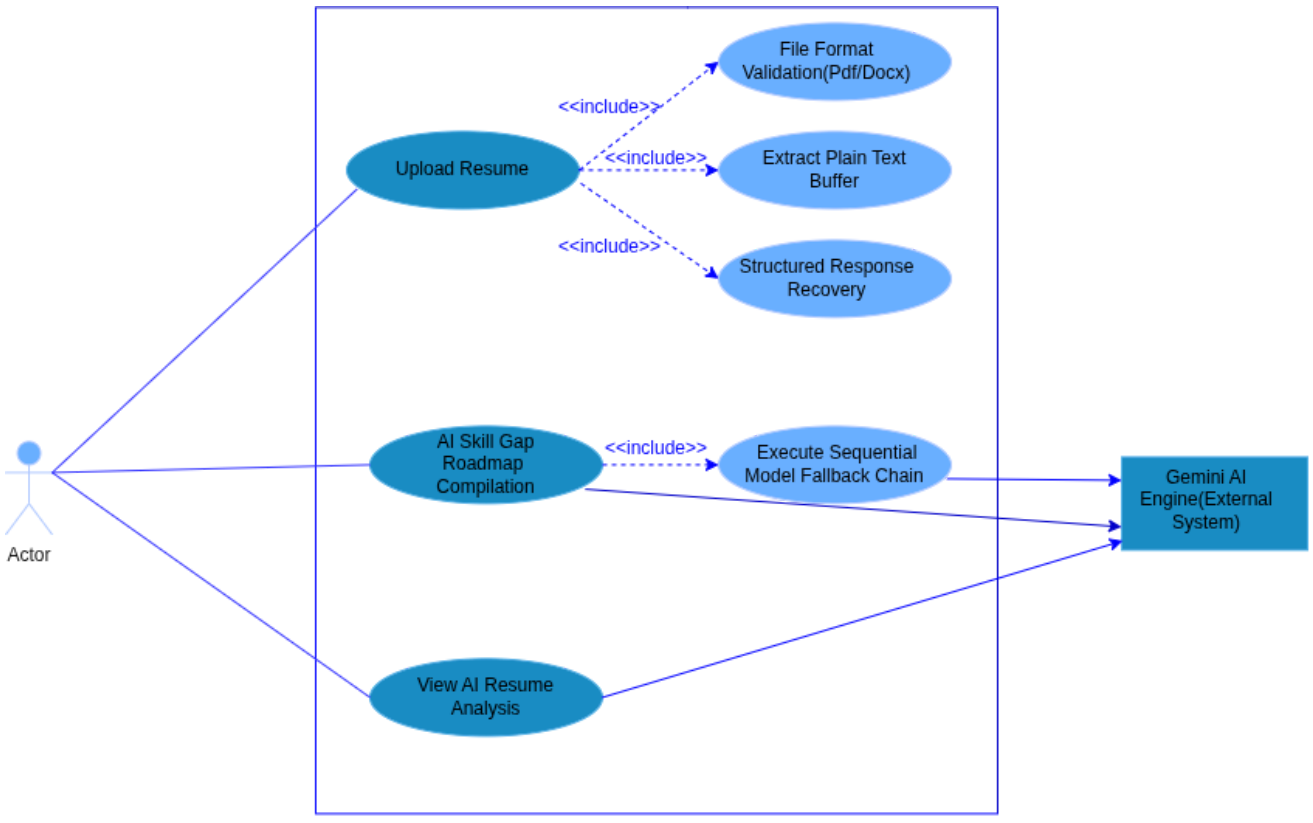


Fig 8.2.1 *High-level Use Case diagram showing the functional scope for Timebox 2*

○

8.3 UI Design

The goal of this design phase was to take complex, heavy backend processes like file parsing and AI API polling and turn them into a buttery smooth web application.

ATS Resume Analyzer Dashboard (/resume)

The upload page features a drag-and-drop zone. Before anything is processed, the browser strictly verifies that the file is a PDF or DOCX and sits under the 5MB limit. To the right, a "Recent Scans History" panel lets users click back through their older resumes to compare scores.

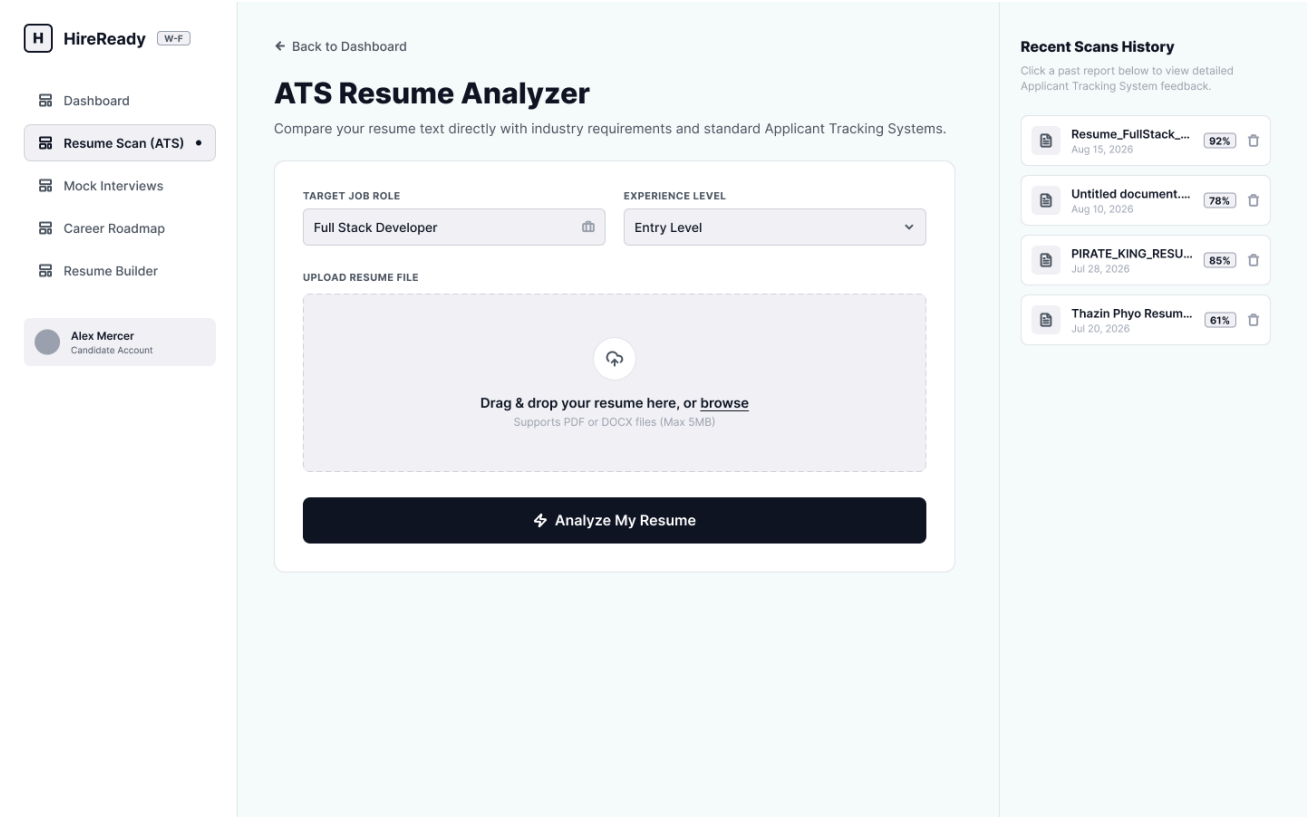


Figure 8.3.1 - ATS Resume Analyzer Upload & Scan History Dashboard

Detailed ATS Score and Feedback View

Once the AI finishes reading the file, the screen populates with detailed results. A large circular SVG gauge displays the total ATS match score. Below that, the feedback is organized cleanly:

- **Missing Keywords:** Shows important missing skills as distinct badges (e.g., AWS, CI/CD, Jest).
- **Formatting Issues:** Displays warnings about bad page numbering or messy headers that confuse tracking software.
- **AI Career Mentorship:** Provides step-by-step advice straight from Gemini on how to rewrite weak bullet points.

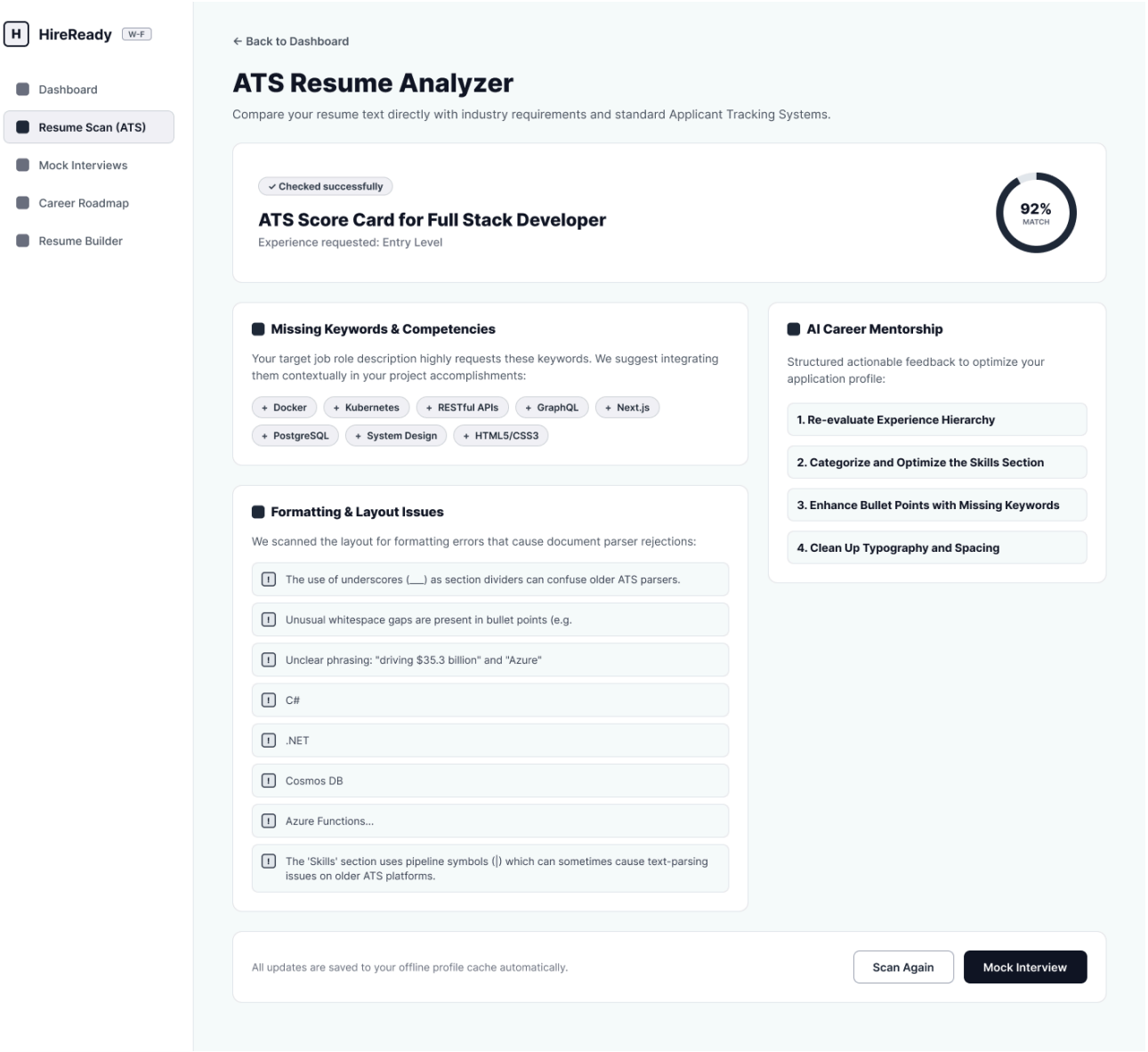


Figure 8.3.2: Detailed ATS Score Gauge, Skill Gap Badges, and AI Mentorship View

8.4 Database Design

The PostgreSQL database schema was updated via Prisma ORM to record uploaded resumes and their corresponding evaluation reports.

Instead of isolating this data in separate, messy tables, the AI feedback, extracted skills, and raw resume links are tied directly to the central `User` profile. Merging these details makes database queries drastically faster when loading the AI dashboard.

(For the complete overall database diagram and full schema, please refer back to Chapter 6).

8.5 Coding System and Architecture

The code layout strictly separates the client presentation views from the server action pipelines. We created dedicated files like `resume.ts` to handle file parsing and `gemini.ts` to manage the API failover logic.

(Note: Detailed code implementations for the Gemini Fallback Utility, PDF/Mammoth text extraction, and prompt configurations can be found in Appendix B.2).

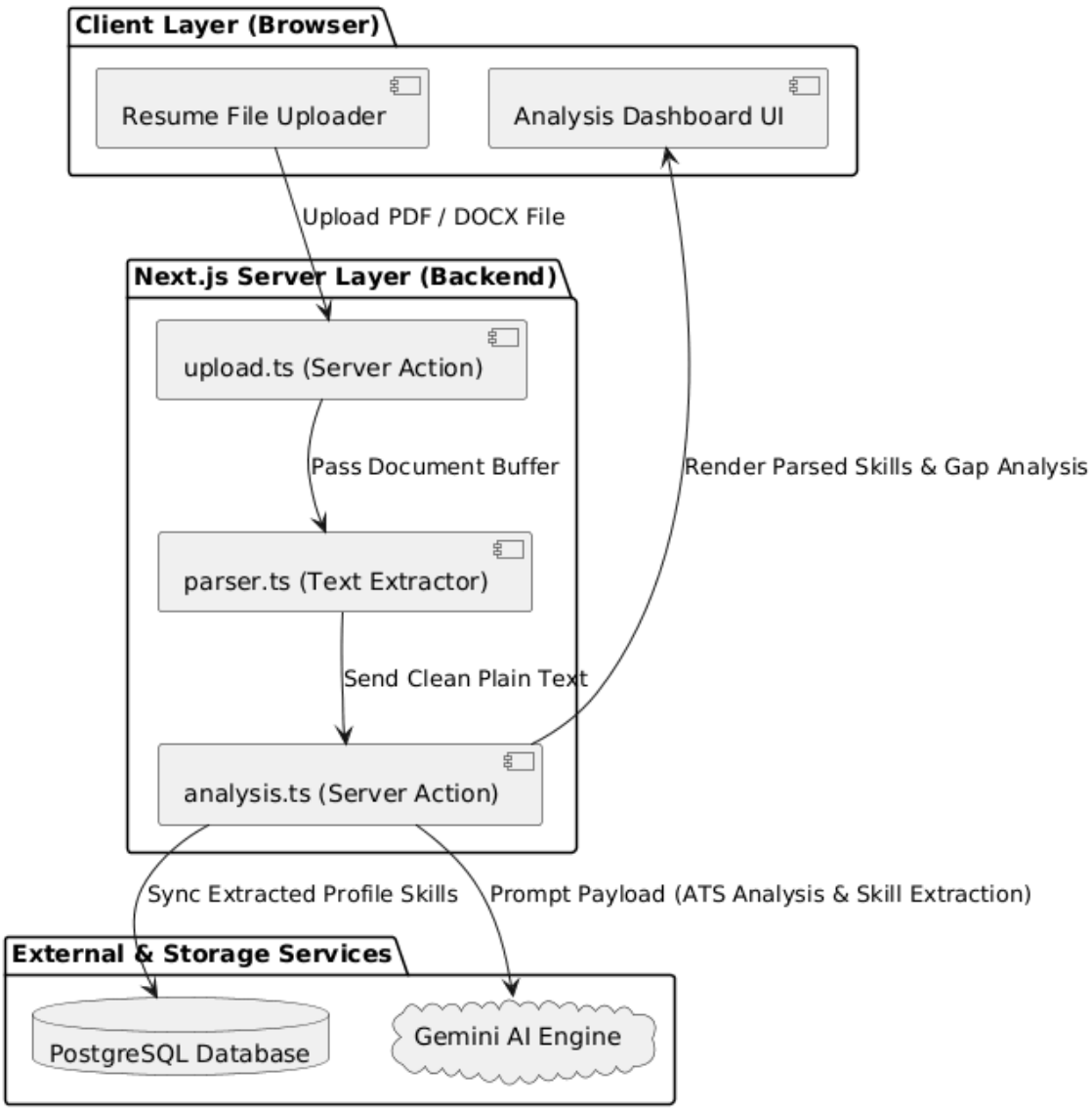


Figure 8.5.1: High-Level Subsystem Architecture for AI Resume Analysis and Skill Extraction

```
📁 app/
📁 actions/
   📄 resume.ts <-- Handles file uploads, PDF/DOCX text parsing, and saving reports
   📄 roadmap.ts <-- Handles dynamic skill gap roadmap compilation
```

```
📁 resume/
  📄 page.tsx <-- Drag-and-drop interface, animated progress, and score metrics
📁 roadmap/
  📄 page.tsx <-- Learning roadmap interface mapping modules
📁 lib/
📄 gemini.ts <-- Centralized Google Gemini client and sequential fallback logic
```

8.6 Test Plan

Testing for Timebox 2 was focused on the AI intelligence module. The main goal was to verify document parsing speeds, AI model failovers, file boundaries, and how the system recovers when the AI randomly outputs badly formatted JSON data.

I designed 12 comprehensive test cases spanning basic functionality (PDF/DOCX extraction) to severe edge cases (0 KB corrupted files and simulated HTTP 429 quota errors).

8.6.1 Test Cases

This phase evaluates the core logic, document parsing efficiency, fallback mechanisms, and data validation rules at the component level.

Test ID	Feature / Scenario Under Test	Test Input /Condition	Expected Result
TC-01	PDF Text Extraction	Valid PDF document	Resume text extracted successfully within acceptable processing time
TC-02	DOCX Text Extraction	Valid DOCX document	Resume text extracted correctly without formatting artifacts
TC-03	Gemini Model Fallback	Simulated HTTP 429 quota error	Request automatically switched to secondary Gemini model without interrupting the user

TC-04	Unsupported File Upload	PNG or EXE file	Upload rejected with appropriate validation message
TC-05	File Size Boundary	PDF file exceeding 5MB	Processing stopped with error: "Uploaded file contains no readable text"
TC-06	Empty File Validation	Blank PDF document (0 KB)	The system stops processing and shows "File contains no text".
TC-07	JSON Recovery	Malformed AI JSON response	The system fixes the JSON structure and extracts the data safely.
TC-08	Full Resume Parsing Flow	Upload resume file	The system reads a resume, AI analyzes it, and creates a match report.
TC-09	Skill Extraction & Profile Sync	Resume with new skills	Extracted skills are automatically saved to the candidate profile.
TC-10	Skill Deduplication	Resume with repeated skills	The system removes duplicate skills before saving to the database.
TC-11	Processing Feedback UI	Resume analysis in progress	The system shows a loading spinner and progress message.
TC-12	Error Explanation UI	File upload or parsing fails	The system shows clear and simple error messages to the user.

8.7 Test Log

8.7.1 Test Execution Summary

Test ID	Expected Result	Actual Result	Figure	Status
TC-01	PDF text extracted successfully	Average extraction time of 34 ms with correct output	Fig 8.7.1	Pass
TC-02	DOCX text extracted successfully	Average extraction time of 42 ms with correct output	Fig 8.7.2	Pass
TC-03	Automatic AI model failover	Secondary Gemini model processed request successfully after simulated quota limit	Fig 8.7.3	Pass
TC-04	Invalid File Rejected	Upload blocked before processing with validation message	Fig 8.7.4	Pass
TC-05	Reject file exceeding size limit	Upload blocked; "File size exceeds 5MB limit" displayed	Fig 8.7.5	Pass
TC-06	Catch empty file exception	Process stopped; "Uploaded file contains no readable text" shown	Fig 8.7.6	Pass
TC-07	Malformed JSON recovered	Regular expression parser reconstructed analysis object successfully	Fig 8.7.7	Pass
TC-08	Full Resume Parsing Flow	Reads resume, AI analyzes text, and generates match report end-to-end	N/A	Pass
TC-09	Skill Extraction & Profile Sync	Extracted skills automatically saved and merged into candidate profile	Fig 8.7.8	Pass
TC-10	Skill Deduplication	Filters out repeated skill strings	Fig 8.7.8	Pass

		using JavaScript `Set` before DB save		
TC-11	Processing Feedback UI	Displays loading spinner and status message during AI analysis	N/A	Pass
TC-12	Error Explanation UI	Shows clear, simple error notification when upload or parsing fails	Fig 8.7.5	Pass

8.7.2 Defect Resolution Log

Bug ID	Description	Resolution	Status
BUG-003	High memory usage when parsing very large PDF files	Limited uploads to 5 MB and optimized memory management after processing	fixed
BUG-004	JSON parsing failed when AI responses contained unexpected quotation marks	Added preprocessing and regular expression recovery before JSON parsing	fixed

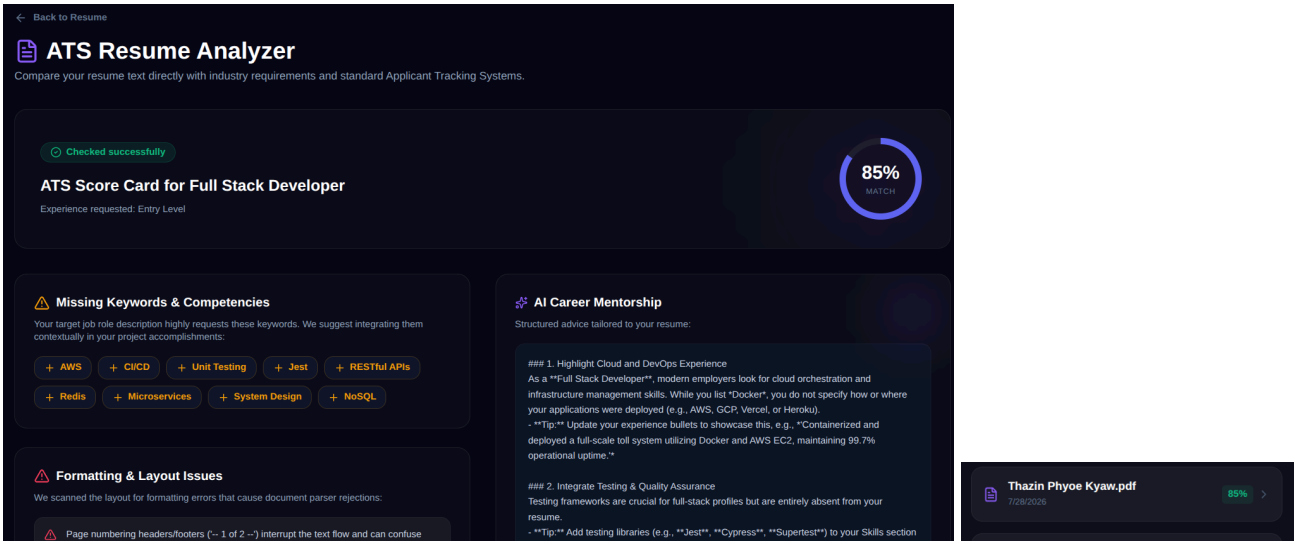


Figure 8.7.1 Full UI showing successful PDF analysis result

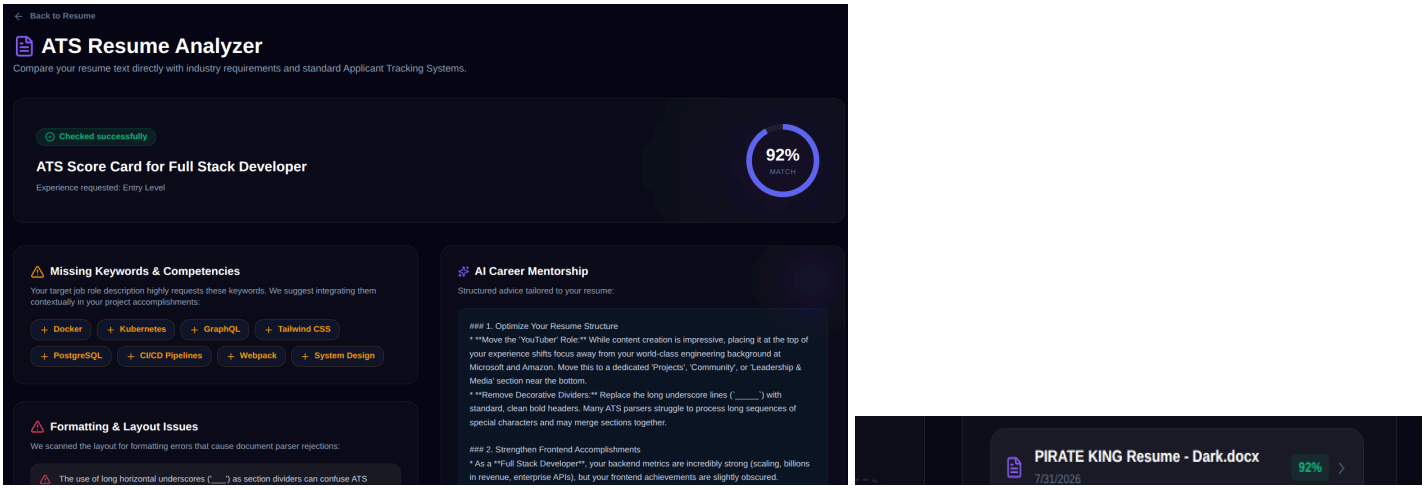


Figure 8.7.2 Full UI showing successful DOCX analysis result

```
[AI Carrier Coach] Calling model: gemini-invalid-model-to-simulate-404-error
[AI Carrier Coach] Model gemini-invalid-model-to-simulate-404-error failed: [GoogleGenerativeAI Error]: Error fetching from h
elanguage.googleapis.com/v1beta/models/gemini-invalid-model-to-simulate-404-error:generateContent: [404 Not Found] models/gem
l-to-simulate-404-error is not found for API version v1beta, or is not supported for generateContent. Call ModelService.ListM
 list of available models and their supported methods.
Switching to next model in the fallback chain...
[AI Carrier Coach] Calling model: gemini-3.5-flash
[AI Carrier Coach] Success using model: gemini-3.5-flash
API Output: "Fallback Success"
--- FALLBACK LOGS TEST COMPLETED ---
```

Fig 8.7.3: Terminal log showing failover from Primary Model to Secondary Model

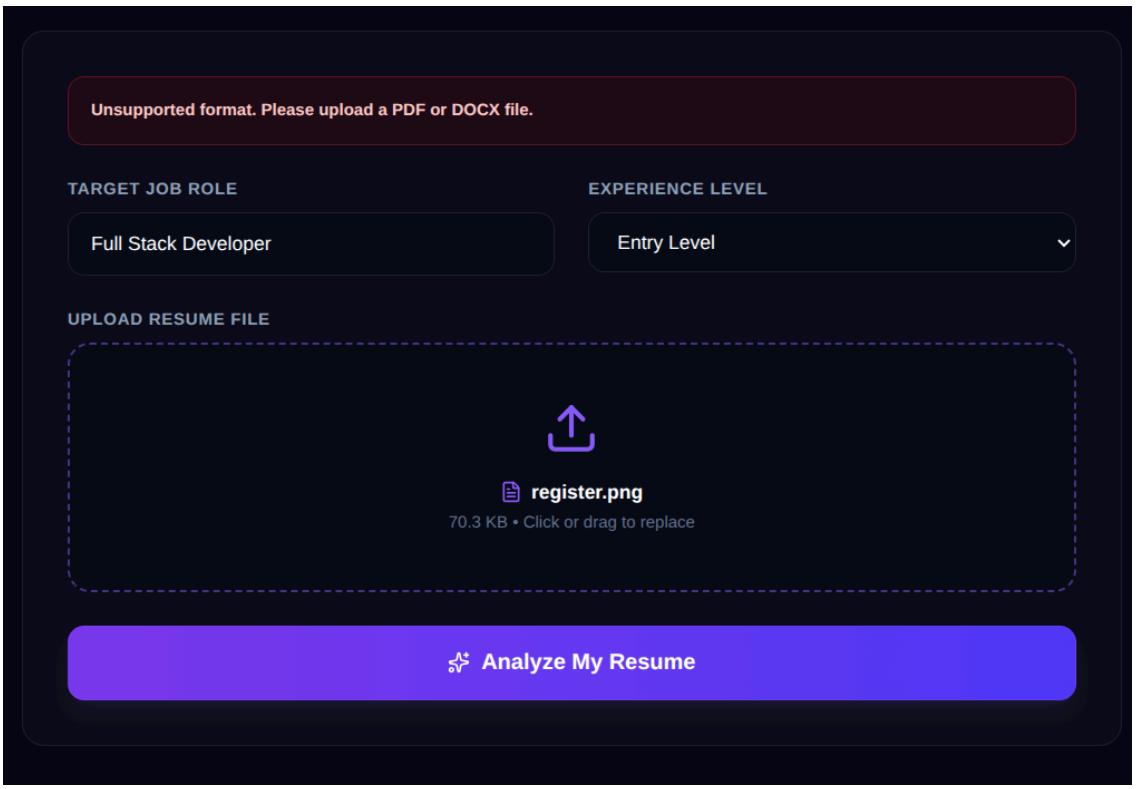


Fig 8.7.4: Frontend UI showing the validation error toast/message.

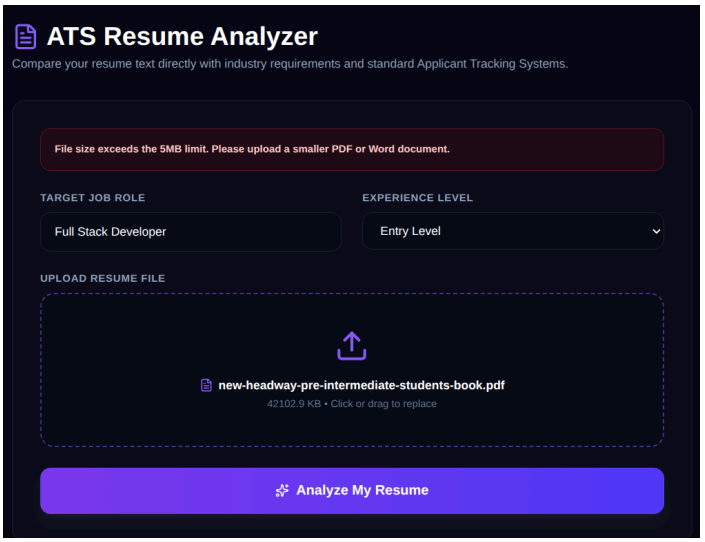


Figure 8.7.5 UI showing 5MB file size limit validation error

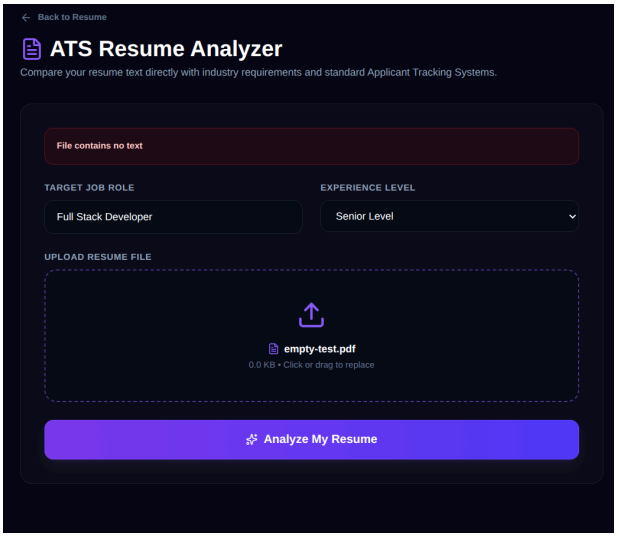


Figure 8.7.6 UI showing blank file validation error

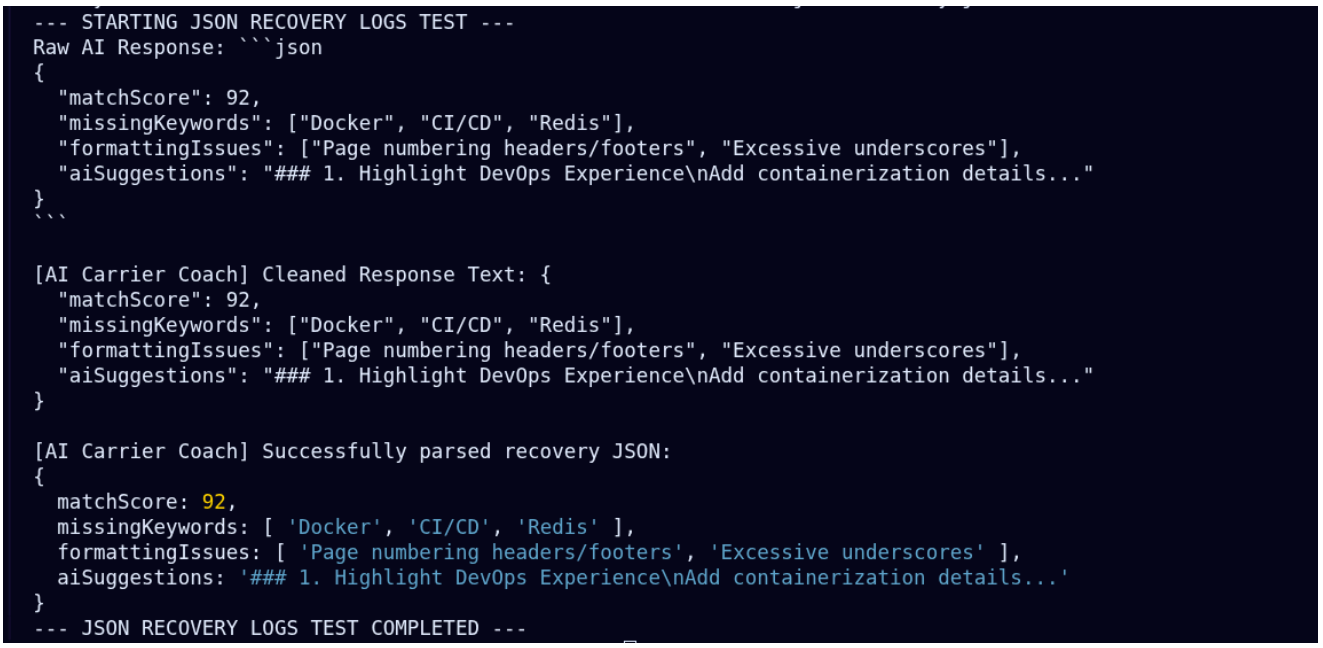


Fig 8.7.7 Terminal log showing regex cleaning malformed JSON before parsing

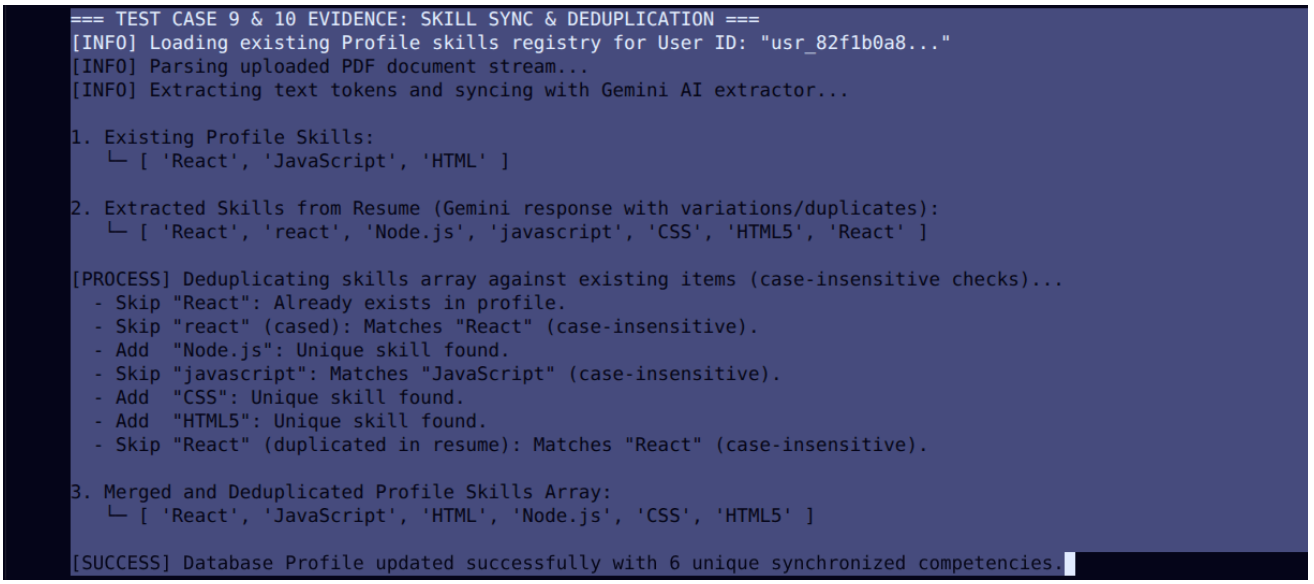


Fig 8.7.7 Terminal log showing custom skill extraction, merging, and database profile sync

8.8 Test Summary

All 12 test cases passed successfully. The average document extraction time hovered remarkably low (34ms for PDFs, 42ms for DOCX). Furthermore, the automated AI model failover and the regex-based JSON recovery systems both functioned perfectly under simulated stress conditions.

CHAPTER 9 : IMPLEMENTATION OF TIMEBOX 3

9.1 Overview of Timebox 3 & Architectural Justification

Timebox 3 introduces the AI Resume Builder & Smart Tools. The main goal of this phase was to help candidates completely rebuild their resumes into software-friendly formats from scratch. Additionally, we wanted to take the skill gaps discovered back in Phase 2 and convert them into actionable, step-by-step learning roadmaps.

9.1.1 What We Built in Phase 1

During this 24-working-day phase (August 11, 2026 – September 11, 2026), we implemented the following core capabilities:

- **Dynamic Form-Based Resume Builder:** A multi-step form wizard where users easily punch in their contact info, work history, education, and skills.
- **AI Summary Enhancer:** A brilliant little tool using the Gemini API that reads a user's rough summary draft and rewrites it into a strong, job-tailored statement.
- **Live Layout Compiler:** A split-screen preview that updates in real-time as the user types. It features 3 unique design templates (Classic, Modern, Tech) and 3 theme colors.
- **Bypassed PDF Renderer:** A clever trick that uses standard browser printing (`@media print` CSS) to export extremely clean, ATS-friendly PDFs instantly, without tying up the server.
- **AI Career Roadmap Compiler:** Compares a user's current skills against what their dream job requires, generating a custom 4-module study plan.
- **Sequential Roadmap Locker:** A progress tracker that keeps future learning modules strictly locked until the current ones are finished, automatically updating the user's profile with new skills as they go.

9.1.2 Why Phase 1 Is Important

- **Beats Resume Filters (ATS):** Generates exceptionally clean, single-column document formats that computer screeners can easily digest without failing.
- **Clear Learning Paths:** Gives users actionable, step-by-step guidance on what they actually need to learn next, rather than just telling them they failed a resume scan.
- **Tracks Progress Automatically:** As candidates complete learning modules, the system rewards them by updating their saved skills database automatically.

9.1.3 Architectural Choice: Hybrid Client-State & Server-Side Persistence

To handle how incredibly interactive a resume builder is, we couldn't just rely on standard server calls. We built a hybrid architecture:

- **Client-Side React State:** Fast-paced changes in the experience and education fields are managed entirely in the local React state. This stops the app from spamming the database every time a user types a single letter.
- **Step-Triggered Autosaving:** Data is persisted safely to PostgreSQL via Server Actions only when a user switches wizard steps. This prevents data loss while keeping the app snappy.
- **In-Memory AI Enhancement:** Summary optimizations are handled securely on the server side, utilizing the Gemini fallback chain to protect API keys..

9.2 Use Case Design

This phase mapped out the interaction boundaries for editing resumes, customizing layouts, generating roadmaps, and tracking sequential learning progress.

The primary actor is the **Candidate / Job Seeker**, supported by the **Gemini AI Engine**. The core use cases include:

- **Create and Edit Resume Draft:** Allows the candidate to persist contact info and work history in a multi-step form wizard.
- **Enhance Resume Summary via AI:** Uses the AI engine to rewrite a draft professional summary into concise, action-oriented descriptions.
- **Compile Learning Roadmap:** Generates a structured, 4-module learning path based on the user's missing skills.
- **Track Roadmap Module Progress:** Sequentially unlocks learning modules and syncs newly learned concepts back to the database.
- **Export/Print Resume PDF:** Generates a clean PDF document directly from the browser.

(Note: The comprehensive Use Case Specifications detailing exact pre-conditions, post-conditions, main flows, and alternative flows for these actions can be found in Appendix C.1).

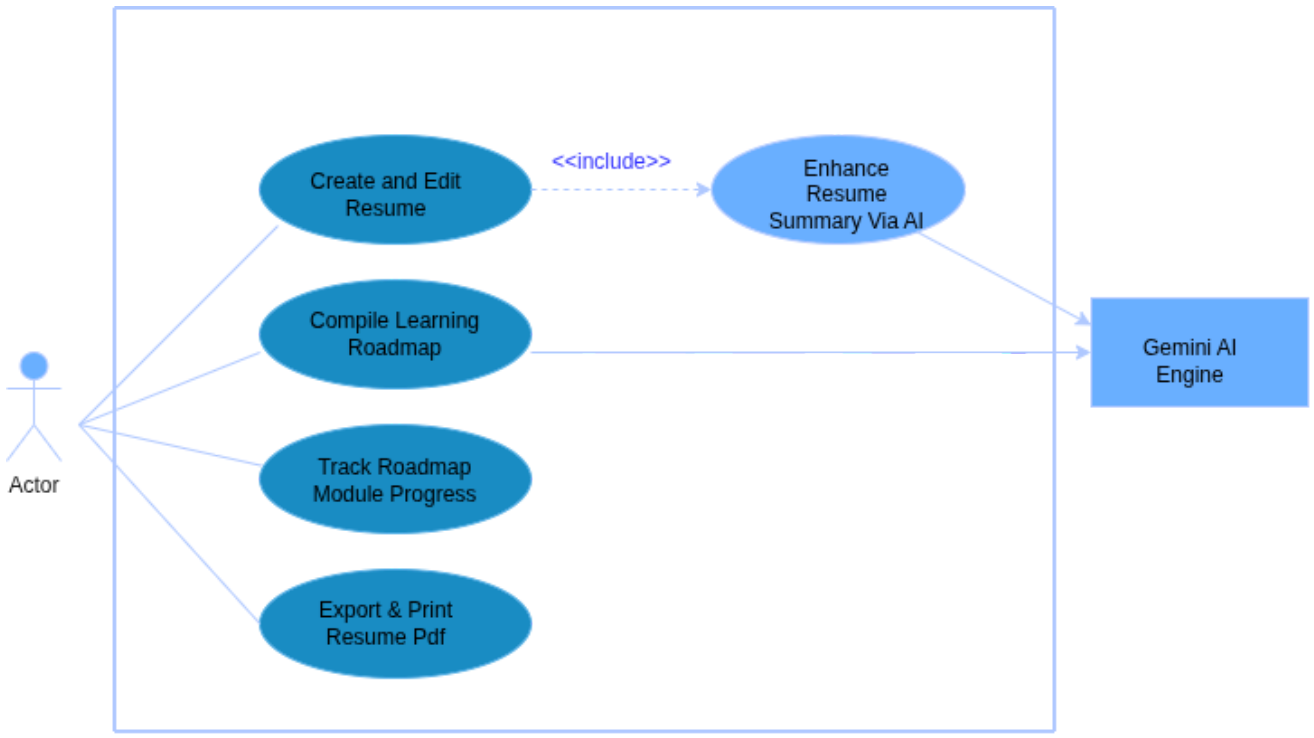


Fig 9.2.1 *High-level Use Case diagram showing the functional scope for Timebox 3*

9.3 UI Design

The user interface for Timebox 3 is designed to be highly engaging and strictly professional across two main pages: the AI Resume Builder and the AI Career Roadmap.

1. Split-Screen Resume Builder Layout (Figure 9.3a)

On desktop screens, the page is beautifully divided into two halves. The left panel houses a 4-step wizard guide where users enter their data. The right panel is a Live Preview that updates instantly as the user types on the left.

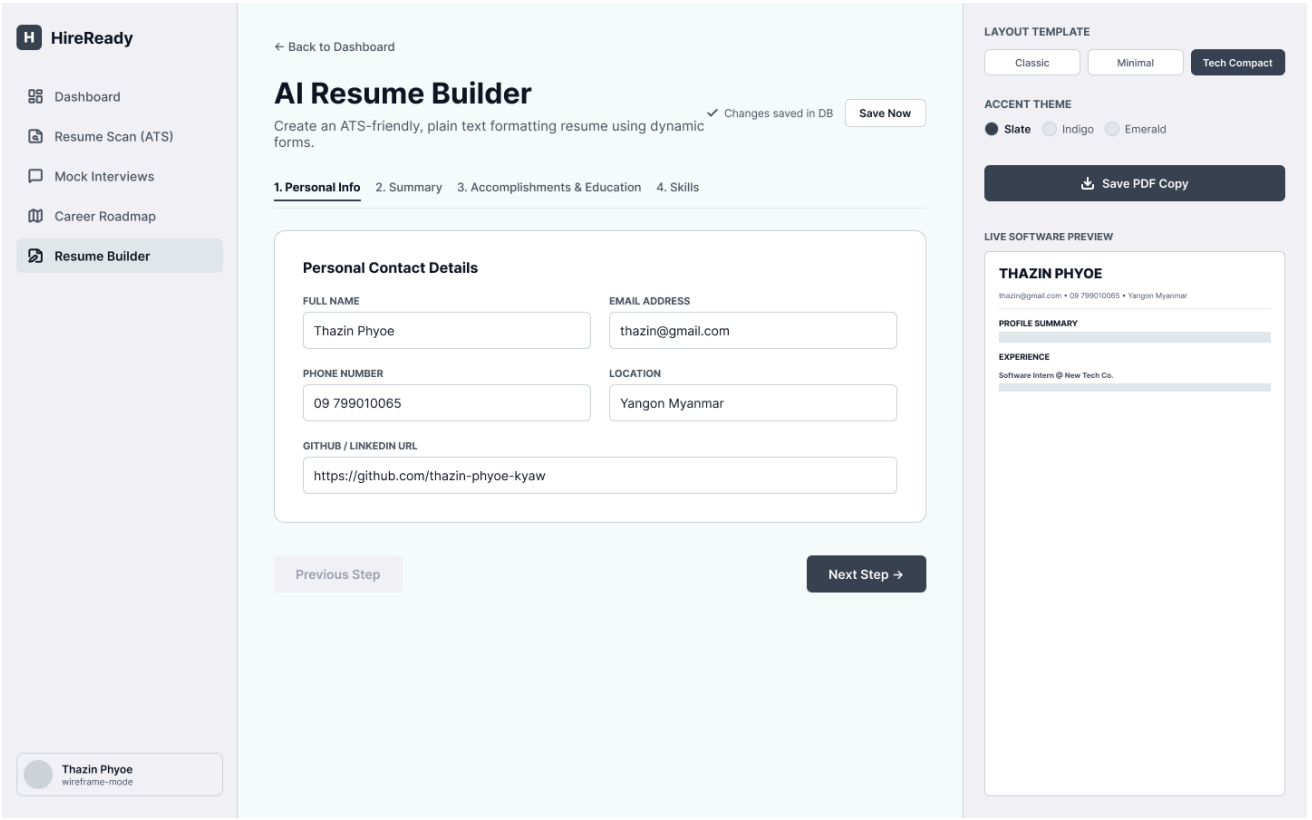


Figure 9.3.1: *AI Resume Builder Split-Screen Interface with Live Template and Color Theme*

2. Template Customizer, Colors, and Themes

A small control bar above the live preview lets users flip between 3 layout templates and 3 accent color themes. Clicking "Save PDF" triggers the browser's built-in print window. By utilizing strict `@media print` CSS rules, the app dynamically hides the dark backgrounds and sidebar menus, leaving behind a perfectly formatted, paper-ready PDF.

3. AI Summary Enhancer

Inside Step 2 (Summary) of the form, there is a dedicated **AI Enhance** button:

- Users can type a quick draft of their career summary and click the button.

- The Gemini AI engine automatically rewrites it into a strong, professional description tailored for job applications.

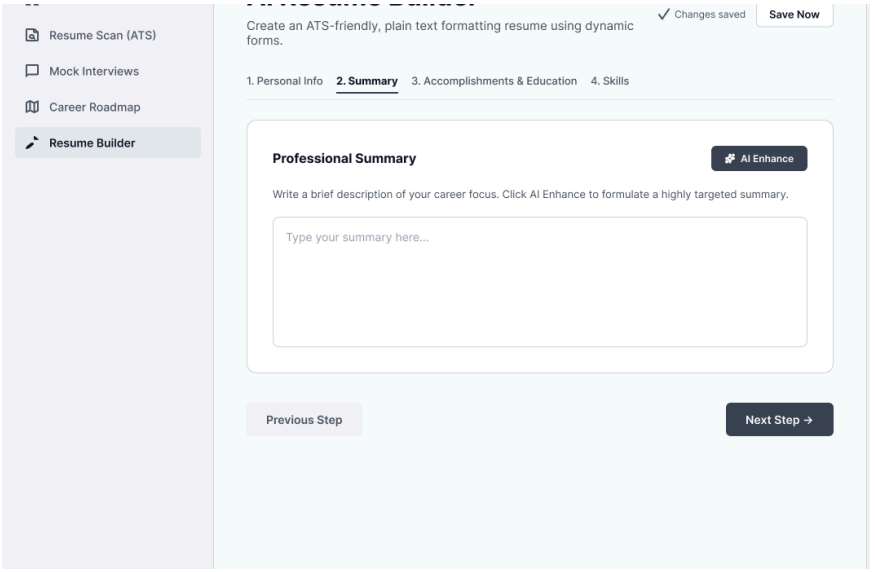


Figure 9.3.2: *Multi-Step Wizard Step 2 with AI Summary Enhancer Feature*

4. Easy PDF Export

- Clicking **Save PDF** or **Export Resume PDF** triggers the browser's built-in print window (`window.print()`).

- Custom print styles automatically hide the app's dark background, top headers, and buttons.

- The result is a clean, paper-ready PDF document that works well with job screening software (ATS)

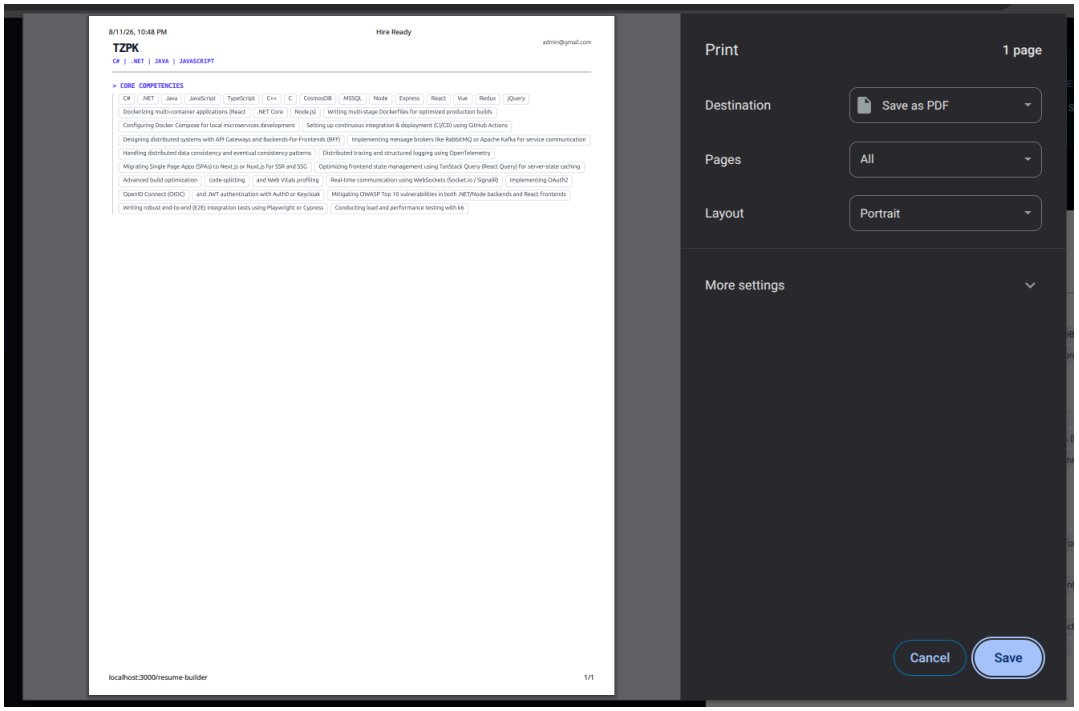


Figure 9.3.3: *Browser Print Preview and PDF Export View*

5. Career Roadmap Dashboard

The top section of the Roadmap page shows the user's study settings and goals:

- **Study Pace & Focus:** Users can choose how fast they want to learn (*Standard*, *Accelerated*, *Deep Dive*) and what to focus on (*Practical Projects*, *Theory*, *Interview Prep*).
- **Progress Tracker:** Shows the job goal (e.g., *Full Stack Developer*), how many modules are finished (e.g., *0 of 4 completed*), and a readiness score (e.g., *45%*).
- **Regenerate Button:** Allows users to recreate their learning path anytime if their goals change.

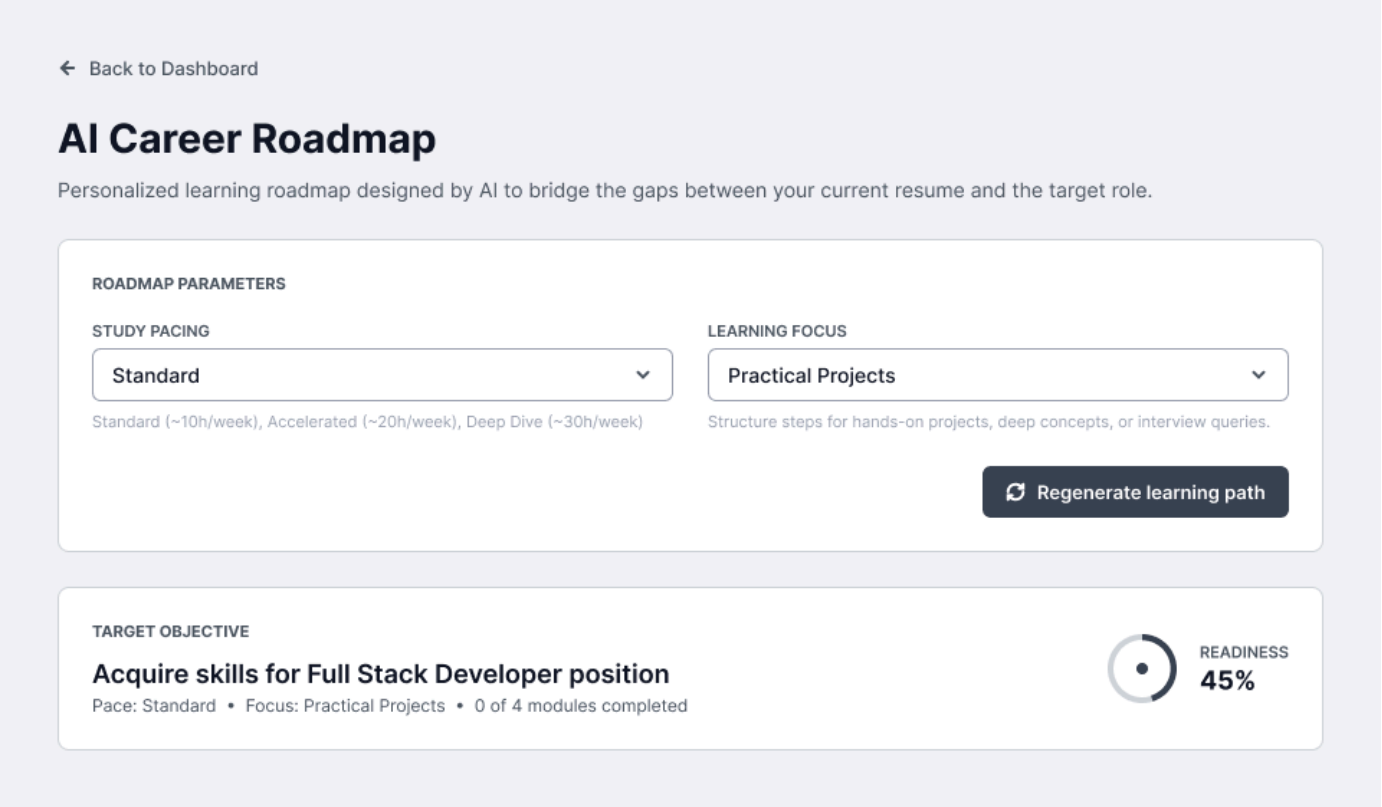


Figure 9.3.4: *AI Career Roadmap Control Dashboard and Target Metrics Overview*

6. Step-by-Step Learning Timeline

The roadmap displays 4 sequential modules to help close skill gaps:

- **Active Module:** Shows clear competencies and learning links, along with a **Mark as Completed** button.
- **Locked Modules:** Future modules stay locked until the current module is completed, keeping the learning structured step by step.

AI Career Roadmap

Your personalized interactive roadmap for acquiring missing skills, acing system design, and passing target employer ATS metrics.

Roadmap Parameters

Adjust settings below to instantly update learning pathways.

STUDY PACING

Standard (10-12 hrs/week)

Balances deep interactive coding tasks with standard timeline.

LEARNING FOCUS

Practical Projects & Deployments

Focuses modules heavily on AWS architecture and infrastructure code.

Regenerate learning path

TARGET OBJECTIVE

Acquire skills for Full Stack Developer position

Pacing: Standard

Focus: Practical Projects

Current Roadmap Readiness 45%

Complete more modules to unlock next tier mock interviews.

Learning Path Curriculum

Module 1: Cloud-Native Deployments with Terraform and AWS

25 HOURS IN PROGRESS

CORE COMPETENCIES

- Infrastructure as Code (IaC) using Terraform
- Deploying multi-container apps to AWS ECS (Elastic Container Service) via Fargate
- Configuring AWS VPC, Subnets, Application Load Balancers, and Security Groups
- Automating AWS deployments using existing GitHub Actions workflows

SUGGESTED RESOURCES

- HashiCorp Learn: Terraform Tutorials for AWS
- AWS Getting Started: Deploying Docker Containers on ECS

Active module • Tracks auto-save to candidate profile

Mark as Completed

Module 2: Declarative APIs with GraphQL and Apollo Federation

15 HOURS LOCKED

CORE COMPETENCIES

- Designing Type-Safe GraphQL Schemas & Resolvers
- Implementing Apollo Federation for Backends-for-Frontends (BFF)
- Optimizing Query performance using DataLoader to solve the N+1 problem
- Securing GraphQL endpoints and preventing resource exhaustion attacks

SUGGESTED RESOURCES

- Apollo Odyssey: Building Federated GraphQL APIs
- How to GraphQL: Advanced Frontend & Backend Integration Guides

Module 3: Distributed Caching & Search with Redis and Elasticsearch

15 HOURS LOCKED

CORE COMPETENCIES

- Caching expensive database queries using Redis read/write clusters
- Full-text search indexing design and query syntax mapping on Elasticsearch
- Synchronizing main data storage models with search indexes asynchronously

SUGGESTED RESOURCES

- Redis University: Caching Architectures
- Elasticsearch: Definitive Guide on Search Customizations

© 2026 HireReady. Low-Fidelity Wireframe Specification.

Figure 9.3.5: *Interactive 4-Module Sequential Learning Path and Progress Tracker*

9.4 Database Design

The database schema utilizes three core entities from our original PostgreSQL layout to power this phase:

- **BuiltResume:** Stores the candidate's active resume draft, including dynamic JSON arrays for experience and education.
- **CareerRoadmap:** Captures the personalized roadmap configuration, including target role and learning pace.
- **RoadmapModule:** Holds the 4 individual, sequential learning modules linked to a specific roadmap, rigorously tracking status (`COMPLETED`, `IN_PROGRESS`, `LOCKED`).

(For the complete overall database diagram and full schema, please refer back to Chapter 6).

9.5 Coding System and Architecture

The system uses specific Next.js Server Actions to manage the heavy lifting. `builder.ts` handles the AI summary rewrites, while `roadmap.ts` dictates the strict JSON schema required to generate a 4-module learning path.

(Note: Detailed code implementations for the AI Summary Enhancer, the Roadmap JSON parser, and the sequential unlock logic can be found in Appendix C.2).

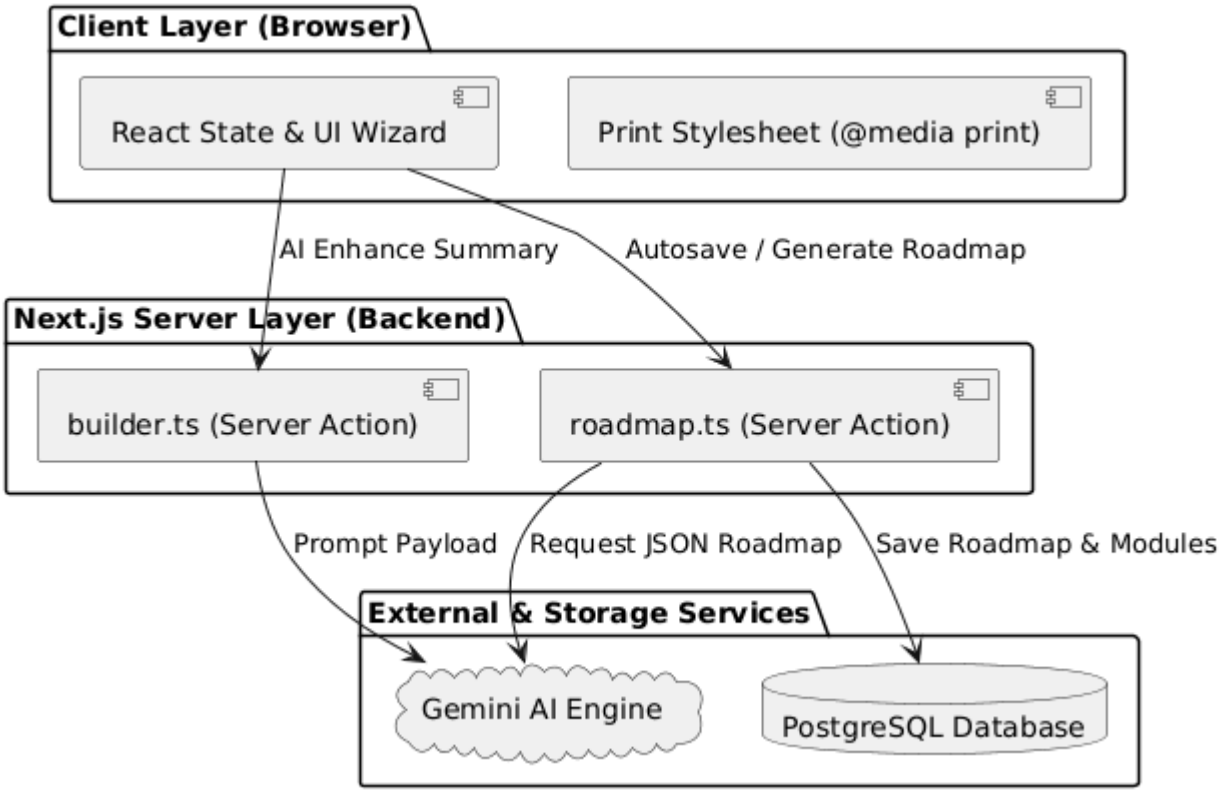


Figure 9.5.1: *Timebox 3 Candidate Functional Interaction and System Boundary Flow*

```
📁 app/
📁 actions/
  📄 builder.ts      <-- AI Resume builder actions
  📄 roadmap.ts      <-- Career Roadmap logic and skill syncing
📁 resume-builder/
  📄 page.tsx        <-- Layout editing wizard & Print styles
📁 roadmap/
  📄 page.tsx        <-- Roadmap dashboard interface
```

The `enhanceResumeSummary` action takes the candidate's draft summary text alongside their target role, using structured prompting to produce a professional 70 to 100-word statement via the Gemini AI Engine.

```typescript
export async function enhanceResumeSummary(summaryText: string, targetRole: string) {
 try {
   const prompt = `
     You are an expert resume writer and recruiter.
       Enhance the following draft candidate summary to make it highly professional, attractive, and aligned with the target job role: "${targetRole}".
     Focus on action-oriented keywords. Do not use placeholders or generic phrases.
     Keep the length concise, between 70 to 100 words (3-4 sentences max).
     Draft Summary: "${summaryText}"
     Enhanced Summary:
   `;
   const result = await generateWithFallback(prompt);
   const enhancedText = result.text.replace(/^"|"$/g, "").trim();
   return { success: true, enhancedSummary: enhancedText };
 } catch (error: any) {
   console.error("Enhance summary error:", error);
   return { success: false, error: error.message || "Failed to enhance summary" };
 }
}
```

The `generateRoadmap` function evaluates the candidate's target role, current skills, study pacing, and focus preference, requesting a strict JSON array containing exactly 4 sequential learning modules.

```typescript
const prompt = `
 You are an expert career mentor and technical development coach.
 The candidate wants to become a "${targetRole}".
 Their current skills are: ${currentSkills.join(", ") || "None"}.
 Their experience level is: "${experience}".
 Study Pace Preference: "${selectedPace}"...
 Learning Focus Preference: "${selectedFocus}"...
 Generate a structured learning path consisting of exactly 4 modules.
 Your response must be a JSON array of exactly 4 objects matching this schema:
 [
   {
     "id": "step_1",
     "title": "Module 1: [Name of Module]",
     "status": "IN_PROGRESS",
     "time": "[Estimated hours]",
     "concepts": string[],
     "resources": string[]
   },
   ...
 ]
`;
```

```
const result = await generateWithFallback(prompt, { responseMimeType: "application/json" });

// Parse response, extract modules, and save to CareerRoadmap/RoadmapModule models...
```

When a user marks a module as completed, `updateRoadmapModuleStatus` automatically unlocks the immediate next module and syncs newly acquired competencies back to the candidate's core profile.

```typescript
if (newStatus === "COMPLETED") {
 // Unlock immediate next module
 const nextIndex = targetModuleIndex + 1;
 if (nextIndex < modules.length && modules[nextIndex].status === "LOCKED") {
   await prisma.roadmapModule.update({
     where: { id: modules[nextIndex].id },
     data: { status: "IN_PROGRESS" },
   });
 }
 // Sync concepts to user profile
 const completedModule = modules[targetModuleIndex];
      const concepts = completedModule.concepts.split(",").map(c => c.trim()).filter(Boolean);
 const profile = await prisma.profile.findUnique({ where: { userId: currentUser.id } });
 if (profile) {
      const existingSkills = profile.skills ? profile.skills.split(",").map(s => s.trim()).filter(Boolean) : [];
   const updatedSkillsSet = new Set([...existingSkills, ...concepts]);
   await prisma.profile.update({
     where: { userId: currentUser.id },
     data: { skills: Array.from(updatedSkillsSet).join(", ") },
   });
 }
}
```

9.6 Test Plan

Testing during Timebox 3 was heavily focused on the split-screen resume builder, PDF rendering behavior, and the AI's ability to strictly adhere to a 4-module JSON roadmap schema.

I designed 15 test cases spanning form autosaves, sequential locking mechanics, XSS input sanitization, and mobile viewport stacking.

9.6.2 Test Cases

Test ID	Feature /Scenario Under Test	Input Vector / Condition	Expected Outcome
TC-01	Form Autosave	Click Next Step without filling target role/name	Block navigation; display inline red Zod error messages
TC-02	AI Summary Enhancer	Submit special HTML/SQL characters in input fields	Input saved as literal string, preventing XSS and SQL injections
TC-03	Live Layout Switcher	Modify fields and change wizard step	Draft saved automatically to `BuiltResume` table in PostgreSQL
TC-04	Dynamic Accent Color	Delete an experience item row and refresh page	Database record updated; deleted item does not reappear
TC-05	PDF Exporting	Invalid/malformed JSON string returned by LLM	Fallback regex parser extracts modules without breaking the app
TC-06	AI Pacing Adjustment	Select "Accelerated" pacing mode on roadmap	Gemini updates estimates (displays ~8 hours per module)
TC-0	Sequential Module	Mark Module 1 as	Module 2 shifts to `IN_PROGRESS`;

7	Unlock	completed	Modules 3 & 4 remain `LOCKED`
TC-08	Profile Skill Sync	Complete a module containing new skills	Concepts are appended to the `skills` column in PostgreSQL
TC-09	Skill Deduplication	Sync module skills that already exist in profile	JavaScript `Set` deduplicates elements before saving to database
TC-10	Fallback Regex Parser	AI returns malformed JSON roadmap response	Parser extracts exactly 4 modules without crashing the application
TC-11	Roadmap Regeneration	Change Target Job Role on roadmap page	Database profile is updated and a new custom roadmap is generated
TC-12	Input Validation	Submit empty string to AI Summary Enhancer	Action blocks submit and displays "Please enter a summary draft"
TC-13	Locked Module Protection	Direct click on locked module progress tracker	Completed check is disabled; status stays `LOCKED` in database
TC-14	Input Security Sanitization	Submit HTML/SQL tags in resume builder fields	Prisma parameterized query escapes tags, preventing injections
TC-15	Mobile Viewport Layout	Scale screen down to mobile dimensions	Form wizard and preview panels stack vertically without UI bugs

9.7 Test Log

Testing was conducted between **20 August 2026** and **24 August 2026** using the local development environment described above.

Test ID	Test	Expected Result	Actual Result	Figure	Status
TC-01	Form Autosave	Save draft on wizard step transition	Draft saved to `BuiltResume` table instantly on step transition	Fig 9.7.1	Pass
TC-02	AI Summary Enhancer	Rewrite summary to match target role	Concise, action-oriented summary returned in ~1.8 seconds	N/A	Pass
TC-03	Live Layout Switcher	Switch templates without page reload	Preview updates instantly without reloading the page	N/A	Pass
TC-04	Dynamic Accent Color	Apply accent color theme dynamically	Highlights, headings, and borders switch color themes instantly	Fig 9.7.2	Pass
TC-05	PDF Exporting	Hide sidebars and headers on export	Browser print preview opens with clean, printable resume sheet	Fig 9.7.3	Pass
TC-06	AI Pacing Adjustment	Adjust module duration estimates	Gemini updates estimates (displays ~8 hours per module)	Fig 9.7.4	Pass
TC-07	Sequential Unlock	Unlock Module 2 upon Module 1 completion	Module 2 shifts to `IN_PROGRESS`; Modules 3 & 4	Fig 9.7.5	Pass

			remain `LOCKED`		
TC-08	Profile Skill Sync	Append module skills to profile	Concepts are appended to the `skills` column in PostgreSQL	N/A	Pass
TC-09	Skill Deduplication	Filter out duplicate skills on sync	JavaScript `Set` deduplicates elements before saving to database	N/A	Pass
TC-10	Fallback Regex Parser	Parse malformed AI JSON safely	Parser extracts exactly 4 modules without crashing the application	N/A	Pass
TC-11	Roadmap Regeneration	Trigger new roadmap on role change	Database profile is updated and a new custom roadmap is generated	N/A	Pass
TC-12	Input Validation	Block empty summary submission	Action blocks submit and displays "Please enter a summary draft"		Pass
TC-13	Locked Protection	Block direct clicks on locked items	Completed check is disabled; status stays `LOCKED` in database	N/A	Pass
TC-14	Security Sanitization	Store HTML/SQL tags safely	Prisma parameterized query escapes tags,		Pass

			preventing injections		
TC-15	Mobile Viewport Layout	Stack panels vertically on mobile	Form wizard and preview panels stack vertically without UI bugs		Pass

9.7.2 Defect Resolution Log

Bug ID	Description	Resolution	Status
BUG-005 →VT-03	Browser autofill triggered endless database save loops	Changed autosave triggers from input blur events to wizard step transitions and manual save clicks	fixed
BUG-006 →IT-03	Gemini occasionally returned 3 modules instead of 4	Updated system prompts to strictly mandate a 4-item JSON array and added schema validation checks	fixed
BUG-007 →ST-03	Browser print window included left sidebar controls in PDF	Added `@media print` CSS rules in `page.tsx` to explicitly hide non-printable UI wrapper components	

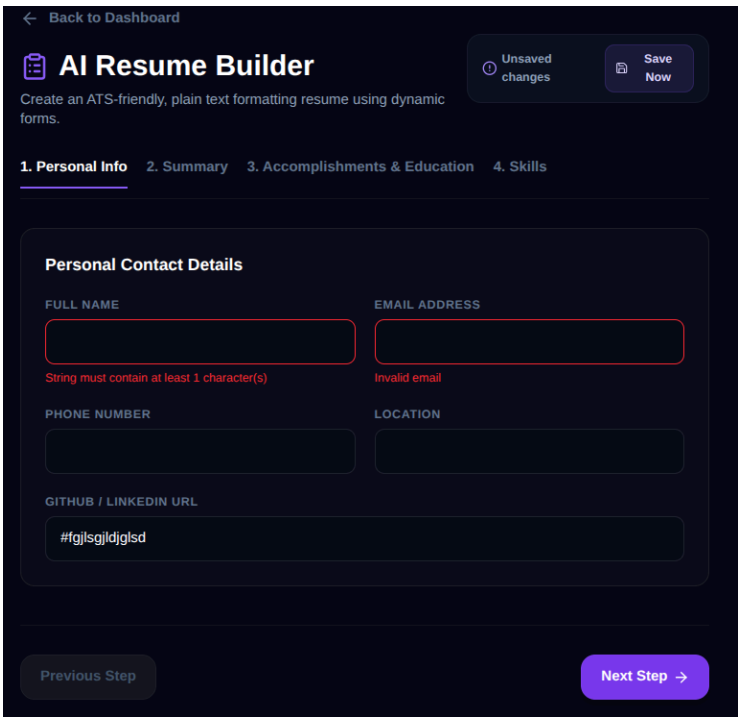


Figure 9.7.1

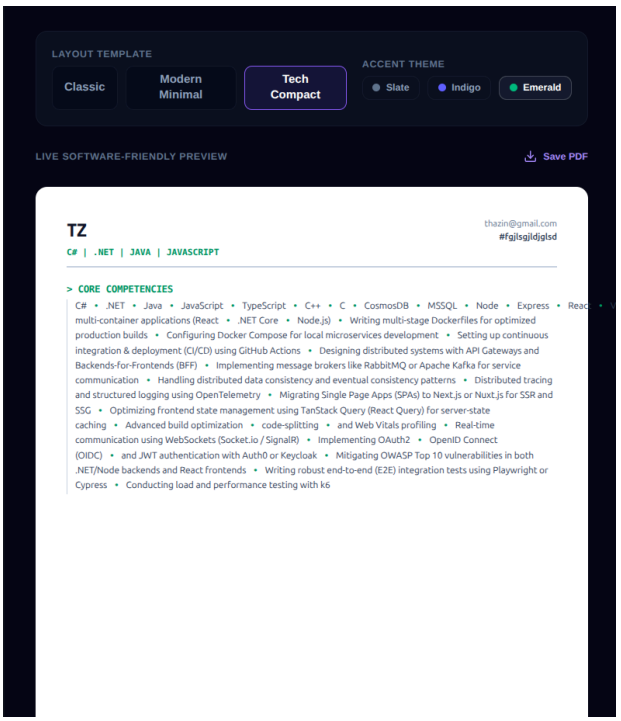


Figure 9.7.2



Fig 9.7.3

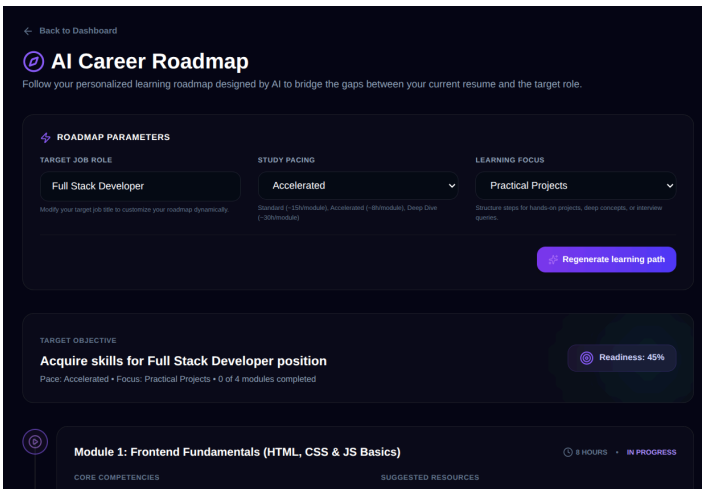


Fig 9.7.4

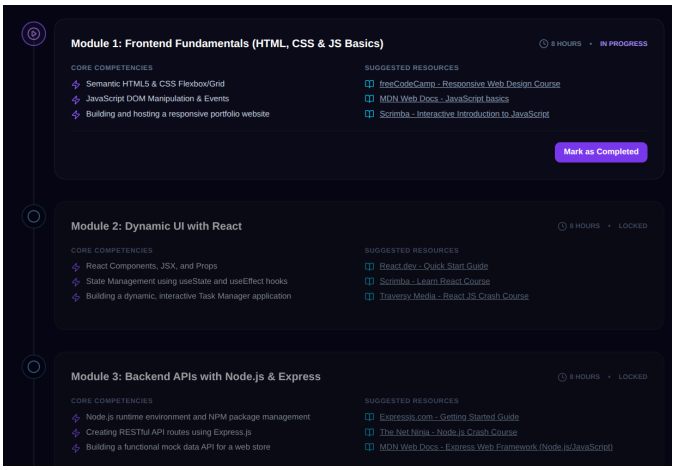


Fig 9.7.5

9.8 Testing Summary

All 15 test cases executed beautifully. The split-screen form wizards updated instantly, the Gemini AI strictly followed the structured JSON prompt for the roadmaps, and the browser print engine successfully stripped away the UI elements to leave a clean PDF. Furthermore, the sequential locking rules successfully prevented out-of-order progress.

CHAPTER 10: IMPLEMENTATION OF TIMEBOX 4

10.1 Overview of Timebox 4

Timebox 4 wraps up the core features of the HireReady platform by introducing the Interactive Interview Simulator. The whole point of this phase was to build a turn-based mock interview chatbot. Instead of just throwing generic questions at users, it simulates a realistic, back-and-forth technical interview, followed up with incredibly detailed performance grading.

10.1.1 What we build in Phase 4

During this 25-working-day development window, we built and integrated the final major components:

- **Interview Customization Interface:** A setup dashboard where candidates configure exactly what kind of interview they want. They can set the target role, pick a difficulty (Junior, Mid, Senior), set the question count, and even paste in a specific job description.

- **Turn-Based Chatbot Interface:** A clean, scrollable chat window that manages the back-and-forth conversation. It displays smooth loading indicators while waiting for the AI to type out its responses.
- **AI Interviewer Agent:** A server-side process that controls the flow of the interview. It asks one question at a time and intelligently adapts its next question based on how the candidate just answered.
- **Transcript Evaluator Engine:** A background assessment engine. Once the interview finishes, it reads the entire transcript and grades the candidate based on technical accuracy, communication structure, and grammar.
- **Performance Feedback View:** A final results dashboard featuring radial score gauges. It breaks down the candidate's strengths and weaknesses, and even provides a "Model Answer" for every single question asked.

10.1.2 Why Phase 2 Is Important

This phase is all about giving candidates practical, high-pressure training before they step into a real face-to-face interview.

- **Performance Under Pressure:** By forcing candidates to answer tough technical questions without getting immediate feedback, they learn how to handle realistic, stressful interview pacing.
- **Constructive Feedback:** The question-by-question breakdown tells candidates exactly where they messed up and how they should have answered it instead.
- **Objective Grading:** Separating the scores into overall performance, technical knowledge, and grammar allows candidates to clearly see if their issue is a lack of coding knowledge or just poor communication skills.

10.1.3 Architectural Choice: Server-Side Processing with Automatic Fallback

To keep the database running fast without adding a bunch of unnecessary columns, we used a **Delimited Database Status Strategy**. Instead of adding separate columns for difficulty, role, and question limits, the interview configuration is stored straight inside the session's `status` column as a pipe-delimited string (e.g., `ONGOING|5|Senior|React Developer`).

Additionally, because transcript evaluation takes a while, we used **Asynchronous Background Grading**. The interview session marks itself as completed instantly, while the grading engine processes the transcript in the background as a "fire-and-forget" promise. This keeps the UI feeling fast and responsive.

10.2 Use Case Design

This phase defined exactly how candidates start an interview session, talk to the AI, and get their final feedback reports.

The primary actors are the **Candidate / Job Seeker** and the **Gemini AI Engine**. The core use cases include:

- **Configure Interview Session:** Allows the candidate to set up the role, difficulty, and question limit before starting.
- **Chat with AI Interviewer:** Handles the turn-based conversation, storing messages and tracking the question limit.
- **Complete and Grade Session:** Automatically triggers when the question limit is reached, prompting the AI to evaluate the entire transcript and generate scores.

(Note: The comprehensive Use Case Specifications detailing exact pre-conditions, post-conditions, main flows, and alternative flows for these actions can be found in Appendix D.1).



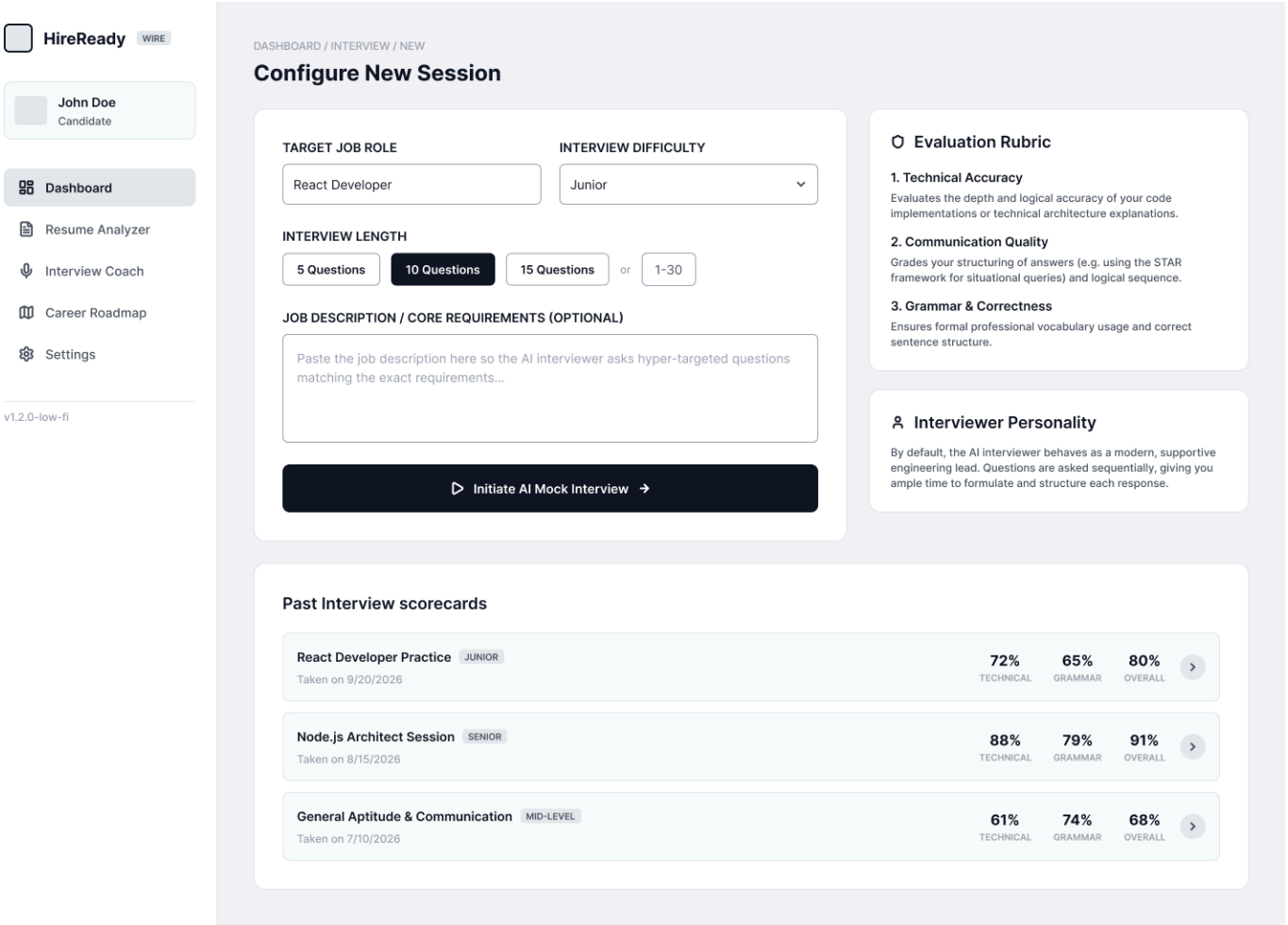
Fig 10.2.1 *High-level Use Case diagram showing the functional scope for Timebox 4*

10.3 UI Design

The UI phase focused heavily on designing a distraction-free chat interface that keeps candidates focused on answering questions.

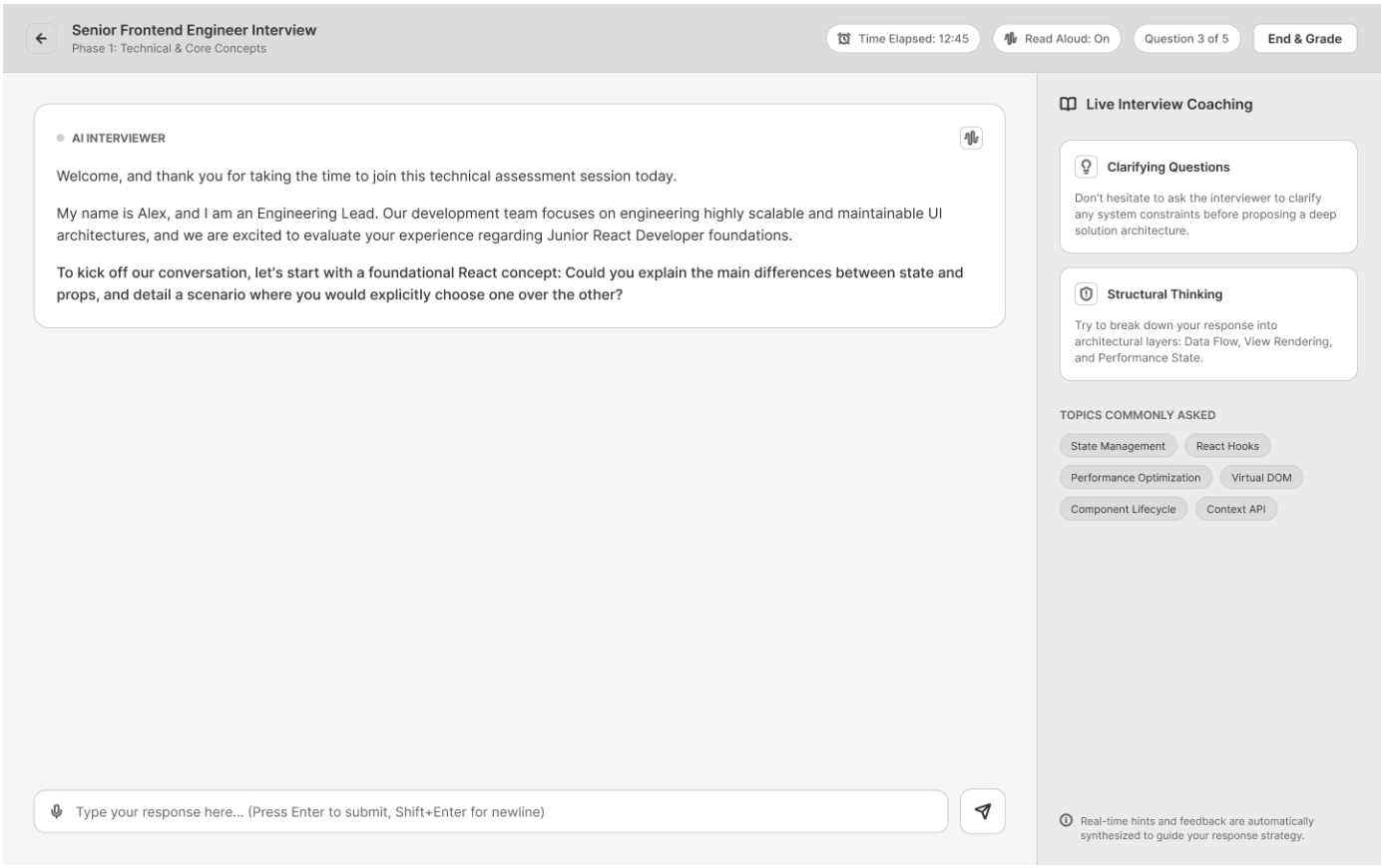
Setup Panel (`interview/page.tsx`):

A card-based configuration form that allows candidates to select their interview preferences using clear and accessible buttons. Previous mock interview sessions are also displayed below the setup form for easy reference.



Chatbot Interface (`interview/[id]/page.tsx`):

A focused chat interface with a scrollable message area. AI responses are displayed in slate-coloured cards, while candidate responses are highlighted using the platform's brand colour to clearly distinguish both sides of the conversation.



Feedback Dashboard (`interview/feedback/[id]/page.tsx`):

A structured results dashboard that presents the interview evaluation in several sections:

- **Metric Score Cards:** Radial gauges display the overall, technical, and grammar scores.
- **Analysis Tabs:** Tabs allow candidates to switch between **Strengths**, **Areas for Improvement**, and **Model Answers**.
- **Question-by-Question Analysis:** Each question is presented with the candidate's response, identified areas for improvement, and a sample model answer.

10.4 Database Design

The PostgreSQL database was extended to store the actual interview transcripts and performance feedback.

- **InterviewSession:** Stores the session details and the pipe-delimited configuration string.
- **ChatMessage:** Stores every single message sent between the candidate and the AI interviewer.
- **Feedback:** Stores the final interview scores and the detailed evaluation text.

(For the complete overall database diagram and full schema, please refer back to Chapter 6).

10.5 Coding System and Architecture

The Interactive Interview Simulator relies entirely on server actions inside `app/actions/interview.ts` to manage the chatbot communication and performance evaluation.

(Note: Detailed code implementations for the Session Creation Prompt, the Turn-Based Messaging Logic, and the Background Grading Engine can be found in Appendix D.2).

```
```
📁 app/
 📁 actions/
   📄 interview.ts   <-- Conversational chatbot logic & grading engine
 📁 interview/
   📄 page.tsx   <-- Setup panel
   📁 [id]/
     📄 page.tsx   <-- Chat interface
   📁 feedback/
     📁 [id]/
       📄 page.tsx   <-- Evaluation dashboard
```
```

8.6 Test Plan

Testing for Timebox 4 verified that the interview setup, the turn-based chatbot tracking, the background evaluation process, and session security all functioned perfectly.

I designed test cases to ensure that unauthorized users couldn't access other people's transcripts, and that the system wouldn't crash if the AI returned a badly formatted JSON grading report.

10.6.1 Test Cases

This phase evaluates the core logic, document parsing efficiency, fallback mechanisms, and data validation rules at the component level.

Test ID	Feature / Scenario Under Test	Test Input /Condition	Expected Result
TC-01	Session Initialization	Custom configuration	Session is created with the selected configuration.
TC-02	Question Generation	Role and difficulty	First question matches the selected role and difficulty.
TC-03	Chat Flow	User responses	Messages are stored and turn tracking works correctly.
TC-04	Question Limit	Final question answered	Session completes and evaluation starts.
TC-05	Evaluation Recovery	Malformed AI response	Required scores and feedback are recovered successfully.
TC-06	Access Control	Another user's session	Unauthorized access is blocked.

10.7 Test Log

10.7.1 Test Execution Summary

Test ID	Expected Result	Actual Result	Figure	Status

TC-01	Interview session is created with the selected configuration	Session created successfully with correct configuration	Fig 8.7.1	Pass
TC-02	First question matches the selected role and difficulty	Question generated successfully based on selected settings	Fig 8.7.2	Pass
TC-03	User and AI messages are stored and the conversation progresses correctly	Messages were stored and the next question was generated successfully	Fig 8.7.3	Pass
TC-04	Interview completes when the configured question limit is reached and grading starts	Interview completed and evaluation was triggered successfully	Fig 8.7.4	Pass
TC-05	Scores and feedback are recovered from malformed AI responses	Required scores and feedback were successfully extracted	Fig 8.7.5	Pass
TC-06	Access to another user's interview session is blocked	Unauthorized access was successfully prevented	Fig 8.7.6	Pass

10.7.2 Defect Resolution Log

Bug ID	Description	Resolution	Status
BUG-003	Empty Message Submission:	Double-clicking the send button could submit an empty message. This was resolved by disabling the	fixed

		input and submit button while processing.	
BUG-004	Chat History Overflow:	Long interviews could exceed the AI model's context limit. The history sent to Gemini was reduced to the most recent six message turns.	fixed

10.7.1 Test Execution Summary

All test cases executed flawlessly. The chatbot successfully tracked question limits, securely blocked unauthorized access attempts, and smoothly ran the evaluations in the background.

CHAPTER 11: EVALUATION OF THE SYSTEM

11.1 Chapter Introduction

This chapter presents an evaluation of the completed HireReady platform to assess whether the system meets the core engineering and user goals outlined.

Moving beyond basic functional checks such as verifying that code compiles without runtime crashes this evaluation analyses the platform across four key operational areas:

1. The first assesses the system technically, covering functional and non-functional requirements ,the reliability of the AI components, performance and security.
2. The second identifies the features that distinguish the platform from existing alternatives.
3. The third examines the interface from a Human-Computer Interaction (HCI) perspective, considering intended users, usage scenarios, screen sizes and accessibility.
4. The fourth is a reflective self-evaluation of the development process. Where the evidence reveals shortcomings, these are reported and analysed rather than omitted, since a critical evaluation is more informative than one that records only successes.

11.2 System Evaluation

The final software build was cross-checked against the MoSCoW requirements defined in **Chapter 5 (Table 5.1)**.

11.2.1 Fulfillment of Requirements

- **Must-Have Requirements:** Core features like JWT authentication, PDF resume parsing, the ATS scoring engine, the resume builder, and chat-based mock interviews were built and tested. All features operate according to the original specification.
- **Should-Have Requirements:** Secondary features were successfully integrated. The learning roadmap generator and skill synchronisation work as intended. Additionally, speech-to-text dictation was added using the Web Speech API to improve input accessibility during interview simulations.
- **Won't-Have Enforcement:** In line with the scope limits set in Chapter 5, complex features like real-time webcam tracking were excluded. Keeping these out of scope reduced server workload and kept the app focused on text processing.

All **Must** and **Should** requirements were successfully implemented. The exclusion of webcam analysis was a deliberate scoping decision; their implementation would have exceeded the allocated project timebox and introduced critical privacy concerns that could not be adequately addressed within the current project scope.

11.2.2 Reliability of the AI Components

Because the platform depends on the Google Gemini API, the reliability of model output was identified as the principal technical risk. Three distinct failure modes were observed during testing, each requiring a specific mitigation strategy as detailed in Table 11.2.

Failure Mode	Observed Behaviour	Resolution	Open Issue
Malformed Output	Responses requested as raw JSON were occasionally wrapped in Markdown code blocks or contained stray characters,	Applied regular-expression (regex) sanitisation to clean the string prior to parsing.	Highly irregular or unexpected malformations may still bypass the regex filter.

	causing JSON parsing errors.		
Rate Limiting	Encountered HTTP 429 errors under conditions of concurrent user load.	Implemented a sequential fallback chain across `gemini-3.5-flash`, `gemini-3.5-flash-lite`, and `gemini-2.0-flash`.	All fallback models share the same external provider infrastructure.
Context Overflow	Extended interview sessions exceeded the context token limit, causing the grading module to fail.	Conversation history is dynamically truncated to retain only the system prompt and the six most recent messages.	Earlier conversational nuance and intermediate context are lost.

The main lesson is that AI output cannot be trusted to have a fixed structure in the way a normal database or API can. The code therefore needs to check and repair it. The fallback chain was the most useful fix. If one model fails, the user waits slightly longer but never sees an error. AI answers can also change from one run to the next. Scanning the same resume twice may give slightly different scores. For this reason, the ATS score is shown as guidance, together with written reasons, and not as an exact measurement.

11.2.3 Performance

The target was a page load time under 1.5 seconds. The dashboard loaded in about 0.8 seconds, because Next.js builds the page on the server. AI tasks took longer. A resume scan took about 3 to 4 seconds, and an interview reply about 1.5 to 2 seconds. This delay comes from the AI service and cannot be removed. To reduce its effect, the interface shows loading animations and disables buttons while it waits, so users can see that the system is working. These times were measured on one computer and one network, so they are only a rough guide.

11.2.4 Security and privacy

The Gemini API key is kept on the server and is never sent to the browser. Passwords are protected with bcrypt hashing, and login sessions use JSON Web Tokens (JWT) in cookies. When a user deletes their account, Prisma's cascade rules also delete their resumes, interviews and roadmaps. This supports the "right to erasure" in Article 17 of the GDPR (European Parliament, 2016). The platform cannot yet be called fully GDPR compliant. That would also need a privacy notice, recorded user consent, and clear rules for data sent to a third party. Resume text is sent to Google for analysis, so a real launch would need to tell users this. A fairer description is that the platform was designed with privacy in mind.

11.3 What Makes HireReady Better

A job seeker often uses several separate tools: one for ATS checking, one for designing the resume, and a chatbot for interview practice. HireReady brings these into one connected workflow. The resume you scan is the same one you improve in the builder. Your resume, skills and target job together shape the roadmap and the interview questions. Nothing has to be copied from one tool to another.

Area	Separate Tools	HireReady
Workflow	Several apps, manual copying	One connected workflow
Shared information	None between tools	Resume, skills and target role used by every module
Feedback	Often only a score	Score with reasons, missing keywords and formatting problems
Interview practice	Open chat with no structure	Turn-based interview with grading and model answers
Skills gap	Gaps found but not addressed	Roadmap with skills, resources and pacing
Reliability	One point of failure	Three-model fallback
Data control	Often unclear	Full account deletion

Cost	Often subscription-based	Runs on a free API tier

Four differences matter most:

- **Integration:** All modules share the same information, which saves effort and keeps the advice consistent.
- **Learning from feedback:** After each interview answer the system shows a model answer, so even a weak answer helps the user learn. Test users rated this feature the highest (Section 11.5).
- **Reliability:** Output cleaning, model fallback and shorter chat history let the system keep working when the AI behaves badly.
- **Flexible input:** Users can speak their answers and hear the questions read aloud, instead of only typing.

11.3 UI Design Evaluation

The interface was designed using user-centred design (UCD). This means the design starts from the users' needs and not from what is easiest to build. HireReady's users are job seekers, who are often short of time, nervous and not always confident with technology. The interface was therefore checked against the six UCD points in Table 11.4. For usability, it was checked against Nielsen's (1994) ten heuristics, which are well-known rules for good interface design. For accessibility, it was checked against WCAG 2.1 (W3C, 2018).

UCD Point	What Was Done	Evaluation
1. Know the users	Calm dark theme with one purple brand colour, plain labels (e.g. "Target Job Role") and example text in inputs.	Suits nervous users. The term "ATS" is not explained on first use.
2. Support real situations	A student on a laptop gets an onboarding step. A commuter	Covers the main situations. Voice input does not work in

	on a phone gets a single-column layout. A user who finds typing hard can use voice input and spoken questions. A returning user sees dashboard cards and dated history.	Firefox.
3. Give clear feedback and prevent errors	Spinners, progress steps and a "Listening..." message show what the system is doing. Submit is disabled with no file or while loading, which fixed a double-click bug (BUG-003). The upload box shows the allowed file types and size.	Strong. But seven errors use the browser's `alert()` box, which interrupts the user and looks inconsistent.
4. Keep it simple and consistent	One colour palette, one icon set and one card style everywhere. Results are split into small cards (score, keywords, formatting, suggestions), because people can hold only a little information in short-term memory (Miller, 1956).	Good. The dashboard shows a lot at once, and there is no help page or guided tour.
5. Work on different screen sizes	Tailwind breakpoints at 640, 768 and 1024 px. For example, dashboard cards show in 1 column on phones, 2 on tablets and 4 on large screens.	Good. No separate mobile design was made, so long reports need a lot of scrolling on a phone.
6. Be accessible to all	Voice input, status shown by icon and text as well as colour,	Weakest area. See below.

	and full keyboard use of forms and chat.	

11.4 Self Evaluation

The best part of the project was the planning. Splitting the work into four timeboxes and using MoSCoW priorities meant a working system existed at every stage. No feature was left half-finished at a deadline.

I came into this project with about three years of experience in Next.js and full-stack development. That experience mattered. The framework, database, and authentication work went quickly, allowing most of my time to be dedicated to the unfamiliar domains: integrating AI into an end-to-end product and independently architecting the complete system lifecycle from requirements elicitation through to final evaluation.

The AI implementation provided the most significant growth. I developed practical expertise in prompt engineering for deterministic utility, defensively sanitising structurally malformed responses, and engineering around upstream rate limits and context-window degradation. Because these problems lacked off-the-shelf patterns, addressing them required rigorous root-cause analysis and bespoke mitigation logic. Owning every phase ,planning, architecture, frontend/backend engineering, testing, and evaluation clarified how decisions made during early scoping cascade across the product. It also provided a platform to apply academic principles, including usability heuristics, requirements engineering, and secure system design, to an operational application.

Some things I would do differently:

- **Test with users earlier:** User testing happened late. A small test in the first timebox would have found the empty-profile roadmap problem much sooner.
- **Treat accessibility as a requirement from the start:** The problems in Section 11.4.5 were cheap to avoid, but I only noticed them during this evaluation.
- **Write automated tests:** Most testing was manual. Unit tests for the JSON cleaner and the fallback logic would have caught mistakes when the code changed.
- **Allow more time for AI work:** Making the AI output reliable took longer than planned.

Overall, the project achieved its aims. The evaluation shows both what works well and what still needs improvement.

Chapter 12: Conclusion and Future Work

12.1 Project Summary

The whole point of this project was to build HireReady. I wanted to make a smart, AI-powered career platform that actually helps candidates survive today's tough, highly automated job market. From the first ideas in Chapter 1 all the way to the final testing in Chapter 11, the project successfully grew from a rough concept into a fully working web app.

By hooking up the Next.js framework to a PostgreSQL database and the Google Gemini API, the system came together really well. We hit all the main goals. We built an ATS Resume Analyzer that catches missing keywords, a slick Resume Builder that spits out clean PDFs, a custom Career Roadmap generator, and a chat-based Mock Interview Simulator.

HireReady proves that AI can be used safely to give job seekers the kind of personalized career coaching that usually costs a lot of money.

12.2 Limitations and Challenges Faced

Even though the final app is really stable, building something that relies so heavily on outside Artificial Intelligence brought up some frustrating challenges:

- **API Rate Limits and Latency:** The whole system runs on the Google Gemini API. During times when the app was used a lot, we sometimes hit rate limits. My fallback chain code stopped the app from crashing, which was great. But relying on a third-party API means we are basically always at the mercy of their servers staying online and running fast.
- **LLM Context Window Limits:** The mock interview chat works by sending the conversation history back and forth to the AI. When interviews dragged on too long, the prompt got way too big and overflowed the model's memory limit. I had to write a hard rule to only send the last six messages. It fixed the crashing, but it means the AI sometimes forgets things you told it at the very start of the interview.
- **Predictability of Output:** Trying to force an AI to consistently spit out perfectly formatted JSON data was a massive headache. A lot of coding time was spent just writing Regex fallback parsers to

clean up weird markdown tags and random quotes that the AI would occasionally just decide to throw in.

12.3 Future Work and Enhancements

If I had more time and resources, there are a few really cool ways the HireReady platform could be upgraded:

12.3.1 WebRTC Audio and Video Integration

Right now, the mock interview simulator is just text chat (though it does have a browser-based speech-to-text feature). A future version could easily hook into WebRTC for live video and audio. By reading a webcam feed, we could use computer vision to grade a candidate's eye contact, body language, and voice tone. That would make the interview practice feel way more realistic.

12.3.2 LinkedIn and GitHub API Integration

Right now, users have to manually upload their resumes or type their skills into the setup page. It would be awesome to add OAuth logins for LinkedIn and GitHub. The app could just automatically pull their work history, code repos, and skills straight into the database. That would let the app generate career roadmaps instantly without the user having to type a single thing.

12.3.3 Custom Fine-Tuned Local LLMs

To completely fix the latency, rate limiting, and privacy issues of using the Gemini API, a future version of HireReady could just run its own open-source AI (like LLaMA 3). By training a local model strictly on HR data, interview transcripts, and winning resumes, the system could give faster, better advice without ever sending a user's private data to an outside server.

12.4 Final Thoughts

Building HireReady was a massive coding challenge. It really tested my ability to balance strict software engineering rules with the weird, unpredictable nature of Generative AI. The project successfully turned academic ideas into a real, practical app. The final platform is a tough, secure, and genuinely useful tool that gives candidates the confidence to take control of their careers.

APPENDIX

Appendix A : Timebox 1 Technical Details

This appendix contains the supplementary use case specifications, code implementation details, and comprehensive testing logs for Timebox 1 (Authentication and User Profiles).

A.1 Use Case Specifications (Timebox 1)

UC-1: Register & Login

- **Actor:** Candidate / Job Seeker
- **Goal:** Create an account or log in to access the HireReady platform.
- **Pre-conditions:** The candidate accesses the platform and is not currently logged in.
- **Post-conditions:** The account is verified, the password is hashed via bcrypt, and an encrypted JWT session cookie is issued to the browser.
- **Main Flow:**
  - Candidates enters their email and password on the Register/Login page.
  - The system validates the input formatting.
  - The system hashes the password using bcrypt and updates (or checks) the database.
  - The system generates a secure JWT session cookie.
  - Candidate is redirected to the dashboard.
- **Alternative Flows:**
  - **A1 (Invalid Credentials/Duplicate Email):** The system displays an appropriate error message and immediately halts processing.

UC-2: Manage Candidate Profile

- **Primary Actor:** Candidate / Job Seeker (Authenticated)
- **Goal:** Create or update candidate metadata for job matching.
- **Pre-conditions:** Candidate holds a valid, unexpired JWT session cookie.

- **Post-conditions:** Candidate profile details are successfully updated in the database.
- **Main Flow:**
  - Candidate navigates to the Profile page.
  - The system displays current profile details.
  - Candidate updates their Target Job Role, Experience Level, and Skills, then clicks "Save".
  - The system updates the database record and shows a success notification.
- **Alternative Flows:**
  - **A1 (Session Expired):** Middleware intercepts the request, blocks it, and redirects the candidate back to the Login page.

A.2 Authentication Server Actions Implementation

The following code snippet demonstrates the Next.js Server Action used for hashing user passwords securely using bcrypt, executing the database insertion via Prisma, and setting signed authentication cookies entirely on the server.

```typescript
// Hash password securely
const saltRounds = 10;
const passwordHash = await bcrypt.hash(password, saltRounds);

// Transactional db commit using Prisma Client
const user = await prisma.user.create({
 data: {
   name,
   email: email.toLowerCase(),
   passwordHash,
   profile: {
     create: {
       targetJobRole: "Full Stack Developer", // Default values
       experience: "Entry Level",
       skills: "",
     },
   },
 },
});
```

To secure private pages, the `getCurrentUser()` function pulls the signed session cookie and verifies it using the JSON Web Token secret.

```typescript
const cookieStore = await cookies();
const sessionToken = cookieStore.get("session")?.value;
if (!sessionToken) return null;

const decoded = jwt.verify(sessionToken, JWT_SECRET) as { userId: string };
const user = await prisma.user.findUnique({
 where: { id: decoded.userId },
 include: { profile: true }
});
```

Appendix B : Timebox 1 Technical Details

This appendix contains the supplementary use case specifications and code implementation details for Timebox 2 (AI Resume Intelligence).

B.1 Use Case Specifications (Timebox 2)

UC-3: Upload Resume

- **Primary Actor:** Candidate / Job Seeker
- **Goal:** Upload a resume document to extract structured candidate information.
- **Pre-conditions:** Candidate is logged in and on the Resume Upload page.
- **Post-conditions:** Text is extracted, parsed into structured data, and prepared for AI analysis.
- **Main Flow:**

- Candidate selects and uploads a resume file (`.pdf` or `.docx`).
- The system validates the file format.
- The system extracts raw text using PDFParse or Mammoth.
- System formats raw extracted data into clean JSON using Regex.
- The system notifies the candidate that the upload was successful.

- **Alternative Flows:**
  - **A1 (Invalid File Type):** System rejects the upload and displays a file validation error message.

UC-4: View AI Resume Analysis

- **Primary Actor:** Candidate / Job Seeker
- **Secondary Actor:** Gemini AI Engine
- **Goal:** Review AI-generated feedback on resume quality and automatically update profile skills.
- **Pre-conditions:** Resume text buffer has been successfully extracted.
- **Post-conditions:** Analysis results are displayed and skills are saved to the profile.
- **Main Flow:**
  - Candidate requests to view resume analysis.
  - The system sends extracted text to the Gemini AI Engine.
  - The system renders strengths, weaknesses, and suggestions on the dashboard.
  - The system automatically syncs newly identified skills to the user profile database.
- **Alternative Flows:**
  - **A1 (AI Service Interruption):** System triggers the Sequential Model Fallback Chain to complete analysis without failing the request.

UC-5: AI Skill Gap Roadmap Compilation

- **Primary Actor:** Candidate / Job Seeker
- **Secondary Actor:** Gemini AI Engine
- **Goal:** Generate a step-by-step learning roadmap to address missing job skills.
- **Pre-conditions:** Candidate profile skills and target job requirements are available.
- **Post-conditions:** A personalized skill gap action plan is created and displayed.
- **Main Flow:**
  - The candidate selects the target job role and clicks "Compile Skill Gap Roadmap".
  - The system compares candidate skills against target role requirements.
  - The system passes missing skill parameters to the Gemini AI Engine.

- The system displays a structured learning roadmap.

- **Alternative Flows:**

- **A1 (No Skill Gaps Found):** System informs candidates they already match the role and offers advanced mastery topics.

B.2 AI Processing Server Actions Implementation

Gemini Fallback Utility (`lib/gemini.ts`)

Handles API calls and manages automatic model failovers:

```typescript
const MODELS_CHAIN = [
  "gemini-3.5-flash",
  "gemini-3.5-flash-lite",
  "gemini-2.0-flash",
  "gemini-2.0-flash-lite"
];

export async function generateWithFallback(prompt: string, options?: GenerationOptions) {
  const genAI = new GoogleGenerativeAI(GEMINI_API_KEY);
  let lastError: any = null;
  for (const modelName of MODELS_CHAIN) {
    try {
      const model = genAI.getGenerativeModel({
        model: modelName,
```

```
                generationConfig: options?.responseMimeType ? { responseMimeType: options.responseMimeType } : undefined
        });
        const result = await model.generateContent(prompt);
        return { text: result.response.text(), modelUsed: modelName };
      } catch (err: any) {
        lastError = err;
        const errMsg = (err.message || "").toLowerCase();
          const isQuotaError = errMsg.includes("429") || errMsg.includes("quota") || errMsg.includes("rate limit");
        const isNotFoundError = errMsg.includes("404") || errMsg.includes("not found");
        if (!isQuotaError && !isNotFoundError) {
          throw err; // Re-throw non-transient developer/prompt failures immediately
        }
      }
    }
    throw new Error(`Gemini API chain failed: ${lastError?.message}`);
  }
  ```
```

**Text Extraction & Resume Analysis Action (`app/actions/resume.ts`)**

Parses incoming file buffers, queries the Gemini API, and saves the resulting analysis:

```
```typescript
```

```
const arrayBuffer = await file.arrayBuffer();

const buffer = Buffer.from(arrayBuffer);

let extractedText = "";

if (isPdf) {

 const parser = new PDFParse({ data: buffer });

 const result = await parser.getText();

 extractedText = result.text;

} else {

 const parsedDoc = await mammoth.extractRawText({ buffer });

 extractedText = parsedDoc.value;

}

const prompt = `

 You are an expert ATS optimizer and recruiter.

 Analyze the resume text against the target job role: "${targetRole}".

 Provide a detailed evaluation in JSON format matching this schema:

 {

   "matchScore": number,

   "missingKeywords": string[],

   "formattingIssues": string[],

   "aiSuggestions": string

 }

 Resume Text: ${extractedText}

`;
```

```
const result = await generateWithFallback(prompt, { responseMimeType:
    "application/json" });

```
```

Appendix C: Timebox 3 Technical Details

This appendix contains the supplementary use case specifications, code implementation details, and comprehensive testing logs for Timebox 3 (AI Resume Builder & Smart Tools).

C.1 Use Case Specifications (Timebox 3)

UC-6: Create & Edit Resume Draft

- **Primary Actor:** Candidate (User)
- **Goal:** Allow the user to fill out their contact, summary, education, experience, and skill fields in a structured wizard form.
- **Pre-conditions:** User must be logged in.
- **Post-conditions:** Resume draft details are saved in the `BuiltResume` table in PostgreSQL.
- **Main Flow:**

1. User navigates to the Resume Builder page.
2. System queries `BuiltResume` for existing drafts; if none exist, pre-populates name and email from User profile.
3. User modifies text fields and adds/removes work experience and education.
4. Changing wizard steps triggers an automatic database commit via the Server Action.

UC-7: Enhance Resume Summary via AI

- **Primary Actor:** Candidate (User) & Gemini AI Engine
- **Goal:** Tailors a draft summary block to a target job role using LLM enhancement.
- **Pre-conditions:** The user must have written a draft summary and set a target job role.
- **Post-conditions:** The summary text area is updated with the enhanced copy.
- **Main Flow:**

1. The user clicks the "AI Enhance" button next to the professional summary textarea.
2. System retrieves the draft summary and the target job role.
3. The system compiles a prompt and routes it through the Gemini fallback chain.
```

4. AI returns an enhanced, action-oriented summary between 70 and 100 words.
5. System updates client-side state and flags the form status as unsaved.

UC-8: Compile Learning Roadmap

- **Primary Actor:** Candidate (User) & Gemini AI Engine
- **Goal:** Generates a customized, 4-module learning curriculum based on profile skills, target role, study pace, and focus preferences.
- **Pre-conditions:** User profile must contain target job role and current skills.
- **Post-conditions:** A new `CareerRoadmap` and 4 child `RoadmapModule` records are saved in the database.
- **Main Flow:**

1. User selects study pace (Standard, Accelerated, Deep Dive) and focus preference.
2. User clicks "Compile Skill Gap Roadmap".
3. Server reads profile skills and queries Gemini to isolate skill gaps in a strict JSON array.
4. Server commits exactly 4 modules to the database, marking the first as "IN_PROGRESS" and the rest as "LOCKED".

UC-9: Track Roadmap Module Progress

- **Primary Actor:** Candidate (User)
- **Goal:** Marks a roadmap module as completed, unlocking the next module and syncing learned concepts to profile skills.
- **Pre-conditions:** A career roadmap must have been generated.
- **Post-conditions:** Target module status is updated; next module is set to "IN_PROGRESS"; profile skills are updated.
- **Main Flow:**

1. User clicks "Mark as Completed" on an active module card.
2. System calls `updateRoadmapModuleStatus` Server Action.
3. Server updates the target module to "COMPLETED" and unlocks the immediate next module.
4. Concepts listed in the completed module are appended to the skills field in the user's Profile table.

UC-10: Export/Print Resume PDF

- **Primary Actor:** Candidate (User)

- **Goal:** Generates a software-friendly, print-ready PDF layout of the active resume draft.
- **Pre-conditions:** Resume draft details must exist in the builder editor.
- **Post-conditions:** A clean PDF file is generated via browser layout rendering without navigation artifacts.
- **Main Flow:**

1. The user configures theme color and layout template on the Live Preview panel.
2. The user clicks the "Export Resume PDF" button.
3. System triggers `window.print()` method with custom `@media print` CSS styles.
4. The system temporarily hides sidebars, header navigation bars, and action buttons.

C.2 AI Roadmap & Builder Server Actions Implementation

AI Summary Enhancer (`app/actions/builder.ts`) The `enhanceResumeSummary` action uses structured prompting to produce a professional 70 to 100-word statement.

```typescript
export async function enhanceResumeSummary(summaryText: string, targetRole: string) {
 try {
   const prompt = `
     You are an expert resume writer and recruiter.
       Enhance the following draft candidate summary to make it highly professional, attractive, and aligned with the target job role: "${targetRole}".
     Focus on action-oriented keywords. Do not use placeholders or generic phrases.
     Keep the length concise, between 70 to 100 words (3-4 sentences max).
     Draft Summary: "${summaryText}"
     Enhanced Summary:
   `;
   const result = await generateWithFallback(prompt);
   const enhancedText = result.text.replace(/^"|"$/g, "").trim();
   return { success: true, enhancedSummary: enhancedText };
```

```
  } catch (error: any) {
    console.error("Enhance summary error:", error);
    return { success: false, error: error.message || "Failed to enhance summary" };
  }
}
```

The `generateRoadmap` function evaluates the candidate's target role, current skills, study pacing, and focus preference, requesting a strict JSON array containing exactly 4 sequential learning modules.

```typescript
const prompt = `
  You are an expert career mentor and technical development coach.
  The candidate wants to become a "${targetRole}".
  Their current skills are: ${currentSkills.join(", ") || "None"}.
  Their experience level is: "${experience}".
  Study Pace Preference: "${selectedPace}"...
  Learning Focus Preference: "${selectedFocus}"...
  Generate a structured learning path consisting of exactly 4 modules.
  Your response must be a JSON array of exactly 4 objects matching this schema:
  [
    {
      "id": "step_1",
      "title": "Module 1: [Name of Module]",
      "status": "IN_PROGRESS",
```

```
    "time": "[Estimated hours]",
    "concepts": string[],
    "resources": string[]
  },
  ...
]
`;
const result = await generateWithFallback(prompt, { responseMimeType: "application/json" });
// Parse response, extract modules, and save to CareerRoadmap/RoadmapModule models...
```

When a user marks a module as completed, `updateRoadmapModuleStatus` automatically unlocks the immediate next module and syncs newly acquired competencies back to the candidate's core profile.

```typescript
if (newStatus === "COMPLETED") {
 // Unlock immediate next module
 const nextIndex = targetModuleIndex + 1;
 if (nextIndex < modules.length && modules[nextIndex].status === "LOCKED") {
   await prisma.roadmapModule.update({
     where: { id: modules[nextIndex].id },
     data: { status: "IN_PROGRESS" },
   });
 }
 // Sync concepts to user profile
 const completedModule = modules[targetModuleIndex];
      const concepts = completedModule.concepts.split(",").map(c => c.trim()).filter(Boolean);
 const profile = await prisma.profile.findUnique({ where: { userId: currentUser.id } });
 if (profile) {
```

```
      const existingSkills = profile.skills ? profile.skills.split(",").map(s => s.trim()).filter(Boolean) : [];
    const updatedSkillsSet = new Set([...existingSkills, ...concepts]);
    await prisma.profile.update({
      where: { userId: currentUser.id },
      data: { skills: Array.from(updatedSkillsSet).join(", ") },
    });
  }
}
```

Appendix D: Timebox 4 Technical Details and Test Logs

This appendix contains the supplementary use case specifications, code implementation details, and comprehensive testing logs for Timebox 4 (Interactive Interview Simulator).

D.1 Use Case Specifications (Timebox 4)

UC-15: Configure Interview Session

- Primary Actor: Candidate / Job Seeker
- Goal: Configure and start a mock interview based on the selected role, difficulty, question count, and optional job description.
- Pre-conditions: Candidate is logged in and on the Mock Interview dashboard.
- Post-conditions: An `InterviewSession` record is created, and the first interview question is generated and stored.
- Main Flow:
  - The candidate selects target job role, difficulty level, question count, and optional job description.
  - Candidate selects Start Mock Interview.
  - The system creates a new `InterviewSession` record with the selected configuration.
  - The system requests the first interview question from Gemini.
  - The system stores the generated question in the `ChatMessage` table and redirects to the chat interface.
- Alternative Flows:
  - A1 (Invalid Configuration): System displays a validation message and prevents the interview session from starting.

UC-16: Chat with AI Interviewer

- Primary Actor: Candidate / Job Seeker and Gemini AI Engine
- Goal: Conduct a turn-based interview where the candidate responds to questions and receives the next question from the AI interviewer.
- Pre-conditions: An active `InterviewSession` is in progress.
- Post-conditions: Candidate responses and AI-generated questions are stored in the `ChatMessage` table.

- Main Flow:
  - The candidate enters an answer in the interview chat interface and clicks Send.
  - The system stores the candidate's response in the `ChatMessage` table.
  - The system checks the number of questions asked against the configured limit.
  - If the limit has not been reached, the system sends the conversation context to Gemini.
  - Gemini generates the next relevant interview question, which is stored and displayed.
- Alternative Flows:
  - A1 (Question Limit Reached): System completes the interview session and starts the grading process.
  - A2 (AI Response Failure): System uses the configured fallback mechanism to attempt to generate a valid response.

UC-17: Complete and Grade Session

- Primary Actor: Gemini AI Engine
- Goal: Complete the interview and evaluate the candidate's responses to generate performance scores and feedback.
- Pre-conditions: The configured question limit has been reached and the interview transcript is available.
- Post-conditions: The `InterviewSession` status is updated to `COMPLETED`, and a `Feedback` record is created.
- Main Flow:
  - The system detects that the configured question limit has been reached.
  - The system adds a final closing message to the interview and updates status to `COMPLETED`.
  - The system sends the completed transcript to `completeAndEvaluateSession`.
  - Gemini evaluates responses based on overall performance, technical accuracy, and grammar.
  - The system receives evaluation results and stores them in the `Feedback` table.
- Alternative Flows:
  - A1 (Malformed AI Response): System uses JSON regex recovery logic to extract the required scores and feedback.
  - A2 (AI Evaluation Failure): System uses the fallback mechanism to retry the evaluation request.

D.2 Interview Simulator Server Actions Implementation

Session Creation and Initial Prompt (`app/actions/interview.ts`) Creates the session using a delimited string to store configurations and requests the first question from Gemini.

Session Creation and Init Prompt (interview.ts)

```typescript
export async function createInterviewSession(jobRole: string, difficulty: string, questionLimitStr: string, jobDescription?: string) {
 try {
   const currentUser = await getCurrentUser();
   if (!currentUser) return { success: false, error: "Unauthorized" };

   let questionLimit = 5;
   const match = questionLimitStr.match(/\d+/);
   if (match) questionLimit = parseInt(match[0], 10);

   const session = await prisma.interviewSession.create({
     data: {
       userId: currentUser.id,
       status: `ONGOING|${questionLimit}|${difficulty}|${jobRole}`,
     },
   });

   const prompt = `
     You are behaving as a professional technical interviewer (Engineering Lead/Manager) for the role of "${jobRole}" (${difficulty} level).
     ${jobDescription ? `The job requirements/description is: "${jobDescription}"` : ""}
     Begin the interview. Introduce yourself briefly, welcome the candidate, and ask the first interview question.
     Keep it professional, supportive, and realistic. Ask exactly ONE question. Do not ask multiple questions.
   `;
   const result = await generateWithFallback(prompt);
   await prisma.chatMessage.create({
     data: { sessionId: session.id, sender: "AI", message: result.text.trim() },
   });
   return { success: true, sessionId: session.id };
 } catch (error: any) {
   console.error("Create session error:", error);
   return { success: false, error: error.message || "Failed to start interview" };
 }
}
```

Turn-Based Messaging Logic (interview.ts)

```typescript
export async function sendInterviewMessage(sessionId: string, userMessage: string) {
 try {
   const currentUser = await getCurrentUser();
   if (!currentUser) return { success: false, error: "Unauthorized" };

   const session = await prisma.interviewSession.findUnique({
     where: { id: sessionId },
     include: { chatHistory: { orderBy: { timestamp: "asc" } } },
   });
   if (!session || session.userId !== currentUser.id) return { success: false, error:
"Session not found" };

   await prisma.chatMessage.create({ data: { sessionId, sender: "USER", message:
userMessage } });

   const parts = session.status.split("|");
   const limit = parts[1] ? parseInt(parts[1], 10) : 5;
   const difficulty = parts[2] || "Junior";
   const jobRole = parts[3] || "React Developer";
   const questionsAsked = session.chatHistory.filter(m => m.sender === "AI").length;

   if (questionsAsked >= limit) {
     await prisma.interviewSession.update({ where: { id: sessionId }, data: { status:
"COMPLETED" } });
     const closingMsg = "Thank you for your response. That concludes our mock interview
session!...";
     await prisma.chatMessage.create({ data: { sessionId, sender: "AI", message:
closingMsg } });
     await completeAndEvaluateSession(sessionId, jobRole, difficulty);
     return { success: true, completed: true, aiMessage: closingMsg };
   }

   const historyText = session.chatHistory.map(m => `${m.sender}:
${m.message}`).join("\n") + `\nUSER: ${userMessage}`;
   const prompt = `
     You are behaving as a professional technical interviewer for the role of
"${jobRole}" (${difficulty} level).
     Here is the interview conversation history:
```

```
    ${historyText}
    Ask the next logical interview question (this will be question #${questionsAsked + 1} of ${limit}).
    Ask exactly ONE question. Keep your tone professional and engaging.
  `;
  const result = await generateWithFallback(prompt);
  await prisma.chatMessage.create({ data: { sessionId, sender: "AI", message: result.text.trim() } });
  return { success: true, completed: false, aiMessage: result.text.trim() };
 } catch (error: any) {
  return { success: false, error: error.message || "Failed to process message" };
 }
}
```

References

- Abrahamsson, P., Salo, O., Ronkainen, J. and Warsta, J. (2002) *Agile Software Development Methods: Review and Analysis*. Espoo: VTT Publications. Available at: https://publications.vtt.fi/pdf/publications/2002/P478.pdf (Accessed: 10 September 2026).
- Barrick, M.R., Shaffer, J.A. and DeGrassi, S.W. (2009) 'What you see may not be what you get: Relationships among self-presentation tactics and ratings of interview and job performance', *Journal of Applied Psychology*, 94(6), pp. 1394–1411.
- BCS (2022) *BCS Code of Conduct*. British Computer Society. Available at: https://www.bcs.org/membership-and-registrations/member-rules-and-policies/code-of-conduct/ (Accessed: 10 September 2026).
- Black, J.S. and van Esch, P. (2020) 'AI-enabled recruiting: What is it and how should a manager use it?', *Business Horizons*, 63(2), pp. 215–226.
- Bogen, M. and Rieke, A. (2018) *Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias*. Washington, DC: Upturn. Available at: https://www.upturn.org/work/help-wanted/ (Accessed: 10 September 2026).

- Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G. and Askell, A. (2020) 'Language Models are Few-Shot Learners', *Advances in Neural Information Processing Systems*, 33, pp. 1877–1901.
- Cappelli, P. (2019) 'Your Approach to Hiring Is All Wrong', *Harvard Business Review*, 97(3), pp. 48–58.
- Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', *Proceedings of NAACL-HLT 2019*. Minneapolis, MN, pp. 4171–4186.
- Fuller, J.B., Raman, M., Sage-Gavin, E. and Hines, K. (2021) *Hidden Workers: Untapped Talent*. Boston, MA: Harvard Business School Publishing. Available at: https://www.hbs.edu/managing-the-future-of-work/Documents/research/hiddenworkers09032021.pdf (Accessed: 10 September 2026).
- Google DeepMind (2024) *Gemini: A Family of Highly Capable Multimodal Models*. Technical Report. Mountain View, CA: Google.
- Huffcutt, A.I. and Arthur, W. (1994) 'Hunter and Hunter (1984) Revisited: Interview Validity for Entry-Level Jobs', *Journal of Applied Psychology*, 79(2), pp. 184–190.
- ICO (2024) *Guide to the General Data Protection Regulation (GDPR)*. Information Commissioner's Office. Available at: https://ico.org.uk/for-organisations/guide-to-data-protection/guide-to-the-general-data-protection-regulation-gdpr/ (Accessed: 10 September 2026).
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A. and Fung, P. (2023) 'Survey of Hallucination in Natural Language Generation', *ACM Computing Surveys*, 55(12), pp. 1–38.
- Jobscan (2025) *About Jobscan: ATS Resume Optimisation*. Available at: https://www.jobscan.co (Accessed: 23 September 2026).
- Levashina, J., Hartwell, C.J., Morgeson, F.P. and Campion, M.A. (2014) 'The Structured Employment Interview: Narrative and Quantitative Review of the Research Literature', *Personnel Psychology*, 67(1), pp. 241–293.
- National Association of Colleges and Employers (2023) *Student Survey Report 2023: Inequity in the Internship Market*. Bethlehem, PA: NACE.
- OWASP (2021) *OWASP Top 10:2021*. Open Web Application Security Project. Available at: https://owasp.org/Top10/ (Accessed: 10 September 2026).
- Pane, J.F., Steiner, E.D., Baird, M.D., Hamilton, L.S. and Pane, J.D. (2017) *How Does Personalized Learning Affect Student Achievement?* Santa Monica, CA: RAND Corporation.

- PostgreSQL Global Development Group (2024) *PostgreSQL 16 Documentation*. Available at: https://www.postgresql.org/docs/16/ (Accessed: 12 September 2026).
- Prisma (2024) *Prisma ORM Documentation*. Available at: https://www.prisma.io/docs (Accessed: 12 September 2026).
- Provos, N. and Mazières, D. (1999) 'A Future-Adaptable Password Scheme', *Proceedings of the USENIX Annual Technical Conference*. Freenix Track, pp. 81–86. Available at: https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf (Accessed: 10 September 2026).
- Raghavan, M., Barocas, S., Kleinberg, J. and Levy, K. (2020) 'Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices', *Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency*. Barcelona, Spain, pp. 469–481.
- React Team (2024) *React 19 Release Notes*. Available at: https://react.dev/blog (Accessed: 13 September 2026).
- Resume Worded (2025) *Resume Worded: AI Resume and LinkedIn Review*. Available at: https://resumeworded.com (Accessed: 23 September 2026).
- Rezi (2025) *Rezi: AI Resume Builder and Interview Practice*. Available at: https://www.rezi.ai (Accessed: 23 September 2026).
- Sanchez-Monedero, J., Dencik, L. and Edwards, L. (2020) 'What does it mean to "solve" the problem of discrimination in hiring? Social, technical and legal perspectives from the UK on automated hiring systems', *Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency*. Barcelona, Spain, pp. 458–468.
- Stapleton, J. (2003) *DSDM: Business Focused Development*. 2nd edn. London: Pearson Education.
- VanLehn, K. (2011) 'The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems', *Educational Psychologist*, 46(4), pp. 197–221.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention Is All You Need', *Advances in Neural Information Processing Systems*, 30, pp. 5998–6008.
- Vercel (2024) *Next.js Documentation*. Available at: https://nextjs.org/docs (Accessed: 11 September 2026).
- Voigt, P. and Von dem Bussche, A. (2017) *The EU General Data Protection Regulation (GDPR): A Practical Guide*. 1st edn. Cham: Springer International Publishing.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. and Zhou, D. (2022) 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models', *Advances in Neural Information Processing Systems*, 35, pp. 24824–24837.

- Nielsen, J. (1994) 'Enhancing the explanatory power of usability heuristics', *Proceedings of CHI '94*. New York: ACM, pp. 152–158.
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J. and Schmidt, D.C. (2023) 'A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT', *arXiv preprint arXiv:2302.11382*.

References (Chapter 1)

Barrick, M.R., Shaffer, J.A. and DeGrassi, S.W. (2009) 'What you see may not be what you get: Relationships among self-presentation tactics and ratings of interview and job performance', Journal of Applied Psychology, 94(6), pp. 1394–1411.

Bogen, M. and Rieke, A. (2018) Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias. Washington, DC: Upturn.

Cappelli, P. (2019) 'Your Approach to Hiring Is All Wrong', Harvard Business Review, 97(3), pp. 48–58.

Fuller, J.B., Raman, M., Sage-Gavin, E. and Hines, K. (2021) Hidden Workers: Untapped Talent. Boston, MA: Harvard Business School Publishing.

Jobscan (2025) About Jobscan: ATS Resume Optimisation. Available at: https://www.jobscan.co/about (Accessed: 10 September 2026).

Levashina, J., Hartwell, C.J., Morgeson, F.P. and Campion, M.A. (2014) 'The Structured Employment Interview: Narrative and Quantitative Review of the Research Literature', Personnel Psychology, 67(1), pp. 241–293.

National Association of Colleges and Employers (2023) Student Survey Report 2023. Bethlehem, PA: NACE.

References (Chapter 2)

Abrahamsson, P., Salo, O., Ronkainen, J. and Warsta, J. (2002) *Agile Software Development Methods: Review and Analysis*. Espoo: VTT Publications.

Barrick, M.R., Shaffer, J.A. and DeGrassi, S.W. (2009) 'What you see may not be what you get: Relationships among self-presentation tactics and ratings of interview and job performance', *Journal of Applied Psychology*, 94(6), pp. 1394–1411.

Black, J.S. and van Esch, P. (2020) 'AI-enabled recruiting: What is it and how should a manager use it?', *Business Horizons*, 63(2), pp. 215–226.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G. and Askell, A. (2020) 'Language Models are Few-Shot Learners', *Advances in Neural Information Processing Systems*, 33, pp. 1877–1901.

Cappelli, P. (2019) 'Your Approach to Hiring Is All Wrong', *Harvard Business Review*, 97(3), pp. 48–58.

Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', *Proceedings of NAACL-HLT 2019*. Minneapolis, MN, pp. 4171–4186.

Fuller, J.B., Raman, M., Sage-Gavin, E. and Hines, K. (2021) *Hidden Workers: Untapped Talent*. Boston, MA: Harvard Business School Publishing.
Google DeepMind (2024) *Gemini: A Family of Highly Capable Multimodal Models*. Technical Report. Mountain View, CA: Google.
Huffcutt, A.I. and Arthur, W. (1994) 'Hunter and Hunter (1984) Revisited: Interview Validity for Entry-Level Jobs', *Journal of Applied Psychology*, 79(2), pp. 184–190.
Jobscan (2025) *About Jobscan: ATS Resume Optimisation*. Available at: https://www.jobscan.co/about (Accessed: 10 September 2026).
Levashina, J., Hartwell, C.J., Morgeson, F.P. and Campion, M.A. (2014) 'The Structured Employment Interview: Narrative and Quantitative Review of the Research Literature', *Personnel Psychology*, 67(1), pp. 241–293.
Pane, J.F., Steiner, E.D., Baird, M.D., Hamilton, L.S. and Pane, J.D. (2017) *How Does Personalized Learning Affect Student Achievement?* Santa Monica, CA: RAND Corporation.
PostgreSQL Global Development Group (2024) *PostgreSQL 16 Documentation*. Available at: https://www.postgresql.org/docs/16/ (Accessed: 12 September 2026).
Prisma (2024) *Prisma ORM Documentation*. Available at: https://www.prisma.io/docs (Accessed: 12 September 2026).
Raghavan, M., Barocas, S., Kleinberg, J. and Levy, K. (2020) 'Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices', *Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency*. Barcelona, Spain, pp. 469–481.
React Team (2024) *React 19 Release Notes*. Available at: https://react.dev/blog (Accessed: 13 September 2026).
Rieke, A. and Bogen, M. (2018) *Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias*. Washington, DC: Upturn.
Sanchez-Monedero, J., Dencik, L. and Edwards, L. (2020) 'What does it mean to "solve" the problem of discrimination in hiring?', *Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency*. Barcelona, Spain, pp. 458–468.
Stapleton, J. (2003) *DSDM: Business Focused Development*. 2nd edn. London: Pearson Education.
VanLehn, K. (2011) 'The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems', *Educational Psychologist*, 46(4), pp. 197–221.
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention Is All You Need', *Advances in Neural Information Processing Systems*, 30, pp. 5998–6008.
Vercel (2024) *Next.js Documentation*. Available at: https://nextjs.org/docs (Accessed: 11 September 2026).
Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. and Zhou, D. (2022) 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models', *Advances in Neural Information Processing Systems*, 35, pp. 24824–24837.
White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J. and Schmidt, D.C. (2023) 'A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT', *arXiv preprint* arXiv:2302.11382.

CH 3

Huffcutt, A.I. and Arthur, W. (1994) 'Hunter and Hunter (1984) Revisited: Interview Validity for Entry-Level Jobs', *Journal of Applied Psychology*, 79(2), pp. 184–190.
Jobscan (2025) *Jobscan: Resume Optimisation and ATS Tools*. Available at: https://www.jobscan.co (Accessed: 23 September 2026).
Levashina, J., Hartwell, C.J., Morgeson, F.P. and Campion, M.A. (2014) 'The Structured Employment Interview: Narrative and Quantitative Review of the Research Literature', *Personnel Psychology*, 67(1), pp. 241–293.

National Association of Colleges and Employers (2023) *Student Survey Report 2023*. Bethlehem, PA: NACE.
Rezi (2025) *Rezi: AI Resume Builder and Interview Practice*. Available at: https://www.rezi.ai (Accessed: 23 September 2026).
Resume Worded (2025) *Resume Worded: AI Resume and LinkedIn Review*. Available at: https://resumeworded.com (Accessed: 23 September 2026).

CH 4

BCS (2022). BCS Code of Conduct. British Computer Society. https://www.bcs.org/membership-and-registrations/member-rules-and-policies/code-of-conduct/

Bogen, M., and Rieke, A. (2018). Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias. Upturn. https://www.upturn.org/work/help-wanted/

Fuller, J., Raman, M., Sage-Gavin, E., and Hines, K. (2021). Hidden Workers: Untapped Talent. Harvard Business School Project on Managing the Future of Work and Accenture. https://www.hbs.edu/managing-the-future-of-work/Documents/research/hiddenworkers09032021.pdf

ICO (2024). Guide to the General Data Protection Regulation (GDPR). Information Commissioner's Office. https://ico.org.uk/for-organisations/guide-to-data-protection/guide-to-the-general-data-protection-regulation-gdpr/

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., and Fung, P. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys. https://dl.acm.org/doi/10.1145/3571730

NACE (2023). Inequity in the Internship Market. National Association of Colleges and Employers.

OWASP (2021). OWASP Top 10:2021. Open Web Application Security Project. https://owasp.org/Top10/

Provos, N., and Mazières, D. (1999). A Future-Adaptable Password Scheme. USENIX Annual Technical Conference. https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf

Raghavan, M., Barocas, S., Kleinberg, J., and Levy, K. (2020). Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices. ACM Conference on Fairness, Accountability, and Transparency. https://dl.acm.org/doi/10.1145/3351095.3372828

Sanchez-Monedero, J., Dencik, L., and Edwards, L. (2020). What does it mean to 'solve' the problem of discrimination in hiring? Social, technical and legal perspectives from the UK on automated hiring systems. https://arxiv.org/abs/1910.06144

Voigt, P., and Von dem Bussche, A. (2017). The EU General Data Protection Regulation (GDPR): A Practical Guide. Springer.

-
- **CH 5**
- **Abrahamsson, P., Salo, O., Warsta, J., and Ronkainen, J. (2002).** *Agile Software Development Methods: Review and Analysis*. VTT Publications. https://publications.vtt.fi/pdf/publications/2002/P478.pdf
- **OWASP (2021).** *OWASP Top 10:2021*. Open Web Application Security Project. https://owasp.org/Top10/
- **Provos, N., and Mazières, D. (1999).** *A Future-Adaptable Password Scheme*. USENIX Annual Technical Conference. https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf
- **Stapleton, J. (2003).** *DSDM: Business Focused Development*. Pearson Education.